

ỨNG DỤNG MÔ HÌNH PCA, SVM, RANDOM FOREST, LOGISTIC REGRESSION VÀ CNN TRONG PHÂN LOẠI CƠ CHẾ NỘC ĐỘC RẮN

Có độc vs. Không độc - Độc máu vs. Độc thần kinh

GVDH: ThS. Trương Quang Nhật
SVTH: Vương Nguyễn Hồng Ngọc
MSSV: K234131494
Trường Đại học Kinh tế - Luật
(ĐHQG TP.HCM)

**HEMOTOXIC
VENOM**

**NEUROTOXIC
VENOM**

KHOA TOÁN KINH TẾ



HỌC PHẦN: GIỚI THIỆU VỀ HỌC MÁY

ĐỀ TÀI

**ỨNG DỤNG MÔ HÌNH PCA, SVM, RANDOM FOREST,
LOGISTIC REGRESSION VÀ CNN TRONG
PHÂN LOẠI CƠ CHẾ NỘC ĐỘC RẮN**

GVHD: ThS. Trương Quang Nhật

SVTH: Vương Nguyễn Hồng Ngọc

MSSV: K234131494

LỜI CẢM ƠN

Thành quả của bài đồ án này là sự đúc kết từ quá trình tiếp thu kiến thức và sự hỗ trợ quý báu mà em may mắn nhận được trong suốt thời gian qua. Em xin trân trọng gửi lời tri ân sâu sắc nhất đến giảng viên hướng dẫn - ThS. Trương Quang Nhật. Trong suốt thời gian thực hiện đề tài ứng dụng mô hình PCA, SVM, Random Forest, Logistic Regression và CNN trong phân loại cơ chế nọc độc rắn, thầy đã luôn tận tình dìu dắt và giúp em xác định rõ những khía cạnh trọng tâm cần giải quyết. Chính những phản hồi chuyên môn sắc bén cùng sự định hướng chi tiết từ thầy là yếu tố mang tính quyết định, giúp em có cơ hội khắc phục thiếu sót, tinh chỉnh nghiên cứu và đạt được kết quả hoàn thiện nhất.

Bên cạnh đó, em cũng xin bày tỏ lòng biết ơn đến các nhà khoa học và tác giả của những công trình, bộ dữ liệu được trích dẫn trong báo cáo. Những tài liệu đã xuất bản của họ đóng vai trò là nguồn tài nguyên vô giá, mang lại cho em những góc nhìn sâu sắc và toàn diện hơn trong suốt quá trình phân tích và xây dựng mô hình.

Trân trọng



Vương Nguyễn Hồng Ngọc

LỜI CAM ĐOAN

Bằng văn bản này, em xin khẳng định toàn bộ nội dung được trình bày trong đồ án này là thành quả từ quá trình tự tìm hiểu và nghiên cứu độc lập, nghiêm túc. Công trình này hoàn toàn mang tính nguyên bản. Mọi thông tin, dữ liệu học thuật được kế thừa từ các nguồn tài liệu bên ngoài đều đã được em ghi nhận, chú thích và trích dẫn minh bạch, tuân thủ đúng các quy chuẩn về liêm chính học thuật.

TP. HCM, ngày 29 tháng 03 năm 2026

Sinh viên thực hiện



Vương Nguyễn Hồng Ngọc

MỤC LỤC NỘI DUNG

LỜI CẢM ƠN.....	i
LỜI CAM ĐOAN.....	ii
CHƯƠNG 1: MỞ ĐẦU	1
1.1 Bối cảnh và lý do chọn đề tài.....	1
1.2 Mục tiêu nghiên cứu.....	5
1.2.1 Mục tiêu tổng quát.....	5
1.2.2 Mục tiêu cụ thể.....	5
1.3 Đối tượng và phạm vi nghiên cứu.....	5
1.3.1 Đối tượng nghiên cứu:	5
1.3.2 Phạm vi nghiên cứu:	6
1.4 Phương pháp nghiên cứu.....	6
1.5 Cấu trúc đồ án	7
CHƯƠNG 2: GIỚI THIỆU BÀI TOÁN VÀ DỮ LIỆU	7
2.1 Giới thiệu bài toán phân loại rắn độc.....	7
2.1.1 Phát biểu bài toán: Phân loại nhị phân	7
2.1.2 Các mô hình được sử dụng	7
2.1.3 Lý do lựa chọn mô hình	8
2.2 Giới thiệu về bộ dữ liệu (Dataset)	9
2.2.1 Nguồn gốc và quy mô dữ liệu	9
2.2.1.1 Nguồn gốc dữ liệu.....	9
2.2.1.2 Quy mô và chiến lược gán nhãn phân cấp.....	10
2.2.2 Đặc trưng sinh học của các class trong tập dữ liệu	11
2.2.2.1 Nhóm rắn không độc	11
2.2.2.2 Nhóm rắn độc	18
2.2.2.2.1 Độc máu (Hemotoxic).....	18
2.2.2.2.2 Độc thần kinh (neurotoxin)	21
CHƯƠNG 3: GIỚI THIỆU THUẬT TOÁN VÀ MINH HỌA PYTHON.....	23
3.1 Cơ sở lý thuyết về các thuật toán Học máy đã sử dụng.....	23
3.1.1 Phân tích thành phần chính (Principal Component Analysis - PCA)	23
3.1.2 Máy học Vector hỗ trợ (Support Vector Machine - SVM)	26
3.1.2.1 The separating hyperplane.....	27

3.1.2.2 The maximum-margin hyperplane	28
3.1.2.3 The soft margin	29
3.1.2.4 The kernel function.....	29
3.1.3 Rừng ngẫu nhiên (Random Forest)	31
3.1.4 Hồi quy Logistic (Logistic Regression)	31
3.1.5 Mạng nơ-ron tích chập (Convolutional Neural Networks - CNN).....	33
3.1.6 Label encoding	35
3.1.7 Confusion matrix	37
3.1.8 Khái niệm Precision.....	38
3.1.9 Khái niệm Recall.....	38
3.2 Minh hoạ bằng Python.....	40
3.2.1 Xử lý ảnh lỗi, nhiễu và định dạng lại kích thước	40
3.2.2 Giảm chiều dữ liệu bằng PCA.....	42
3.2.3 Thực hiện bài toán phân loại bằng SVM.....	42
3.2.4 Đánh giá mô hình SVM.....	43
3.2.5 Thực hiện bài toán phân loại bằng random forest	48
3.2.6 Thực hiện bài toán phân loại bằng Logistic Regression	51
3.2.7 Thực hiện bài toán phân loại các loại độc bằng CNN	73
3.3 So sánh hiệu năng giữa các mô hình (CNN vs. SVM/RF/LR).....	81
CHƯƠNG 4: MỞ RỘNG	82
CHƯƠNG 5: KẾT LUẬN	82
TÀI LIỆU THAM KHẢO	84

DANH MỤC HÌNH ẢNH

Hình 1: Giả định tình huống gặp rắn	1
Hình 2: Sai lầm trong sơ cứu rắn cắn dẫn đến biến chứng nguy hiểm	2
Hình 3: Thông tin sai lệch về rắn hổ trĩ lan truyền trên mạng xã hội	3
Hình 4: Hộp sọ của một con rắn taipan và các mặt cắt của răng nanh bên trái của nó cho thấy mối quan hệ giữa rãnh nọc độc và sự phát triển ở chân răng. Ảnh: A.Palci from Flinders University .	3
Hình 5: Nhận diện sai	4
Hình 6: Rắn lục núi	4
Hình 8: Sơ đồ phân cấp nhãn dữ liệu rắn độc và rắn không độc	10
Hình 8: So sánh phân bố dữ liệu: Train vs Test.....	10
Hình 9: Phân bố số lượng ảnh theo loại nọc độc (Cấp độ 2)	11
Hình 10: Rắn chuột miền Đông	11
Hình 11: Trinket Snake	12
Hình 12: Rắn nước sọc.....	12
Hình 13: Green vine snake.....	13
Hình 14: Trăn Ấn Độ	13
Hình 15: Banded Racer	14
Hình 16: Rắn chuột đen	14
Hình 17: Trăn cát	15
Hình 18: Rắn hoa cỏ sọc	15
Hình 19: Rough-scaled sand boa	16
Hình 20: Red Sand Boa.....	16
Hình 21: Aesculapian snake.....	17
Hình 22: Rắn ráo trâu.....	17
Hình 23: Gulf Hammock Rat Snake	18
Hình 24: Russell's Viper.....	18
Hình 25: Saw Scaled Viper.....	19
Hình 26: Deinagkistrodon acutus.....	20
Hình 27: Daboia siamensis	20
Hình 28: Hổ mang Ấn Độ.....	21
Hình 29: Hổ mang một mắt kính	21
Hình 30: Rắn hổ chúa	22

Hình 31: Rắn cạp nia.....	23
Hình 32: Ý tưởng PCA	24
Hình 33: Phân tích thành phần chính (PCA) trên hình ảnh	24
Hình 34: Phương pháp PCA ở góc nhìn đại số.....	25
Hình 35: Sử dụng PCA trong việc nén hình ảnh	25
Hình 36: Biểu đồ PCA 2 chiều	26
Hình 37: Trực quan hóa tập dữ liệu chữ số 3 qua không gian PCA	26
Hình 38: Ví dụ a.....	27
Hình 39: Ví dụ b	28
Hình 40: Ví dụ g Tập dữ liệu chứa một 'lỗi', là hồ sơ biểu hiện gen được khoan tròn	29
Hình 41: Ví dụ i	29
Hình 42: Ví dụ j	30
Hình 43: Ví dụ k Hình 44: Ví dụ l.....	30
Hình 45: Sơ đồ mặt lưng của não mèo, thể hiện các điểm đi vào của 45 lần chọc vi điện cực. Các vết chọc nằm giữa các đường nét đứt là những vị trí mà các tế bào có trường cảm thụ nằm trong hoặc gần vùng trung tâm võng mạc	33
Hình 46: Nguyên lý sinh học của Hubel và Wiesel.....	34
Hình 47: Một ví dụ về các mối liên kết giữa các tế bào và phản hồi của các tế bào sau khi hoàn thành quá trình tự tổ chức.	34
Hình 48: Sơ đồ mô phỏng cơ chế trích xuất đặc trưng cục bộ và xử lý phân tầng trong kiến trúc mạng CNN	35
Hình 49: Label encoding.....	36
Hình 50: https://viblo.asia/p/tim-hieu-ve-confusion-matrix-trong-machine-learning-Az45bRpo5xY	37
Hình 51: https://miai.vn/2020/06/16/oanh-gia-model-ai-theo-cach-mi-an-lien-chuong-2-precision-recall-va-f-score/	37
Hình 52: Minh họa AUC.....	39
Hình 53: Thông tin cấu hình của mô hình	42
Hình 54: Kết quả đầu ra cho thấy mô hình đã được khởi tạo thành công	43
Hình 55: Ma trận nhầm lẫn: Venomous vs Non-venomous	44
Hình 56: Đường cong ROC	45
Hình 57: Kiểm tra 10 mẫu test.....	47

CHƯƠNG 1: MỞ ĐẦU

1.1 Bối cảnh và lý do chọn đề tài

Đặt ra tình huống giả định: Một người vừa bị rắn cắn vào chân ở khu vực cách xa trung tâm y tế.



Hình 1: Giả định tình huống gặp rắn

Giữa sự hoảng loạn tột độ, cơn đau nhói truyền đến từ vết thương và những mảnh ký ức chập vá từ các bộ phim sinh tồn, một câu hỏi mang tính sống còn ngay lập tức được đặt ra trong tâm trí người này: *Liệu con rắn vừa cắn tôi có độc hay không?* Đối mặt với tình thế này, nạn nhân và người sơ cứu thường bị đẩy vào một ván cược rủi ro. Theo số liệu thống kê từ Tổ chức Y tế Thế giới WHO, ước tính mỗi năm trên khắp thế giới, khoảng 5.4 triệu người bị rắn cắn và có đến 1.8 đến 2.7 triệu ca bị nhiễm độc. Việc không thể nhận diện được loài rắn khiến họ không biết nên xử lý vết thương như một vết cắn thông thường, hay phải áp dụng các biện pháp khẩn cấp. Nếu chẩn đoán sai hoặc chần chừ, nọc độc sẽ ngấm sâu, dẫn đến các biến chứng hoại tử nghiêm trọng. Hậu quả là nạn nhân có thể buộc phải đối mặt với quyết định tàn khốc: Phẫu thuật cắt bỏ chi để bảo toàn tính mạng, hoặc tệ nhất là tử vong.

Trong điều kiện hoảng loạn và thiếu chuyên môn thì trí nhớ không phải là bộ lọc thông tin đáng tin cậy để miêu tả lại hình dáng con rắn cho bác sĩ. Việc “đoán mò” xem rắn có độc hay không dựa trên kinh nghiệm dân gian mang lại rủi ro y tế quá lớn, không thể chấp nhận được trong công tác cấp cứu hiện đại.

Thực trạng này càng trở nên nhức nhối tại Việt Nam. Sở hữu đặc trưng khí hậu đa dạng cùng hệ sinh thái phong phú trải dài từ miền núi xuống đồng bằng, Việt Nam trở thành môi trường sinh sống lý tưởng cho vô số quần thể động vật, trong đó ghi nhận sự hiện diện của hơn 50 loài rắn độc. Trong nhiều trường hợp, dù nạn nhân nhận thức được việc bị rắn cắn, sự thiếu hụt kiến thức phân loại nọc độc vẫn dẫn đến các sai lầm nghiêm trọng trong công tác sơ cứu. Đáng lo ngại hơn, tâm lý hoảng loạn thường đẩy nạn nhân tìm đến chủ trương “có bệnh thì vái tứ phương” trù phương pháp khoa học.

Tự đắp lá chữa rắn cắn, nam thanh niên bị nhiễm độc nặng



ĐỨC CƯỜNG

▶ Nghe đọc bài 1:39

1.0



Nữ miền Nam

Tuổi Trẻ trên Google News

Sau khi bị rắn cắn, nam thanh niên 27 tuổi ở Khánh Hòa đã tự đắp lá cây theo mẹo dân gian khiến tình trạng nhiễm độc trở nên nghiêm trọng, phải nhập viện cấp cứu.



Bàn tay bị rắn cắn của bệnh nhân A. bị sưng phù, chuyển đen, có dấu hiệu hoại tử - Ảnh: Bệnh viện Đa khoa Ninh Thuận cung cấp

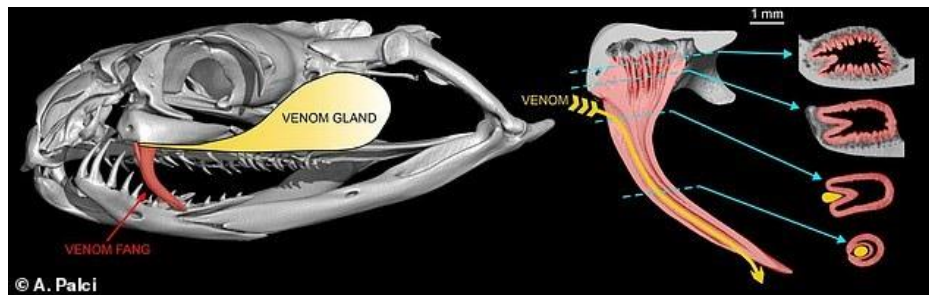
Hình 2: Sai lầm trong sơ cứu rắn cắn dẫn đến biến chứng nguy hiểm

Thực trạng này đang bị làm trầm trọng thêm bởi sự lan truyền thông tin sai lệch trên mạng xã hội. Điển hình như trường hợp của một trang cá nhân mang tên “Đức Sơn Hải Dương” với hơn 1.2 triệu lượt theo dõi, thường xuyên đăng tải hàng ngàn bài thuốc cổ truyền chữa rắn cắn. Mặc dù được một bộ phận dư luận tung hô vì hoạt động “chữa bệnh miễn phí”, các phương pháp này tiềm ẩn rủi ro y tế cực kỳ lớn do thiếu cơ sở khoa học và thường xuyên đánh tráo khái niệm sinh học. Dưới góc độ giải phẫu học, nhiều thông tin được các cá nhân này lan truyền là hoàn toàn phản khoa học.



Hình 3: Thông tin sai lệch về rắn hổ trỉ lan truyền trên mạng xã hội

Cùng xem qua bài đăng này, quan niệm cho rằng nọc độc có thể lây nhiễm qua đường nước bọt do rắn nhai trúng tuyến độc của con mồi là sai lệch cơ bản về tập tính sinh học bởi rắn không nhai nên khả năng làm vỡ tuyến độc của con rắn khác để dính vào nước dãi hầu như rất thấp. Các loài rắn độc sở hữu cấu tạo răng nanh dạng ống kim tiêm chuyên biệt, liên kết trực tiếp với tuyến nọc để bơm độc chất thẳng vào cơ thể nạn nhân.



Hình 4: Hộp sọ của một con rắn taipan và các mặt cắt của răng nanh bên trái của nó cho thấy mối quan hệ giữa rãnh nọc độc và sự phát triển ở chân răng. Ảnh: A.Palci from Flinders University

Theo mô phỏng 3D của Triep và cộng sự (2013), sự co thắt của bó cơ *musculus adductor mandibulae externus superficialis* tạo ra áp suất dương cực lớn, là lực đẩy duy nhất tổng xuất nọc độc. Dưới áp lực này, nọc độc bị ép lao qua vi ống trong răng nanh ra ngoài với vận tốc cực cao. Các tạp chất bên ngoài hoàn toàn không thể ngược dòng áp suất để xâm nhập ngược vào hệ thống tuyến nọc khép kín. Sự nguy hiểm của các phương pháp tự phát này còn nằm ở việc người thực hiện không có khả năng nhận diện sinh học cơ bản. Chủ tài khoản trên đã đăng tải một bài post chẩn đoán sai một loài vô hại thành một loài có độc nguy hiểm.



Hình 5: Nhận diện sai

Thực tế, đây là rắn rằn đốm (Tên latin: *Boiga multomaculata*) - thuộc nhóm vô hại.



Hình 6: Rắn lục núi

Việc giao phó tính mạng cho những cá nhân thiếu kiến thức nền tảng về định danh phân loài là một sự đánh cược vô cùng nguy hiểm đối với bệnh nhân cần sự hỗ trợ khẩn cấp.

Nhằm nâng cao nhận thức về rắn, bảo vệ sự đa dạng sinh học, giải quyết triệt để sự phán đoán cảm tính và loại bỏ hoàn toàn những “ván cược” sinh tử nêu trên, đồ án này đề xuất giải pháp ứng dụng Machine Learning, trí tuệ nhân tạo AI và Thị giác máy tính để nhận diện tự động qua hình ảnh. Việc huấn luyện sẽ được áp dụng trên tập dữ liệu phong phú về các loài rắn độc và không độc được thu thập tại Ấn Độ, một trong những quốc gia có tính đa dạng sinh học về rắn và tỷ lệ tai nạn do rắn cắn cao (thuộc một trong những quốc gia bị ảnh hưởng nặng nề nhất bởi nạn rắn cắn). Mô hình sẽ được huấn luyện với kỳ vọng trở thành công cụ đắc lực, hỗ trợ đưa ra kết quả phân loại ngay lập tức với độ chính xác cao, giúp đưa ra hỗ trợ ra quyết định sơ cứu kịp thời và ngăn chặn các kết cục xấu nhất cho nạn nhân.

1.2 Mục tiêu nghiên cứu

1.2.1 Mục tiêu tổng quát

Nghiên cứu và xây dựng một hệ thống phân loại tự động khả năng mang nọc độc của rắn thông qua dữ liệu hình ảnh. Hệ thống không chỉ dừng lại ở việc nhận diện ranh giới cơ bản giữa rắn có độc và không có độc, mà còn tiến sâu vào việc phân định cơ chế gây độc đặc thù: nọc độc máu (Hemotoxic) và nọc độc thần kinh (Neurotoxic). Qua đó, đề xuất mô hình Học máy/Học sâu tối ưu nhất làm cơ sở phát triển công cụ hỗ trợ sơ cứu y tế khẩn cấp, góp phần giảm thiểu tỷ lệ tử vong và kết cục tồi tệ.

1.2.2 Mục tiêu cụ thể

Để đi đến mục tiêu tổng quát, đồ án sẽ tập trung thực hiện một cách cụ thể hoá các mục tiêu sau:

- Tiền xử lý và gán nhãn dữ liệu: Làm sạch và chuẩn hóa tập dữ liệu ảnh các loài rắn tại Ấn Độ. Phân loại và gán nhãn tập dữ liệu thành hai nhóm chính yếu: Rắn không độc (Non-venomous), Rắn độc (venomous). Từ kết quả của việc phân loại độc - không độc, tiếp tục tiến hành phân rã và gán nhãn chi tiết dựa trên cơ chế nọc độc đặc thù, bao gồm: Độc máu (Hemotoxic) và Độc thần kinh (Neurotoxic).
- Trích xuất đặc trưng và tối ưu hóa: Ứng dụng thuật toán phân tích thành phần chính (PCA) để thực hiện giảm chiều dữ liệu, giúp loại bỏ các thông tin dư thừa, chỉ giữ lại những đặc trưng hình thái cốt lõi và quan trọng nhất của đối tượng.
- Xây dựng và huấn luyện mô hình phân loại: Xây dựng, tinh chỉnh và triển khai các thuật toán nhằm giải quyết toàn diện bài toán phân loại nhị phân. Các phương pháp được lựa chọn bao gồm hai nhóm chính:
 - + Nhóm học máy truyền thống: Support Vector Machine, Random Forest và Logistic Regression.
 - + Nhóm học sâu: Mô hình mạng nơ-ron tích chập (CNN)
- Đánh giá và so sánh hiệu suất: Kiểm thử các mô hình đã huấn luyện trên tập dữ liệu đánh giá độc lập. Đo lường và đối chiếu năng lực nhận diện thông qua các hệ số đánh giá chuẩn mực: Độ chính xác - Accuracy, Độ chụm - Precision, Độ phủ - Recall và F1-Score.
- Đề xuất giải pháp tối ưu: Phân tích ưu, nhược điểm của từng thuật toán, từ đó xác định mô hình mang lại hiệu năng cao nhất trong việc phân định cơ chế nọc độc làm tiền đề tích hợp cho các ứng dụng thực tế.

1.3 Đối tượng và phạm vi nghiên cứu

1.3.1 Đối tượng nghiên cứu:

Đồ án tập trung nghiên cứu việc ứng dụng các thuật toán học máy và học sâu vào bài toán xử lý và phân loại hình ảnh sinh học. Đối tượng phân tích cụ thể bao gồm:

- Tập dữ liệu hình ảnh: Các hình ảnh thu thập về các loài rắn phổ biến tại khu vực Ấn Độ, mang các đặc trưng hình thái khác biệt.
- Hệ thống phân loại đặc tính sinh học: Cơ chế gán nhãn dữ liệu theo hai cấp độ: Rắn độc (Venomous) - Không độc (Non Venomous); và Độc máu (Hemotoxic) - Độc thần kinh (Neurotoxic).

- Các mô hình phân loại: Cơ sở lý thuyết và kiến trúc của các thuật toán Mạng nơ-ron tích chập, Support Vector Machine, Random Forest và Logistic Regression kết hợp với kỹ thuật giảm chiều dữ liệu PCA.

1.3.2 Phạm vi nghiên cứu:

Giới hạn của đề án được khoanh vùng trong các phạm vi cụ thể sau:

- Phạm vi địa lý: Dữ liệu hình ảnh chỉ tập trung vào các loài rắn phân bố tại tiểu lục địa Ấn Độ, không mở rộng ra các khu vực địa lý khác trên thế giới.

- Phạm vi dữ liệu thực nghiệm: Đề tài sử dụng hai bộ dữ liệu hình ảnh cốt lõi:

+ Bộ dữ liệu cấp độ 1 (Có độc / Không độc): Tổng quy mô 2.044 hình ảnh. Trong đó, tập huấn luyện gồm 1.775 ảnh (715 ảnh không độc, 1.060 ảnh có độc) và tập kiểm thử gồm 269 ảnh (128 ảnh không độc, 141 ảnh có độc).

+ Bộ dữ liệu cấp độ 2 (Độc máu / Độc thần kinh): Được trích xuất từ tập huấn luyện của bộ dữ liệu đầu tiên, tập trung vào nhóm rắn có độc với quy mô 996 hình ảnh (396 ảnh Độc máu và 600 ảnh Độc thần kinh). Tập dữ liệu này tiếp tục được chia theo tỷ lệ 80% cho huấn luyện và 20% cho đánh giá.

- Phạm vi thuật toán: Áp dụng các kiến trúc cơ bản của nhóm Học máy truyền thống và Mạng CNN tiêu chuẩn để giải quyết bài toán phân loại hình ảnh. Đề tài không mở rộng sang các bài toán phức tạp hơn như phát hiện đối tượng hay phân vùng hình ảnh.

- Phạm vi đánh giá: Chất lượng của mô hình được đánh giá hoàn toàn qua các độ đo tiêu chuẩn của bài toán phân loại (Accuracy, Precision, Recall, F1-Score) trên tập dữ liệu kiểm thử.

1.4 Phương pháp nghiên cứu

Để đạt được mục tiêu phân loại các loài rắn có độc/không độc và phân loại loại nọc độc, nghiên cứu này áp dụng phương pháp nghiên cứu chủ yếu như sau:

Phương pháp nghiên cứu lý thuyết: Giai đoạn này tập trung vào việc thu thập, chọn lọc và tổng hợp các tài liệu sinh học về đặc trưng hình thái của các loài rắn tại Ấn Độ, cũng như cơ chế tác động của nọc độc (Độc máu - Hemotoxic, Độc thần kinh - Neurotoxic). Đồng thời, nghiên cứu nền tảng lý luận về các thuật toán Học máy, Học sâu và kỹ thuật xử lý ảnh, đặc biệt là phương pháp giảm chiều dữ liệu PCA. Phần lý thuyết này đóng vai trò nền tảng định hướng cho quá trình hiểu rõ dữ liệu đầu vào và thiết kế kiến trúc mô hình phù hợp.

Phương pháp thực nghiệm - mô hình hóa: Xây dựng quy trình xử lý dữ liệu thực tế bằng ngôn ngữ lập trình Python. Quá trình thực nghiệm gồm các bước: tiền xử lý dữ liệu ảnh (làm sạch, chuẩn hóa kích thước, gán nhãn), trích xuất đặc trưng bằng PCA và chia tập dữ liệu (Train/Test). Sau đó, tiến hành xây dựng và huấn luyện các mô hình phân loại từ cơ bản đến phức tạp, bao gồm nhóm Học máy (SVM, Random Forest, Logistic Regression) và Mạng nơ-ron tích chập (CNN). Ghi nhận toàn bộ thông số học tập (loss, accuracy) qua các vòng lặp để quan sát quá trình hội tụ của mô hình.

Phương pháp đánh giá - so sánh: Kiểm thử các mô hình đã huấn luyện trên tập dữ liệu đánh giá độc lập. Sử dụng Ma trận nhầm lẫn và các thang đo chuẩn mực bao gồm Accuracy, Precision, Recall và F1-Score để đo lường độ tin cậy. Dựa trên kết quả định lượng, tiến hành đối chiếu và so

sánh chéo hiệu năng giữa kiến trúc Học sâu (CNN) và nhóm Học máy truyền thống nhằm đánh giá tính ổn định, từ đó rút ra kết luận về mô hình tối ưu nhất cho bài toán.

1.5 Cấu trúc đồ án

Nội dung đồ án được tổ chức và trình bày một cách hệ thống thông qua 5 chương cốt lõi:

- *Chương 1 - Mở đầu:* Khái quát bối cảnh, lý do lựa chọn đề tài. Định hình mục tiêu, phạm vi và phương pháp nghiên cứu.
- *Chương 2 - Giới thiệu bài toán và dữ liệu:* Phát biểu bài toán xử lý ảnh phân loại nhị phân. Phân tích sâu bộ dữ liệu 2.044 ảnh, bao gồm nguồn gốc, quy mô, tính phân phối và bóc tách các đặc trưng thị giác của các loài rắn bản địa Ấn Độ.
- *Chương 3 - Giới thiệu thuật toán và minh họa bằng Python:* Trình bày cơ sở lý thuyết của các mô hình phân loại SVM, Random Forest, Logistic regression, CNN và thuật toán PCA. Triển khai thực nghiệm trên Python từ bước tiền xử lý ảnh, huấn luyện mô hình cho đến khi đánh giá hiệu suất tổng thể.
- *Chương 4 - Mở rộng:* Đề xuất các giải pháp tối ưu hóa kiến trúc mạng nơ-ron trong tương lai và phân thảo tiềm năng tích hợp thuật toán vào các thiết bị di động, hướng tới xây dựng một hệ thống nhận diện thông minh theo thời gian thực
- *Chương 5 - Kết luận:* Tóm lược những giá trị thực tiễn và kết quả định lượng. Nhìn nhận những hạn chế còn tồn đọng và gợi ý các hướng nghiên cứu sâu hơn.

CHƯƠNG 2: GIỚI THIỆU BÀI TOÁN VÀ DỮ LIỆU

2.1 Giới thiệu bài toán phân loại rắn độc

2.1.1 Phát biểu bài toán: Phân loại nhị phân

Trong lĩnh vực trí tuệ nhân tạo và y tế, việc phân loại nhanh chóng và chính xác các loài sinh vật có nọc độc là một bài toán mang tính sống còn. Đặc biệt trong thực tế, hình ảnh các loài rắn thường mang đặc điểm phi tuyến tính cao, bị nhiễu bởi môi trường tự nhiên như ánh sáng, góc chụp, bối cảnh lá cây, đất đá,... và có sự nguy trang sinh học phức tạp. Các phương pháp nhận diện thủ công bằng mắt thường hoặc hệ thống phân loại dựa trên quy tắc theo cách truyền thống rất dễ bị đánh lừa, dẫn đến sai số chết người trong các tình huống sơ cứu khẩn cấp. Để vượt qua giới hạn đó, nhóm các phương pháp thị giác máy tính kết hợp học máy và Học sâu đã được áp dụng nhằm tự động hóa quá trình trích xuất đặc trưng và đưa ra quyết định một cách chuẩn xác nhất. Bài toán được đặt ra ở đây là bài toán Phân loại nhị phân dựa trên dữ liệu hình ảnh. Mục tiêu của đồ án là phân tích một bức ảnh đầu vào chứa thực thể rắn và tự động dự đoán, phân bổ nó vào đúng một trong hai lớp rời rạc: nhóm “Có độc” (Venomous) hoặc nhóm “Không độc” (Non Venomous). Rồi tiếp tục phân loại nhóm “Có độc” vào độc máu (hemotoxic) hoặc độc thần kinh (neurotoxic)

2.1.2 Các mô hình được sử dụng

Trong việc giải quyết bài toán phân loại hình ảnh sinh học phức tạp, đồ án tiếp cận theo hai luồng phương pháp song song: Học sâu cùng Mạng nơ-ron tích chập (CNN) và Học máy truyền thống gồm các thuật toán như SVM, Random Forest, Logistic Regression kết hợp thuật toán Phân tích thành phần chính (PCA).

2.1.3 Lý do lựa chọn mô hình

Hình ảnh rắn trong tự nhiên thường mang đặc điểm phi tuyến tính cao, bị nhiễu bởi môi trường (bối cảnh, ánh sáng, góc chụp) và sự nguy trang sinh học. Trong một tài liệu nghiên cứu về thị giác máy tính ứng dụng cho cánh tay robot gấp đồ vật, Citron và cộng sự, 2025 đã chứng minh rằng kiến trúc Mạng nơ-ron tích chập (CNN) hoạt động cực kỳ ổn định và vượt trội so với nhiều mô hình khác khi phải xử lý các khung hình có độ che khuất và nhiễu cảnh nền cao. Sự tương đồng về thách thức thị giác này chính là cơ sở khoa học vững chắc để đề án lựa chọn CNN làm mô hình cốt lõi, giúp hệ thống tự động bóc tách các đặc trưng của rắn mà không bị đánh lừa bởi môi trường tự nhiên.

Việc sử dụng thuật toán Phân tích thành phần chính (PCA) mang lại những lợi ích vượt trội, đóng vai trò là một công cụ tối ưu hóa không thể thiếu trong cả phân tích dữ liệu sinh thái lẫn các bài toán thị giác máy tính. Dưới góc độ phân tích môi trường, các phương pháp truyền thống thường bỏ sót bản chất đa biến của các quần xã sinh vật khi phản ứng với biến đổi môi trường. PCA giải quyết vấn đề này bằng cách xem xét đồng thời nhiều loài cùng các mô hình đồng biến thiên của chúng. Khi được ứng dụng vào bài toán nhận diện hình ảnh (như phân loại ảnh rắn), PCA thể hiện sức mạnh vượt trội khi thực hiện các phép kết hợp tuyến tính trên các biến ban đầu, giúp biến đổi dữ liệu phức tạp thành dạng dễ hình dung hơn, từ đó làm nổi bật các mô hình và xu hướng biến động tương đối của sinh vật. Thay vì phải xử lý trực tiếp không gian khổng lồ chứa hàng vạn điểm ảnh thô, PCA hoạt động như một bộ lọc tự nhiên giúp tự động loại bỏ các thông tin dư thừa, nhiễu loạn từ môi trường (như bối cảnh, nền đất, ánh sáng) và chỉ giữ lại một tập hợp nhỏ các thành phần chính biểu diễn cốt lõi sự biến thiên của bức ảnh. Theo hướng tiếp cận thiết kế thực nghiệm trong nghiên cứu của Rimal và cộng sự, 2025, việc lựa chọn đồng thời Logistic Regression (LR), Support Vector Machine (SVM) và Random Forest (RF) tạo ra một khung phân tích so sánh toàn diện, bao quát ba trường phái cốt lõi của Học máy:

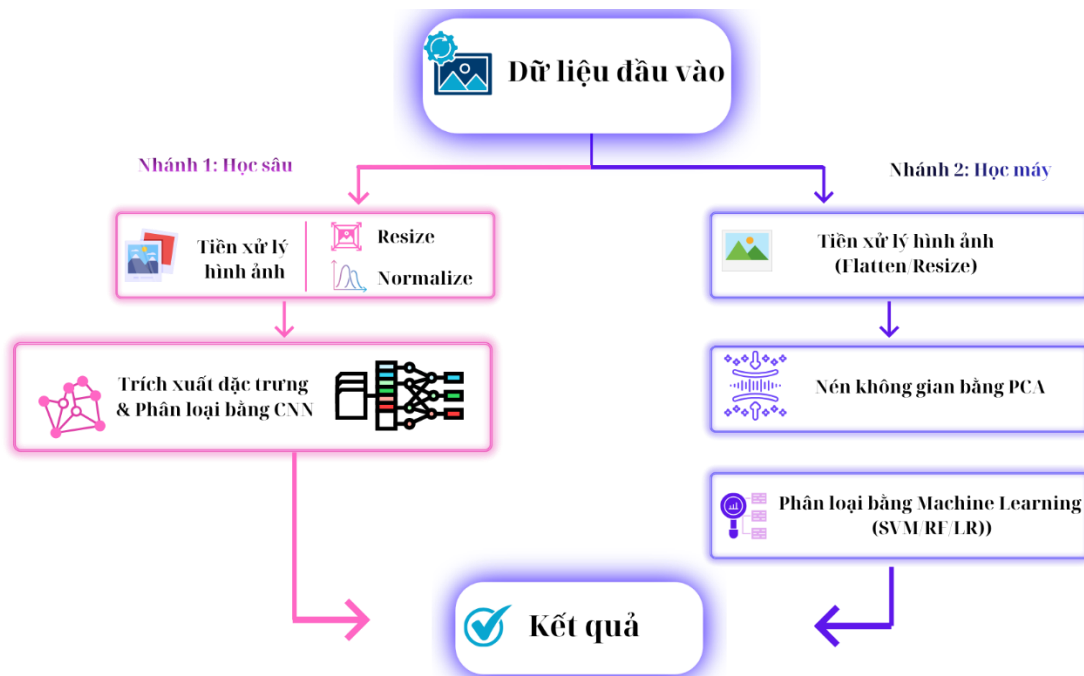
Logistic Regression là một thuật toán thống kê chuẩn mực dành riêng cho bài toán phân loại nhị phân. Điểm mạnh của Logistic Regression là sự đơn giản, tốc độ tính toán nhanh và khả năng cung cấp xác suất dự đoán rõ ràng. Trong quy trình nghiên cứu, Logistic Regression đóng vai trò là mô hình cơ sở lý tưởng; việc đánh giá hiệu suất của Logistic Regression giúp hệ thống xác định xem các đặc trưng (features) sau khi qua PCA có tính phân tách tuyến tính tốt hay không trước khi áp dụng các thuật toán phức tạp hơn.

Support Vector Machine tiếp cận bài toán dưới góc độ hình học không gian, hoạt động bằng cách tìm kiếm một siêu phẳng tối ưu để phân tách các lớp dữ liệu với khoảng cách biên lớn nhất. Thuật toán này đặc biệt tốt và giữ được độ ổn định cao khi xử lý dữ liệu trong không gian đa chiều. Đối với ranh giới sinh học phức tạp giữa rắn độc và không độc, khả năng sử dụng các hàm nhân kernel của SVM giúp mô hình xử lý tốt các vùng dữ liệu giao thoa phi tuyến tính.

Random Forest là đại diện tiêu biểu của phương pháp học tổng hợp, hoạt động bằng cách xây dựng và kết hợp dự đoán từ hàng trăm cây quyết định ngẫu nhiên. Cấu trúc này giúp Random Forest có khả năng chịu lỗi ổn định đối với dữ liệu nhiễu và các điểm ngoại lai - những yếu tố rất thường gặp trong hình ảnh động vật hoang dã. Hơn nữa, Random Forest khắc phục triệt để hiện tượng quá khớp, đảm bảo mô hình có khả năng tổng quát hóa cao trên dữ liệu thực tế.

2.1.3 Sơ đồ quy trình giải quyết bài toán

Để giải quyết bài toán trên, đề án đề xuất quy trình thực hiện song song hai luồng thuật toán được minh họa qua sơ đồ sau



Hình 7: Sơ đồ quy trình xử lý và phân loại hình ảnh

2.2 Giới thiệu về bộ dữ liệu (Dataset)

2.2.1 Nguồn gốc và quy mô dữ liệu

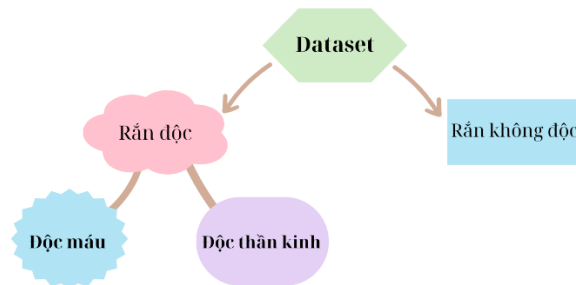
Để phục vụ cho bài toán phân loại hình ảnh sinh vật có nọc độc, đề án sử dụng dữ liệu hình ảnh kỹ thuật số tập trung vào các loài rắn phân bố đặc thù tại khu vực tiểu lục địa Ấn Độ. Chất lượng, quy mô và cách thức tổ chức dữ liệu đóng vai trò tiên quyết đến khả năng học đặc trưng của các mô hình Học sâu và Học máy.

2.2.1.1 Nguồn gốc dữ liệu

Tập dữ liệu hình ảnh được kế thừa trực tiếp từ bộ cơ sở dữ liệu mở mang tên “Snake Dataset” do tác giả Aditya Sharma công bố trên nền tảng Kaggle. Dữ liệu gốc bao gồm hình ảnh của các loài rắn bản địa tại Ấn Độ và được tác giả tổ chức cấu trúc lưu trữ sẵn thành các thư mục huấn luyện (train) và kiểm thử (test). Điểm giá trị nhất của bộ dữ liệu này là các hình ảnh đều được chụp trong môi trường sống tự nhiên. Do đó, hình ảnh chứa đựng sự đa dạng về góc độ, điều kiện ánh sáng và đặc biệt bị nhiễu bởi bối cảnh sinh thái phức tạp (đất, đá, lá cây khô,...). Đặc tính này phản ánh đúng độ khó thực tế của bài toán thị giác máy tính, đảm bảo mô hình không bị phụ thuộc vào môi trường phòng thí nghiệm.

2.2.1.2 Quy mô và chiến lược gán nhãn phân cấp

Thay vì xử lý toàn bộ dữ liệu trên một mặt phẳng đa lớp phức tạp, đề án tổ chức hệ thống dữ liệu theo chiến lược phân cấp sát với tư duy chẩn đoán y tế.

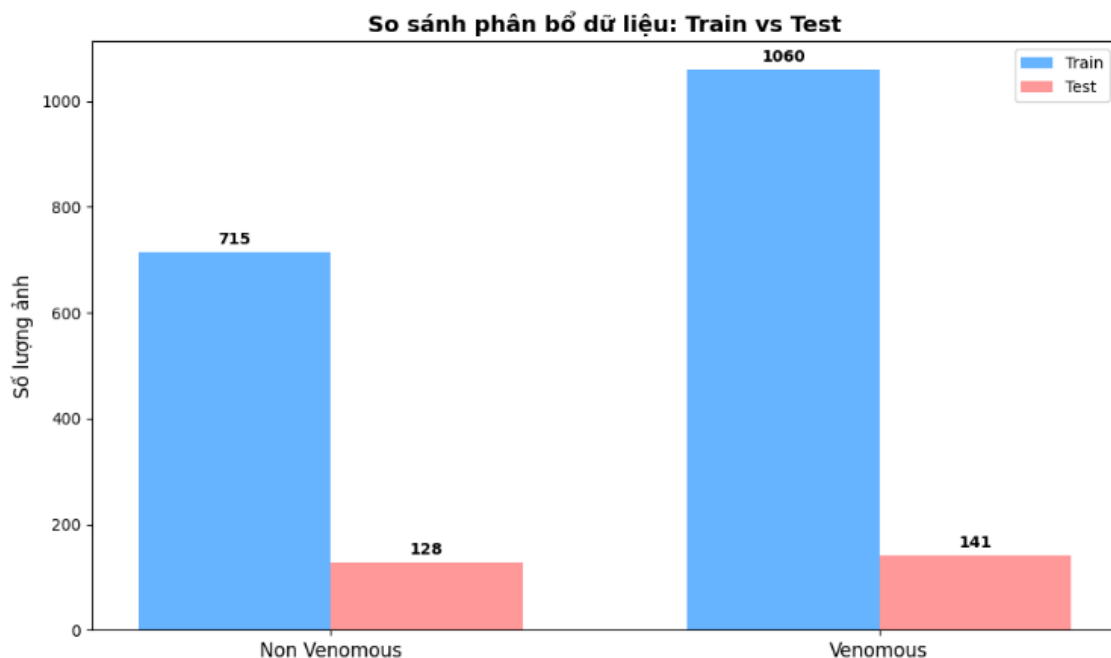


Hình 8: Sơ đồ phân cấp nhãn dữ liệu rắn độc và rắn không độc

Dữ liệu thực nghiệm được chia thành hai bộ cốt lõi:

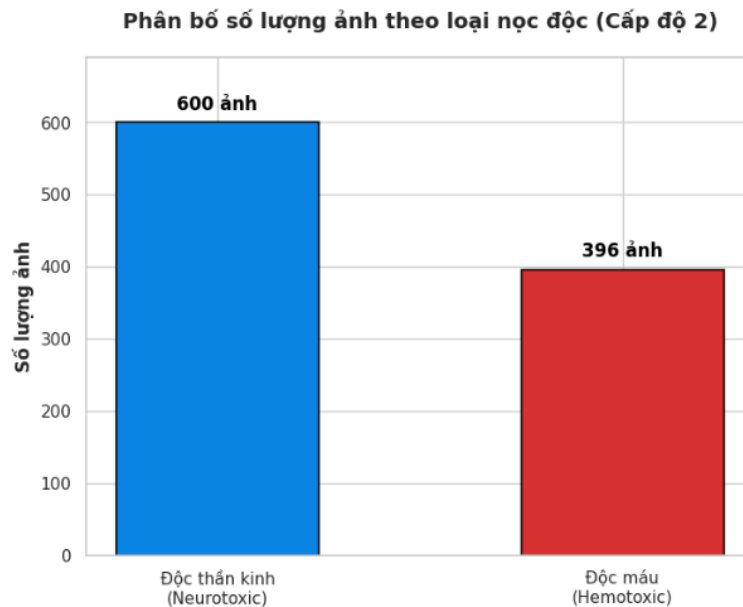
Bộ dữ liệu Cấp độ 1 (cho mục đích phân loại nhị phân: Có độc / Không độc): Tổng quy mô đạt 2.044 hình ảnh. Để đảm bảo tính khách quan trong việc đánh giá mô hình, tập dữ liệu được phân chia cụ thể như sau:

- *Tập huấn luyện*: Gồm 1.775 ảnh, trong đó có 715 ảnh rắn không độc và 1.060 ảnh rắn có độc. Tập dữ liệu này cung cấp khối lượng thông tin chính để thuật toán trích xuất các đặc trưng hình thái chung.
- *Tập kiểm thử*: Gồm 269 ảnh hoàn toàn độc lập với quá trình huấn luyện, bao gồm 128 ảnh rắn không độc và 141 ảnh rắn có độc, được sử dụng để đo lường hiệu suất thực tế.



Hình 8: So sánh phân bố dữ liệu: Train vs Test

Bộ dữ liệu Cấp độ 2 (cho mục ích phân loại chi tiết nọc độc: Độc máu / Độc thần kinh): Bộ dữ liệu này được trích xuất trực tiếp từ nhóm rắn có độc thuộc tập huấn luyện của Cấp độ 1, nhằm đi sâu vào bài toán phân loại nọc độc. Tổng quy mô tập dữ liệu này đạt 996 hình ảnh, bao gồm 396 ảnh đại diện cho nhóm nọc độc máu (Hemotoxic) và 600 ảnh đại diện cho nhóm nọc độc thần kinh (Neurotoxic). Tập dữ liệu Cấp độ 2 tiếp tục được phân chia nghiêm ngặt theo tỷ lệ 80% dành cho việc huấn luyện mô hình và 20% dành cho khâu đánh giá.



Hình 9: Phân bố số lượng ảnh theo loại nọc độc (Cấp độ 2)

2.2.2 Đặc trưng sinh học của các class trong tập dữ liệu

2.2.2.1 Nhóm rắn không độc



Eastern Rat Snake (Rắn chuột miền Đông)

Hình 10: Rắn chuột miền Đông

Một loài rắn lớn không độc, dài từ 1 đến 2 mét. Chúng có vảy đen bóng trên lưng và bụng màu sáng, cổ và cằm màu trắng. Đầu của rắn chuột miền Đông rộng hơn cổ và phần còn lại của cơ thể.

The Trinket Snake (*Coelognathus helena* - rắn trang sức)



Hình 11: Trinket Snake

Dài 100 -140cm (tối đa 168cm) với thân hình gầy, khá phổ biến trên khắp Ấn Độ, cũng như Nepal, Sri Lanka và Bangladesh. Đây là một loại rắn không có nọc độc, chưa bao giờ giết chết con người, thay vào đó nổi tiếng hơn với vẻ đẹp của mình.

Checkered keelback (*Xenochrophis piscator* – rắn nước sọc)



Hình 12: Rắn nước sọc

Dài trung bình 75 cm nhưng có thể dài tới 135 cm. Phần trên cơ thể có màu xám, nâu hoặc ô liu với hoa văn vân đen và hai vệt đen phía sau mỗi mắt. Phần dưới cơ thể có màu trắng nhạt với viền đen ở mỗi bên. Một số cá thể có thể có đốm cam ở hai bên thân dưới. Chúng thường được tìm thấy trong ao, suối và mương rãnh ở các cánh đồng canh tác. Không có độc.

Green vine snake (*Oxybelis fulgidus*)



Hình 13: Green vine snake

Dài khoảng 1,5–2 m (59–79 in). Đuôi dài và rất mỏng manh, nhưng chủ yếu được sử dụng để giữ trong khi vớ lấy con mồi. Đầu có hình dạng khí động học và rất nhọn, miệng rất lớn và kéo dài gần như toàn bộ chiều dài của đầu. Lưỡi dài và màu xanh lục. Nọc độc có trong dịch nước bọt của *O. fulgidus* có tác dụng nhanh trên động vật nhỏ, nhưng ít hoặc không có tác dụng đối với con người, Xếp vào nhóm không độc hay chính xác hơn là vô hại.

Indian Python (*Python molurus* - Trăn Ấn Độ)



Hình 14: Trăn Ấn Độ

Trăn Ấn Độ dài từ 3-6 m với trọng lượng trung bình là 31-58 kg. Sở hữu thân dài, chắc nịch với các màu từ trắng đến vàng và được bao phủ bởi các hoa văn đốm màu nâu sẫm đến rám nắng. Phạm vi màu sắc khác nhau tùy thuộc vào nơi chúng sống. Ví dụ, quần thể ở các khu rừng đồi của Assam và Tây Ghats có xu hướng sẫm màu hơn các mẫu vật của Đông Ghats và Cao nguyên Deccan. Chúng có đầu hình tam giác với vảy nhỏ và hạt, mõm cùn và mắt nhỏ với đồng tử dọc. Nhiều mẫu vật có sọc từ gần mũi kéo dài qua mắt đến phía sau đầu. Tuy nhiên, phần trước mắt của chúng bắt đầu mờ dần khi già đi và chỉ có thể nhìn thấy các sọc sau mắt. Vảy của chúng nhỏ, nhẵn và bóng. Không có nọc độc.

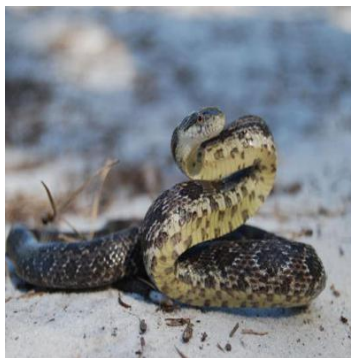
Banded Racer (*Argyrogena fasciolata*)



Hình 15: Banded Racer

Rắn sọc *Argyrogena fasciolata* (Shaw, 1802) là một loài rắn không độc thuộc họ Colubridae được tìm thấy ở tiểu lục địa Ấn Độ. Mồm tù, hơi cong và nhô cao. Vảy mồm lớn, chiều rộng lớn hơn chiều cao; phần vảy nhìn thấy được từ trên xuống có chiều dài bằng hơn một nửa khoảng cách từ nó đến vảy trán. Phần lưng có màu vàng ô liu hoặc nâu, nổi bật với các dải sọc ngang hẹp màu trắng, nâu và đen nằm xen kẽ ở nửa thân trước. Các dải sọc này có thể biến mất hoàn toàn khi rắn trưởng thành. Phần bụng có màu vàng đồng nhất. Tổng chiều dài cơ thể đạt 1.060 mm, trong đó phần đuôi dài 220 mm.

Black Rat Snake (Obsolete Elaph - Rắn chuột đen)



Hình 16: Rắn chuột đen

Kích thước 42” - 72” (khoảng 106 – 183 cm). Đây là loài rắn siết mồi, leo trèo cây rất giỏi. Thức ăn của chúng bao gồm ếch nhái, thằn lằn, rắn nhỏ, chim và trứng. Loài này sống khá phổ biến ở khu vực Pine Barrens.

Arabian sand boa (*Eryx jayakari* – Trăn cát ả rập)



Hình 17: Trăn cát

Trăn cát Ả Rập là một loài rắn nhỏ phát triển với tổng chiều dài (bao gồm cả đuôi) khoảng 38 cm (15 inch). Đôi mắt rất nhỏ và nằm trên đỉnh đầu, có mõm cùn và hình nêm. Màu sắc của loài rắn này là màu xám vàng hoặc nâu cát, lốm đốm với các đốm trắng và có dải ngang với các vết sẫm màu.

The buff striped keelback (*Amphiesma stolatum* - Rắn hoa cỏ sọc)



Hình 18: Rắn hoa cỏ sọc

Rắn hoa cỏ sọc không có nọc độc, dài khoảng 50 - 80cm (con cái thường lớn hơn con đực), nổi bật với các sọc dọc lưng và 19 hàng vảy lưng có gờ nổi (trừ hàng ngoài cùng nhẵn bóng). Về cấu trúc phần đầu, vảy mõm chạm 6 vảy lân cận, vảy mũi không chạm vảy môi trên thứ hai, có 1 vảy thái dương đơn và 8 vảy môi trên (vảy 3, 4, 5 chạm viền mắt). Phần dưới cơ thể gồm 118 - 161 vảy bụng, vảy hậu môn chia đôi và 46 - 89 vảy dưới đuôi xếp thành từng cặp.

Rough-scaled sand boa (*Eryx/ Gongylophis conicus conicus*)



Hình 19: Rough-scaled sand boa

Là một loài hoàn toàn không có nọc độc, thuộc họ Boidae, nổi tiếng với bản tính rất hiền lành và hiếm khi cắn để tự vệ. Về ngoại hình, đây là loài có thân hình mập mạp, kích thước từ trung bình đến lớn; con non dài từ 10 đến 20 cm, trong khi con trưởng thành có thể đạt chiều dài từ 61 đến 76 cm. Đặc điểm nhận diện nổi bật của chúng là màu nền cơ thể thường có sắc nâu nhạt, xám hoặc ngả vàng, kết hợp với một loạt các đốm rộng màu nâu sẫm hình ngoằn ngoèo chạy dọc trên lưng. Ngoài ra, loài này còn có lớp vảy có gờ ở phần gần đuôi và chiếc đuôi tù, cùn thường được chúng để lộ ra ngoài nhằm đánh lạc hướng kẻ thù khi cuộn tròn phòng thủ.

Red Sand Boa (*Eryx johnii*)



Hình 20: Red Sand Boa

Là một loài rắn không độc thuộc phân họ Erycinae của họ Boidae. Cá thể trưởng thành của loài *E. johnii* hiếm khi dài quá 61 cm (bao gồm cả đuôi), mặc dù đôi khi chúng có thể đạt tới 91 cm. Thích nghi với việc đào hang, đầu có hình nệm với lỗ mũi hẹp và mắt rất nhỏ. Thân hình trụ với những vảy nhỏ bóng loáng ở lưng. Đuôi cùn, tròn và không tách rời khỏi thân, trông như bị cụt. Màu sắc thay đổi từ nâu đỏ đến vàng nhạt

Aesculapian snake (*Zamenis longissimus*)



Hình 21: Aesculapian snake

Đây là một loài rắn siết môi hoàn toàn không có nọc độc, có nguồn gốc bản địa từ châu Âu. Khi mới nở, rắn con có chiều dài khoảng 30 cm. Khi trưởng thành, chúng sở hữu thân hình thon dài, mảnh mai với tổng chiều dài (tính cả đuôi) thường dao động từ 110 - 160 cm, và nặng khoảng 350 - 890 gram. Tuy nhiên, một số cá thể có thể phát triển đến 200 cm, thậm chí đạt kỷ lục lên tới 225 cm. Về ngoại hình, loài rắn này có màu sẫm hoặc màu đồng đặc trưng. Đặc biệt, lớp vảy của chúng rất nhẵn mịn, tạo ra hiệu ứng ánh kim loại bóng bẩy và bắt mắt.

Oriental rat snake (*Ptyas mucosa* – Rắn ráo trâu/ Rắn chuột Ấn Độ)



Hình 22: Rắn ráo trâu

Là loài rắn hoàn toàn không có nọc độc. Mặc dù là một loài rắn lớn với chiều dài trung bình của cá thể trưởng thành từ 1,5 đến 1,95 mét (kỷ lục từng ghi nhận lên tới 3,7 mét), cơ thể chúng lại vô cùng thon dài và mảnh mai; một con rắn dài 2 mét cũng chỉ có đường kính thân rơi vào khoảng 4 đến 6 cm. Về màu sắc, bề ngoài của chúng có thể thay đổi rất linh hoạt tùy theo môi trường sống, từ màu nâu nhạt ở những vùng đất khô hạn cho đến gần như đen tuyền tại các khu rừng ẩm ướt.

Gulf Hammock Rat Snake (*Pantherophis obsoleta williamsi*)



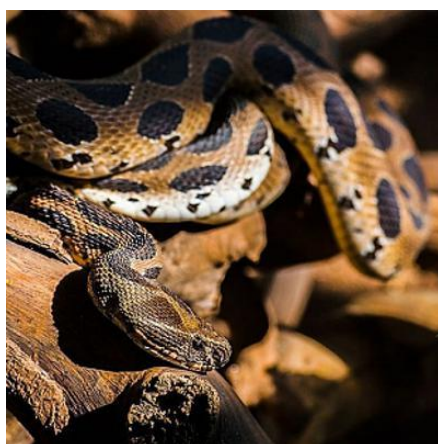
Hình 23: Gulf Hammock Rat Snake

Đây là một loài rắn siết môi lớn hoàn toàn không có nọc độc và vô hại với con người, nhưng lại có tập tính phòng thủ khá cao. Khả năng leo trèo của chúng rất xuất sắc, thường xuyên được tìm thấy trên các cành cây cao. Ngoại hình của loài này rất đa dạng: ở khu vực phía bắc chúng thường có nền da màu xám, trong khi ở phía nam lại mang màu cam nhạt, nổi bật với cả họa tiết sọc dọc và đốm hình yên ngựa (những họa văn này được giữ nguyên từ lúc nhỏ cho đến khi trưởng thành). Vảy ở hai bên sườn nhẵn bóng nhưng phần vảy lưng lại có gờ nhẹ.

2.2.2.2 Nhóm rắn độc

2.2.2.2.1 Độc máu (Hemotoxic)

Russell's Viper (*Daboia russelii*)



Hình 24: Russell's Viper

Rắn lục Russell là một loài rắn cực độc có chiều dài trung bình khoảng 120cm, thân hình mập mạp với phần đầu dẹt, hình tam giác có vết chữ “V” hoặc “X” đặc trưng. Cơ thể chúng thường có màu

vàng sậm, nâu hoặc rậm nắng, nổi bật với ba dải đốm màu nâu sẫm dọc cơ thể, mỗi đốm được viền bởi một vòng đen và mép ngoài cùng màu trắng hoặc vàng. Chúng sở hữu nanh độc dài có thể tiêm một lượng lớn nọc độc chết người vào cơ thể. Sau khi bị cắn, nạn nhân sẽ cảm thấy đau đớn dữ dội tại vết cắn (cơn đau có thể kéo dài 2-4 tuần), khu vực bị cắn sưng tấy và phỏng rộp, kèm theo các triệu chứng toàn thân nghiêm trọng như chảy máu chân răng, tiểu ra máu, tụt huyết áp, giảm nhịp tim, nôn mửa và suy thận. Nếu không được cấp cứu và tiêm huyết thanh kháng nọc độc kịp thời, nạn nhân có nguy cơ tử vong trong vòng từ 1 đến 14 ngày do nhiễm trùng máu, suy tim, suy hô hấp hoặc suy thận. Vì thuộc họ rắn lục (Viper) nên em xếp loài này vào loại độc máu.

Saw Scaled Viper (*Echis carinatus*)



Hình 25: Saw Scaled Viper

Rắn lục vảy cưa dài trung bình từ 30-90 cm (12-35 inches), toàn thân phủ đầy vảy có gờ sắc nét, màu nền cơ thể đa dạng từ xám, hơi đỏ, vàng nhạt, ô liu hoặc nâu nhạt; phần đầu ngắn, tròn có họa tiết hình đỉnh ba màu trắng trên đỉnh, một dải sáng nổi từ mắt đến góc hàm và phần bụng có màu từ trắng đến hồng nhạt. Nọc độc của chúng chứa tới bốn loại độc tố gồm độc tố tim, độc tố thần kinh, độc tố máu và độc tố tế bào; nọc của rắn cái độc hơn rắn đực, tiết ra trung bình 18 mg (tối đa 72 mg) nọc khô với chỉ số LD50 tiêm tĩnh mạch ở chuột là 0,24 mg/kg. Khi bị cắn, nạn nhân sẽ bị sưng tấy và đau đớn tại chỗ cắn (vết sưng đau có thể lan ra toàn bộ chi trong vòng 12-24 giờ), nổi mụn nước ở những trường hợp cực đoan, kèm theo các triệu chứng toàn thân nguy hiểm như rối loạn đông máu và xuất huyết. Ghi nhận rằng loài này gây ra 4000 đến 5000 ca tử vong mỗi năm ở Ấn Độ. Tuy mang độc tố hỗn hợp, các triệu chứng ghi nhận khi cắn lại mang biểu hiện đặc trưng của độc máu, thuộc chi rắn lục (Viper) nên em sẽ xếp chúng vào nhóm độc máu.

Deinagkistrodon acutus



Hình 26: *Deinagkistrodon acutus*

Có đặc điểm nhận dạng là thân hình mập, chiều dài trung bình từ 0,8 đến 1,0 mét và đạt tối đa 1,57 mét ở con đực. Đầu to, hình tam giác với phần mõm hếch lên trên, đỉnh và hai bên đầu có màu đen đồng nhất, nổi bật với một vết đen kéo dài từ mắt đến góc miệng. Lưng rắn màu nâu nhạt hoặc nâu xám với các hình tam giác màu nâu sẫm ở hai bên sườn giao nhau tạo thành khoảng 20 đốm vuông trên lưng, kết hợp với một hàng đốm đen lớn dọc hai bên gần bụng và phần bụng màu vàng có đốm nâu sẫm. Chúng sở hữu nọc độc máu cực mạnh và gây xuất huyết nghiêm trọng; mỗi nhát cắn thông qua cặp nanh lớn có thể bơm vào lượng nọc lên tới 214 mg (khô) chứa ít nhất bốn loại độc tố xuất huyết (Acutolysin A, B, C, D). Cảm giác khi bị cắn bao gồm cơn đau nhức dữ dội tại chỗ và chảy máu gần như ngay lập tức, tiếp theo là tình trạng sưng tấy nghiêm trọng, phỏng rộp, hoại tử và lở loét. Các triệu chứng toàn thân, điển hình như đánh trống ngực, cũng có thể xuất hiện đột ngột khá sớm sau vết cắn. Dân gian rằng nạn nhân sẽ chết chỉ sau khi đi được 100 bước.

Daboia siamensis (Eastern Russell's viper)



Hình 27: *Daboia siamensis*

Daboia siamensis (còn gọi là Rắn lục Russell phương Đông hay Rắn lục Russell Xiêm) có đặc điểm nhận dạng ở phần lưng mang hoa văn tương tự như loài *D. russelii*, ngoại trừ việc màu sắc của chúng thiên về màu xám hoặc màu ô liu nhiều hơn, kèm theo các đốm nhỏ nằm xen kẽ giữa

các hàng đốm lớn , và phần bụng phía sau của chúng có màu phớt xám. Nọc độc gồm các hỗn hợp cytotoxin gây hoại tử, neurotoxin tác dụng lên thần kinh và haemotoxin gây xuất huyết. Vì thuộc chi rắn lục (viper) nên dù mang độc tố hỗn hợp nhưng để dễ phân loại, em xin xếp chúng vào loại độc máu.

2.2.2.2.2 Độc thần kinh (neurotoxin)

Indian Cobra (Naja naja - Hổ mang Ấn Độ)



Hình 28: Hổ mang Ấn Độ

Rắn hổ mang Ấn Độ là một trong “Tứ đại rắn độc” cực kỳ nguy hiểm, có chiều dài trung bình 1,8 - 2,2 mét với đầu hình bầu dục dẹt và đặc điểm nhận dạng nổi bật nhất là vệt sáng hình chữ “U” trên phần mang khi bành cổ. Nọc độc của chúng chứa hỗn hợp độc tố thần kinh và độc tố tim vô cùng mạnh mẽ, mỗi cú cắn bơm vào cơ thể từ 150 đến 600 mg nọc độc cùng enzyme giúp chất độc lây lan cực nhanh. Chỉ từ 15 phút đến 2 giờ sau khi bị cắn, nạn nhân sẽ xuất hiện các triệu chứng như chóng mặt, đau đầu, buồn nôn, buồn ngủ và suy nhược trầm trọng. Do nọc độc phát tác rất nhanh, nếu không được điều trị kịp thời, nạn nhân sẽ tử vong do ngừng tim và suy hô hấp.

Monocled Cobra (Naja kaouthia - Hổ mang một mắt kính)



Hình 29: Hổ mang một mắt kính

Rắn hổ mang một mắt kính có chiều dài trung bình từ 1,3 đến 1,5 mét (tối đa lên tới gần 2,3 mét), được nhận dạng đặc trưng bởi hoa văn hình chữ “O” ở mặt lưng của phần mang, hai đốm đen ở cổ họng, màu sắc thân đa dạng (đen, nâu, xám, ô liu, vàng) và phần bụng màu cát nhạt. Về nọc độc,

đây là loài cực kỳ nguy hiểm với một trong những loại nọc độc phát tác nhanh nhất thế giới, chứa một lượng lớn độc tố thần kinh (đặc biệt là α -cobratoxin), độc tố tim và cơ. Ngay sau khi bị cắn, nạn nhân sẽ lập tức có cảm giác đau đớn, tê bì, buồn ngủ, gặp các vấn đề về cơ bắp và tê liệt, có thể dẫn đến hoại tử, suy hô hấp hoặc ngừng tim. Đoạn tài liệu không cung cấp thông tin cụ thể về thời gian tử vong mất bao lâu sau khi cắn, nhưng có nhấn mạnh nọc độc tác dụng rất nhanh và tỷ lệ tử vong do loài rắn này gây ra cao nhất là ở khu vực Thái Lan.

King cobra (Ophiophagus hannah - Rắn hổ chúa)



Hình 30: Rắn hổ chúa

Rắn hổ chúa là loài rắn độc dài nhất thế giới với chiều dài trung bình từ 3,18 đến 4 m (kỷ lục ghi nhận đạt 5,85 m). Về đặc điểm nhận dạng, màu sắc cơ thể chúng rất đa dạng tùy theo môi trường sống, từ đen có sọc trắng cho đến xám nâu trơn. Cá thể trưởng thành thường có da màu ô liu sẫm hoặc nâu với phần bụng màu kem hoặc vàng nhạt, đầu to với hai vảy chằm lớn ở chóp đầu và cặp răng nanh ngắn cố định, trong khi đó con non lại có màu đen bóng với các dải màu vàng hẹp. Về nọc độc, mỗi nhát cắn của chúng có thể bơm vào một lượng nọc độc khổng lồ (trung bình khoảng 420 mg trọng lượng khô), bao gồm chủ yếu là độc tố tế bào (cytotoxins), độc tố thần kinh (neurotoxins) và độc tố tim (cardiotoxins). Đối với cảm giác khi bị cắn, nọc độc sẽ nhanh chóng chặn các thụ thể thần kinh, khiến nạn nhân lập tức trải qua những cơn đau dữ dội, kèm theo tình trạng mờ mắt, chóng mặt, buồn ngủ và cuối cùng là tê liệt. Tình trạng nhiễm độc này sẽ tiến triển thành suy sụp tim mạch, và thời gian tử vong có thể xảy ra rất nhanh, chỉ trong vòng sớm nhất là 30 phút sau khi bị tiêm nọc.

Common Krait (*Bungarus caeruleus* - Rắn cạp nia)



Hình 31: Rắn cạp nia

Rắn cạp nia Ấn Độ có chiều dài từ 90 - 175 cm với thân hình trụ màu đen hoặc đen xanh bóng bẩy, nổi bật bởi hàng vảy lưng hình lục giác to và khoảng 40 sọc ngang hẹp màu trắng (các sọc này mờ dần thành đốm khi rắn già đi). Đây là loài cực độc với nọc độc chứa độc tố thần kinh mạnh mẽ; một nhát cắn tiết ra trung bình 10mg nọc nhưng chỉ cần 2 - 3mg đã đủ đoạt mạng người. Vết cắn của chúng đặc biệt nguy hiểm vì gần như không đau, chỉ châm chích nhẹ như kiến hoặc muỗi đốt khiến nạn nhân vô cùng chủ quan. Nếu không được cấp cứu kịp thời, độc tố sẽ nhanh chóng làm đứt gãy tín hiệu thần kinh gây liệt cơ mặt, cảm khẩu, mù lòa, liệt toàn thân và dẫn đến tử vong chỉ trong vòng 4 đến 8 giờ do liệt cơ hô hấp.

CHƯƠNG 3: GIỚI THIỆU THUẬT TOÁN VÀ MINH HỌA PYTHON

*“Chẳng phải liu diu cũng giống nhà
Rắn đầu biếng học lỗi không tha
Thẹn đèn hổ lửa đau lòng mẹ
Nay thét mai gầm rát cổ cha
Ráo mép chỉ quen tuồng lếu láo
Lăn lưng chẳng khỏi vết roi da
Từ nay Trâu Lỗ xin chăm học
Kẻo hổ mang danh tiếng thế gia”*

Lê Quý Đôn

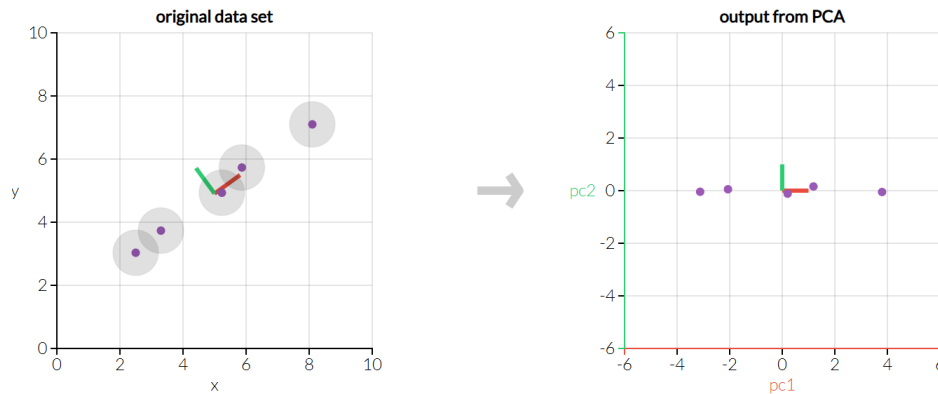
3.1 Cơ sở lý thuyết về các thuật toán Học máy đã sử dụng

3.1.1 Phân tích thành phần chính (Principal Component Analysis - PCA)

Phân tích thành phần chính (PCA) là một thuật toán toán học giúp giảm chiều dữ liệu trong khi vẫn giữ lại hầu hết sự biến thiên trong tập dữ liệu. Nó thực hiện việc giảm chiều này bằng cách xác định các hướng, được gọi là các thành phần chính, mà dọc theo đó sự biến thiên trong dữ liệu

là tối đa. Bằng cách sử dụng một vài thành phần, mỗi mẫu có thể được biểu diễn bằng một số lượng tương đối ít các con số thay vì bằng các giá trị cho hàng ngàn biến số. Sau đó, các mẫu có thể được vẽ đồ thị, giúp đánh giá trực quan sự tương đồng và khác biệt giữa các mẫu và xác định xem các mẫu có thể được phân nhóm hay không.

Như vậy, ý tưởng chính của phương pháp phân tích thành phần chính là giảm số chiều của dữ liệu ban đầu. Ví dụ, với $a \in \mathbb{R}^D \rightarrow b \in \mathbb{R}^K$, sao cho $D > K$, bằng cách giữ lại K phần tử quan trọng nhất.



Hình 32: Ý tưởng PCA

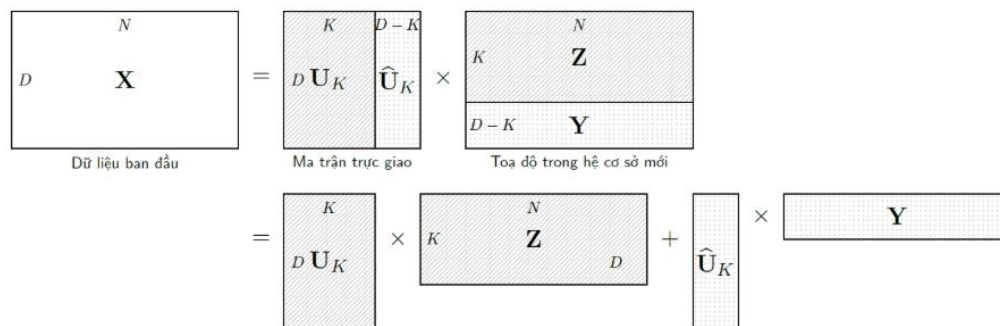
Dưới đây ví dụ trực quan về Phân tích thành phần chính (PCA) được áp dụng trên hình ảnh



Hình 33: Phân tích thành phần chính (PCA) trên hình ảnh

Vấn đề được đặt ra là dựa vào tiêu chí nào để chọn ra K thành phần cốt lõi thay vì các thành phần khác? Cơ sở của việc lựa chọn này nằm ở chỗ: ta sẽ lược bỏ một chiều không gian nếu các điểm dữ liệu trên đó không có sự khác biệt lớn.

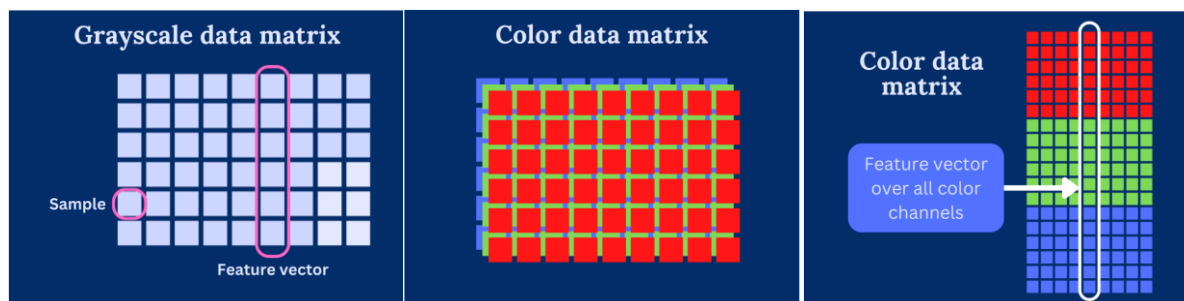
Quy trình thực hiện PCA được minh họa như sau:



Hình 34: Phương pháp PCA ở góc nhìn đại số

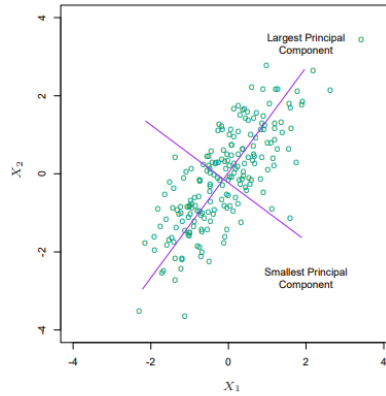
Thuật toán PCA tiến hành tìm kiếm một ma trận trực giao U_K với mục tiêu tập trung những thông tin cốt lõi nhất vào thành phần $U_K \times Z$, đồng thời cắt bỏ phần dữ liệu không quan trọng $\hat{U}_K \times Y$

PCA được ứng dụng rất hiệu quả trong bài toán nén ảnh, cụ thể được minh họa như sau:



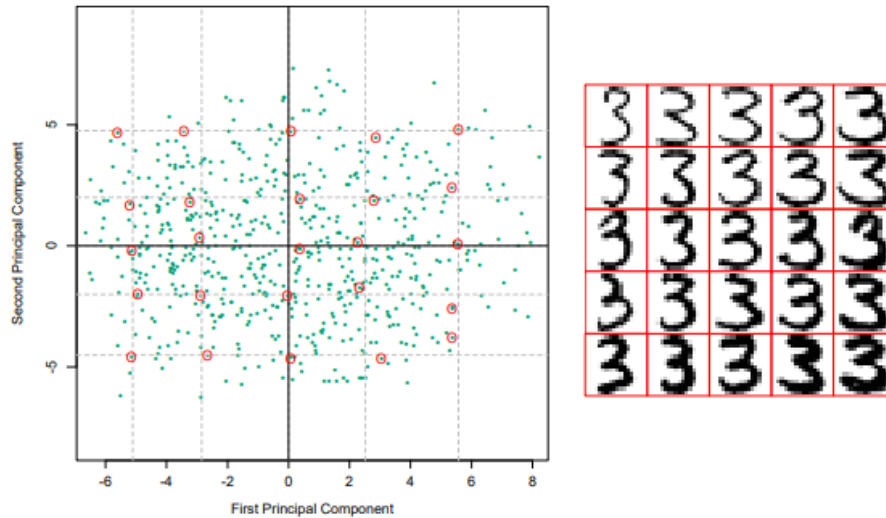
Hình 35: Sử dụng PCA trong việc nén hình ảnh

Về bản chất, PCA thực hiện một chuỗi các phép chiếu dữ liệu không tương quan với nhau và được sắp xếp theo thứ tự phương sai giảm dần. Cụ thể, hướng của thành phần chính đầu tiên v_1 có tính chất là phép chiếu $z_1 = Xv_1$ đạt phương sai mẫu lớn nhất trong số tất cả các tổ hợp tuyến tính chuẩn hóa của các cột của X .



Hình 36: Biểu đồ PCA 2 chiều

Đồng thời, PCA hoạt động như một bộ lọc nhiễu tự nhiên bằng cách loại bỏ các thành phần có giá trị kỳ nhỏ d_j tương ứng với các hướng trong không gian cột của X có phương sai nhỏ.



Hình 37: Trực quan hóa tập dữ liệu chữ số 3 qua không gian PCA

Nhờ công cụ giảm chiều và nén dữ liệu này, hệ thống không cần phải xử lý trực tiếp trên không gian hàng vạn điểm ảnh thô rời rạc (ví dụ: vector $x_i \in \mathbb{R}^D$). Thay vào đó, nó chắt lọc ra một tập hợp nhỏ các “thành phần tinh hoa” nhưng lại biểu diễn được phần lớn sự biến thiên của toàn bộ bức ảnh. Việc trích xuất và sử dụng các đặc trưng hạ chiều này làm đầu vào thay cho dữ liệu thô là một bước tiền xử lý mạnh mẽ, giúp giảm tải gánh nặng tính toán và nâng cao đáng kể hiệu suất phân loại cho các thuật toán Học máy truyền thống như SVM, Random Forest hay Logistic Regression.

3.1.2 Máy học Vector hỗ trợ (Support Vector Machine - SVM)

Máy học vector hỗ trợ (Support Vector Machine - SVM) là một thuật toán máy tính học qua ví dụ để gán nhãn cho các đối tượng. Chẳng hạn, một SVM có thể học cách nhận biết các hoạt động gian lận thẻ tín dụng bằng cách kiểm tra hàng trăm hoặc hàng ngàn báo cáo hoạt động thẻ tín dụng có gian lận và không gian lận. Hoặc, một SVM có thể học cách nhận diện các chữ số viết tay bằng

cách kiểm tra một bộ dữ liệu lớn gồm các hình ảnh quét của các số không, số một viết tay,...SVM cũng đã được áp dụng thành công vào ngày càng nhiều các ứng dụng sinh học đa dạng.

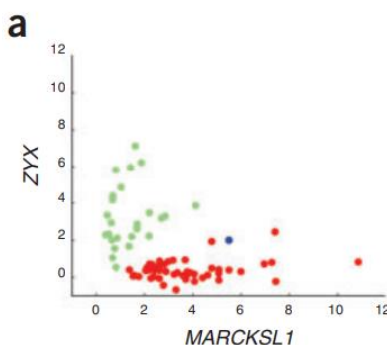
Về bản chất, SVM là một thực thể toán học, một thuật toán (hoặc công thức) để tối đa hóa một hàm toán học cụ thể đối với một tập dữ liệu nhất định. Để hiểu được bản chất của phân loại bằng SVM, người ta chỉ cần nắm vững bốn khái niệm cơ bản:

- (i) Siêu phẳng phân tách (separating hyperplane)
- (ii) Siêu phẳng có lề tối đa (maximum-margin hyperplane)
- (iii) Lề mềm (soft margin)
- (iv) Hàm nhân (kernel function).

3.1.2.1 The separating hyperplane

Ta có bài toán phân loại hồ sơ biểu hiện gen ung thư. Các vi mảng Affymetrix được sử dụng bởi Golub và cộng sự chứa các mẫu dò cho 6.817 gen người. Đối với một mẫu tủy xương nhất định, xét nghiệm vi mảng trả về 6.817 giá trị, mỗi giá trị đại diện cho mức độ mRNA tương ứng với một gen cụ thể. Golub và cộng sự đã thực hiện xét nghiệm này trên 38 mẫu tủy xương, 27 mẫu từ những bệnh nhân bị bệnh bạch cầu lympho cấp tính (ALL) và 11 mẫu từ những bệnh nhân bị bệnh bạch cầu tủy cấp tính (AML).

Để minh họa, hãy tưởng tượng rằng các vi mảng chỉ chứa các mẫu dò cho hai gen là ZYX và MARCKSL1. Hồ sơ biểu hiện gen của chúng ta bao gồm hai con số, có thể dễ dàng được vẽ lên đồ thị như sau:

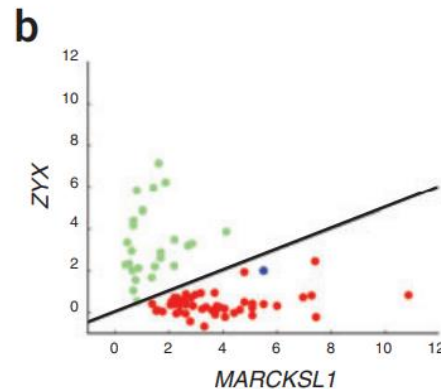


Hình 38: Ví dụ a

Trong hình, các giá trị tỷ lệ thuận với cường độ huỳnh quang trên vi mảng, do đó trên cả hai trục, một giá trị lớn chỉ ra rằng gen đó được biểu hiện cao và ngược lại. Các mức độ biểu hiện được biểu thị bằng một dấu chấm màu đỏ hoặc màu xanh lá cây, tùy thuộc vào việc mẫu đó là từ một bệnh nhân mắc ALL hay AML. SVM phải học cách phân biệt sự khác nhau giữa hai nhóm này và, khi được cung cấp một vector biểu hiện chưa được gán nhãn, chẳng hạn như vector được dán nhãn 'Không xác định' trong hình, có thể dự đoán xem nó tương ứng với bệnh nhân ALL hay AML.

Mắt người rất giỏi trong việc nhận dạng mẫu. Chỉ cần nhìn thoáng qua cũng cho thấy rằng các hồ sơ AML tạo thành một cụm ở vùng trên bên trái của biểu đồ, và các hồ sơ ALL tập trung ở vùng dưới bên phải. Theo tư duy thông thường, có thể phát biểu rằng bệnh nhân mắc AML nếu mức độ

biểu hiện của MARCKSL1 cao gấp đôi mức độ biểu hiện của ZYX, và ngược lại đối với ALL. Về mặt hình học, quy tắc này tương ứng với việc vẽ một đường thẳng giữa hai cụm:



Hình 39: Ví dụ b

Sau đó, việc dự đoán nhãn của một hồ sơ biểu hiện chưa biết rất dễ dàng: Người ta chỉ cần hỏi xem hồ sơ mới nằm ở phía ALL hay phía AML của đường phân tách này.

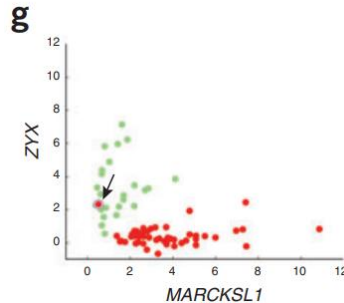
Khái niệm siêu phẳng phân tách có thể được hiểu rõ hơn thông qua việc xem xét số gen (tương trưng cho số lượng đặc trưng) trong tập dữ liệu. Khi chỉ phân tích một gen (đại diện cho không gian một chiều), ranh giới phân chia các mẫu dữ liệu chỉ đơn giản là một điểm. Khi phân tích hai gen (đại diện cho không gian hai chiều), ranh giới này trở thành một đường thẳng. Tương tự, ở không gian ba chiều, chúng ta cần một mặt phẳng để phân tách dữ liệu. Tổng quát hóa về mặt toán học, đối với không gian đa chiều, ranh giới phân cách này được gọi chung là siêu phẳng (hyperplane). Về bản chất, đây chính là vách ngăn quyết định việc một mẫu biểu hiện gen sẽ được phân loại vào nhóm ALL hay AML.

3.1.2.2 The maximum-margin hyperplane

SVM khác biệt so với các thuật toán phân loại dựa trên siêu phẳng khác ở cách thức nó lựa chọn siêu phẳng. Xem lại bài toán phân loại được nhắc đến ở phần siêu phẳng. Ta đã xác định được rằng mục tiêu của SVM là nhận diện một đường thẳng tách biệt các hồ sơ biểu hiện ALL khỏi AML trong không gian hai chiều này. Tuy nhiên, có rất nhiều đường thẳng như vậy tồn tại. Vậy đường thẳng nào dùng để phân loại tốt nhất? SVM sẽ chọn siêu phẳng phân tách có lề tối đa (maximum-margin). Việc lựa chọn siêu phẳng đặc biệt này sẽ giúp tối đa hóa khả năng của SVM trong việc dự đoán và phân loại chính xác các mẫu dữ liệu mới chưa từng thấy. Điều kiện là dữ liệu dùng để huấn luyện SVM và dữ liệu kiểm tra phải có cùng nguồn gốc/phân phối nhưng không bắt buộc hai tập dữ liệu này phải tuân theo một kiểu phân phối cụ thể nào.

3.1.2.3 The soft margin

Cho đến giờ, ta chỉ đưa ra ví dụ phân tách dữ liệu bằng một đường thẳng. Tuy nhiên, nhiều tập dữ liệu thực tế không thể được phân tách rõ ràng như vậy.

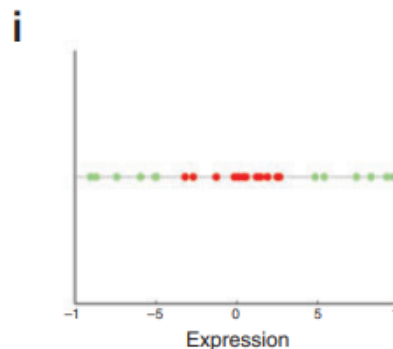


Hình 40: Ví dụ g Tập dữ liệu chứa một 'lỗi', là hồ sơ biểu hiện gen được khoanh tròn

Để xử lý các trường hợp như vậy, thuật toán SVM phải được sửa đổi bằng cách thêm một “lề mềm” cho phép một số điểm dữ liệu vượt qua lề của siêu mặt phẳng phân tách mà không ảnh hưởng đến kết quả cuối cùng. Biên độ mềm này đòi hỏi một tham số do người dùng chỉ định để tránh việc phân loại sai như thêm vào số lượng điểm được phép vi phạm siêu phẳng phân tách và khoảng cách chúng được phép vượt qua đường thẳng. Việc thiết lập tham số này khá phức tạp vì một mặt ta muốn giữ khoảng lề càng rộng càng tốt đối với các điểm dữ liệu đã phân loại đúng, mặt khác lại phải hạn chế số điểm vi phạm ranh giới. Do đó, tham số lề mềm (soft margin parameter) là sự đánh đổi giữa độ rộng của lề và số lượng lỗi vi phạm siêu phẳng.

3.1.2.4 The kernel function

Về bản chất, hàm nhân là một thủ thuật toán học cho phép SVM thực hiện phân loại “hai chiều” trên một tập dữ liệu ban đầu chỉ có một chiều. Tựu trung lại, một hàm nhân sẽ phóng chiếu dữ liệu từ không gian chiều thấp lên một không gian có số chiều cao hơn. Nếu may mắn chọn được một hàm nhân tốt, dữ liệu sẽ trở nên có thể phân tách tuyến tính trong không gian đa chiều mới được tạo ra đó.

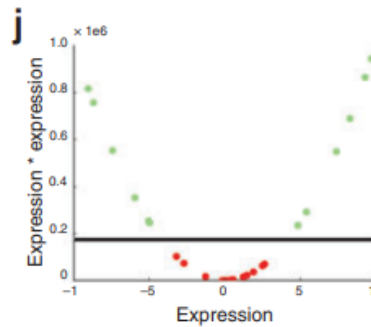


Hình 41: Ví dụ i

Hình trên minh họa một trường hợp về một tập dữ liệu không thể phân tách. Các giá trị của nhóm AML tập trung gần mức 0, còn các điểm của nhóm ALL lại có giá trị tuyệt đối lớn. Vấn đề là

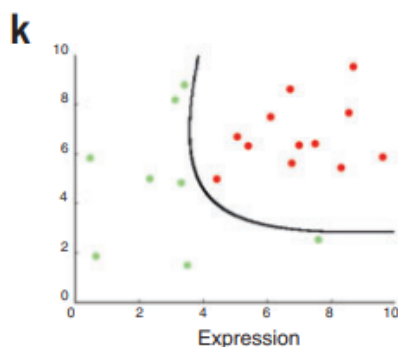
không có bất kỳ một điểm duy nhất nào có thể phân tách hai lớp này, và việc áp dụng “lề mềm” (soft margin) cũng không mang lại tác dụng.

Hàm nhân cung cấp một giải pháp cho vấn đề này bằng cách thêm một chiều (dimension) bổ sung vào dữ liệu

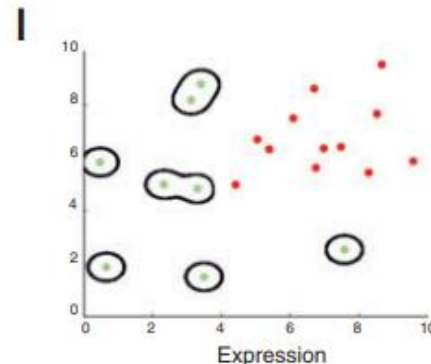


Hình 42: Ví dụ j

Người ta có thể chứng minh được rằng, đối với bất kỳ tập dữ liệu nào có tập dữ liệu không chứa hai đối tượng giống hệt nhau nhưng lại có nhãn trái ngược nhau, luôn tồn tại một hàm nhân cho phép dữ liệu đó được phân tách tuyến tính. Một câu hỏi nảy sinh: tại sao không phải lúc nào cũng phóng chiếu dữ liệu lên một không gian có số chiều cực kỳ cao để chắc chắn tìm được một siêu phẳng phân tách? Việc này giải quyết được nhu cầu phải dùng đến “lề mềm”, trong khi định lý gốc vẫn được áp dụng. Đáng tiếc, việc phóng chiếu lên không gian có số chiều rất cao có thể gây ra nhiều rắc rối do cái gọi là lời nguyền đa chiều: Khi số lượng biến số được xem xét tăng lên, số lượng các giải pháp khả thi cũng tăng theo, nhưng là tăng theo cấp số nhân. Hậu quả là, việc lựa chọn một giải pháp chính xác trở nên khó khăn hơn đối với bất kỳ thuật toán nào.



Hình 43: Ví dụ k



Hình 44: Ví dụ l

Hậu quả khi một tập dữ liệu được phóng chiếu vào không gian có quá nhiều chiều

Dù chứa cùng tập dữ liệu với k, nhưng siêu phẳng được phóng chiếu xuất phát từ một mô hình SVM sử dụng hàm nhân có số chiều rất cao. Kết quả là ranh giới giữa các lớp trở nên quá rập khuôn và bám sát vào các ví dụ cụ thể trong tập dữ liệu huấn luyện. Hiện tượng này gọi là SVM đã bị quá khớp (overfit) với dữ liệu. Rõ ràng, một SVM như vậy sẽ không có khả năng tổng quát hóa tốt khi phải đối mặt với các hồ sơ biểu hiện gen mới.

Câu hỏi được đặt ra là làm thế nào để chọn được một hàm số tốt nhất như vậy? Không may khi phải nói rằng, trong hầu hết các trường hợp, câu trả lời thực tế duy nhất là thử và sai. Bắt đầu với một SVM đơn giản, và sau đó thử nghiệm với nhiều hàm nhân tiêu chuẩn khác nhau. Một hàm nhân tối ưu có thể được lựa chọn từ một tập hợp các hàm nhân cố định chặt chẽ về mặt thống kê thông qua kỹ thuật kiểm chứng chéo. Phương pháp này tốn nhiều thời gian và không đảm bảo một hàm nhân chưa được đưa vào quy trình kiểm chứng chéo lại không thể đem lại hiệu suất tốt hơn.

Hàm nhân mang lại ít nhất hai lợi ích:

- Thứ nhất, các hàm nhân có thể được định nghĩa trên các đầu vào không phải là vector.
- Thứ hai, các hàm nhân cung cấp một khuôn khổ toán học vững chắc để kết hợp nhiều loại dữ liệu khác nhau.

3.1.3 Rừng ngẫu nhiên (Random Forest)

Thuật toán rừng ngẫu nhiên, được L. Breiman đề xuất vào năm 2001, cực kỳ hữu ích trong việc phân loại và hồi quy đa năng. Bằng cách kết hợp một số cây quyết định ngẫu nhiên và tổng hợp các dự đoán của chúng bằng cách lấy trung bình, phương pháp này đã chứng minh tính hiệu quả trong các trường hợp số lượng biến lớn hơn nhiều so với số lượng quan sát. Quy trình học có giám sát này, chịu ảnh hưởng từ các công trình nghiên cứu ban đầu của Amit và Geman (1997), Ho (1998) và Dietterich (2000), hoạt động theo nguyên tắc “chia để trị” đơn giản nhưng hiệu quả: lấy mẫu các phần dữ liệu, xây dựng một cây dự đoán ngẫu nhiên trên mỗi phần nhỏ, sau đó ghép các cây dự đoán này lại với nhau.

Dưới góc độ toán học thì nó được viết lại như sau:

Giả sử ta có tập huấn luyện $D_n = ((X_1, Y_1), \dots, (X_n, Y_n))$ với $X \in \mathcal{X} \subset \mathbb{R}^p, Y \in \mathbb{R}$, mô hình sẽ đi tìm hàm ước lượng nhằm xấp xỉ hàm hồi quy thực tế $m(x) = E[Y|X = x]$. Một mô hình Random Forest bao gồm M cây quyết định, trong đó dự đoán của cây thứ j được ký hiệu là $m_n(x; \theta_j, D_n)$. Ở đây, θ_j là các biến ngẫu nhiên độc lập được sử dụng để lấy mẫu lại tập dữ liệu và lựa chọn ngẫu nhiên các đặc trưng phân chia tại mỗi nút. Kết quả dự đoán tổng thể của mô hình được tính bằng trung bình cộng các dự đoán từ cây con: $m_{M,n}(x) = \frac{1}{M} \sum_{j=1}^M m_n(x, \theta_j, D_n)$ và khi số lượng cây M tiến tới vô cực, mô hình sẽ hội tụ về dạng mức kỳ vọng $m_{\infty,n}(X; D_n) = E_{\Theta}[m_n(x; \Theta, D_n)]$. Quá trình xây dựng từng cây được kiểm soát bởi các tham số cốt lõi: số lượng mẫu lấy ngẫu nhiên a_n , số chiều đặc trưng ngẫu nhiên được xét tại mỗi nút $\mathcal{M}_{try}$, và kích thước tối thiểu để tạo thành một nút $nodesize$. Tại mỗi nút phân chia, thuật toán áp dụng tiêu chuẩn CART-split để tìm điểm cắt tối ưu nhất. Đối với bài toán hồi quy, việc chia cắt này nhằm mục đích đo lường và tối đa hóa mức độ giảm phương sai. Ngược lại, nếu áp dụng cho bài toán phân loại, mô hình không dùng phép lấy trung bình mà sử dụng cơ chế “bầu chọn theo đa số” từ các cây phân loại độc lập, đồng thời tiêu chí chia cắt tại các nút được điều chỉnh để tối ưu hóa dựa trên việc giảm thiểu độ đo tạp chất Gini.

3.1.4 Hồi quy Logistic (Logistic Regression)

Có hai cách để suy nghĩ về mô hình hồi quy logistic (ví dụ: Long 1997; Maddala 1986). Cách đầu tiên bắt đầu với ý tưởng xác suất là một số nằm giữa 0 và 1, và nếu chúng ta chỉ sử dụng hồi quy tuyến tính trên xác suất, chúng ta có thể dễ dàng nhận được các dự đoán nằm ngoài khoảng này. Với hồi quy logistic, chúng ta áp dụng phép biến đổi logit cho các xác suất, có nghĩa là chúng ta

có một mô hình tuyến tính cho log-odds của sự thành công thay vì xác suất thành công. Điều này được thể hiện trong phương trình (1) trong đó y là biến phụ thuộc nhị phân, $P(y=1)$ là xác suất biến phụ thuộc nhận giá trị 1, x_s là các biến giải thích và β_s là tác động của chúng.

$$\ln \left(\frac{P(y=1)}{1-P(y=1)} \right) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 \quad (1)$$

Odds có thể được diễn giải là số lượng “thành công” kỳ vọng trên mỗi “thất bại”, và log-odds là logarit của giá trị đó. Log-odds không có giới hạn trên hay giới hạn dưới, do đó một mô hình tuyến tính sẽ không đưa ra những dự đoán không hợp lệ. Hơn nữa, các hệ số khi được lấy hàm mũ có thể được diễn giải trực tiếp thành các tỷ số odds, tức là tỷ lệ mà theo đó odds thay đổi khi biến giải thích tăng lên một đơn vị.

Đây cũng có thể được xem là một cách khớp một đường cong “hình chữ S” biểu thị tác động của các biến giải thích lên xác suất thành công, vì phương trình (1) có thể được viết lại thành phương trình (2). Bằng cách này, các xác suất dự đoán từ mô hình hồi quy logistic được đảm bảo luôn nằm trong giới hạn cho phép của xác suất.

$$P(y=1) = \frac{\exp(\beta_0 + \beta_1 x_1 + \beta_2 x_2)}{1 + \exp(\beta_0 + \beta_1 x_1 + \beta_2 x_2)} \quad (2)$$

Cách thứ hai để nghĩ về hồi quy logistic là giả định rằng có một xu hướng tiềm ẩn không quan sát được đối với việc trải qua một sự kiện. Nếu xu hướng tiềm ẩn đó vượt qua một ngưỡng (thường là 0) thì sự kiện sẽ xảy ra. Những gì được quan sát thấy là sự kiện đó có xảy ra hay không, chứ không phải là bản thân xu hướng đó. Biến tiềm ẩn này bị ảnh hưởng bởi các biến giải thích và một sai số, như trong phương trình (3), trong đó y^* là xu hướng tiềm ẩn và ε là sai số.

$$y^* = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \varepsilon \quad (3)$$

Trong mô hình này, xác suất trải qua sự kiện chính là xác suất để xu hướng tiềm ẩn lớn hơn 0, theo phương trình (3) có thể được viết thành:

$$P(y=1) = P(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \varepsilon > 0) \quad (4)$$

Vì cả các hệ số β_s và các biến x_s đều cố định, phần không chắc chắn duy nhất là sai số. Hệ quả là phương trình (4) có thể được viết lại thành:

$$P(y=1) = P(\varepsilon > -[\beta_0 + \beta_1 x_1 + \beta_2 x_2]) \quad (5)$$

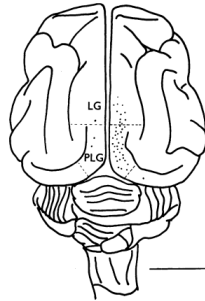
Xác suất để một biến ngẫu nhiên, trong trường hợp này là ε , lớn hơn một số nào đó có thể được suy ra từ hàm phân phối tích lũy. Nếu chúng ta giả định rằng sai số tuân theo phân phối logistic với trung bình bằng 0 và phương sai bằng $\frac{\pi^2}{3}$, thì phương trình (5) sẽ trở thành phương trình (2), từ đó dẫn đến hồi quy logistic.

3.1.5 Mạng nơ-ron tích chập (Convolutional Neural Networks - CNN)

Kiến trúc CNN ban đầu được Y. LeCun giới thiệu vào năm 1989 mang ý tưởng cốt lõi từ việc mô phỏng trực tiếp hệ thống thị giác sinh học của vỏ não thị giác ở động vật. Kể từ đó, CNN đã trở thành “xương sống” cho hàng loạt hệ thống thị giác máy tính hiện đại.

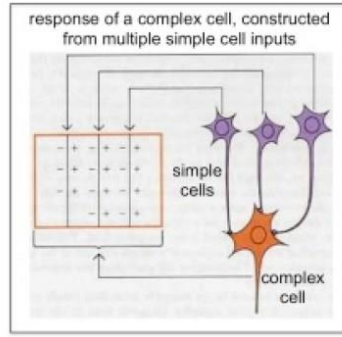
Về mặt bản chất, luồng thuật toán được triển khai trong đồ án này kế thừa hai nguyên lý cơ bản từ sinh học và toán học:

- *Cơ chế phân tích thị giác phân tầng*: Năm 1962, D.H. Hubel và T.N. Wiesel công bố một nghiên cứu mang tính bước ngoặt dựa trên các thí nghiệm ghi lại hoạt động điện của các tế bào đơn lẻ trong vỏ não thị giác của loài mèo.



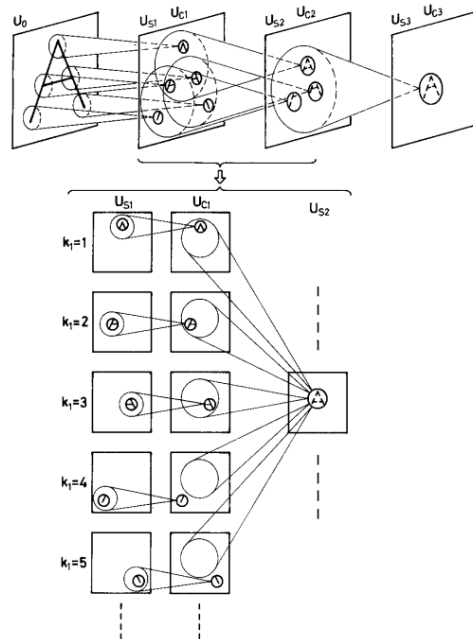
Hình 45: Sơ đồ mặt lưng của não mèo, thể hiện các điểm đi vào của 45 lần chọc vi điện cực. Các vết chọc nằm giữa các đường nét đứt là những vị trí mà các tế bào có trường cảm thụ nằm trong hoặc gần vùng trung tâm võng mạc

- Hai người đã phát hiện ra hệ thống thị giác không nhận diện toàn bộ sự vật cùng một lúc, mà có sự phân chia nhiệm vụ phức tạp theo tầng:
 - + Tế bào đơn giản: Là những tế bào ở tầng thấp, không phản ứng với ánh sáng khuếch tán hay các đốm sáng tròn thông thường. Thay vào đó, “vùng tiếp nhận” của chúng được chia thành các vùng kích thích và ức chế song song nhau. Đồng nghĩa với việc, chúng chỉ phát ra tín hiệu mạnh nhất khi nhìn thấy các chi tiết hình học cơ bản nhất như đường viền (edge), khe sáng hẹp (slit), hoặc một dải tối (dark bar) có hướng cực kỳ chính xác (ví dụ: góc dọc, góc ngang, hoặc nằm chéo)
 - + Tế bào phức tạp: Ở tầng cao hơn, các tế bào phức tạp cũng phản ứng với các đường viền hay khe sáng có hướng cụ thể giống như tế bào đơn giản. Song, điểm khác biệt là chúng phản ứng trên một diện tích rộng hơn nhiều và không quan tâm đến vị trí chính xác của đường nét đó nằm ở đâu trong vùng nhìn.
 - + Sự tổng hợp phân tầng: Hubel và Wiesel đưa ra giả thuyết rằng nhận thức là một chuỗi dây chuyền. Các tế bào phức tạp thực chất nhận tín hiệu đầu vào từ một nhóm nhiều tế bào đơn giản. Quá trình này cứ thế tiếp tục lên các tầng cao hơn: Tế bào đơn giản phát hiện đường thẳng/góc cạnh → tế bào phức tạp → tế bào siêu phức tạp giúp sinh vật nhận diện những đặc trưng phức tạp hơn



Hình 46: Nguyên lý sinh học của Hubel và Wiesel

Nhà nghiên cứu Kunihiko Fukushima là người đầu tiên dịch nguyên lý sinh học của Hubel và Wiesel thành một mạng nơ-ron đa tầng mang tên “Neocognitron” vào năm 1980. Mạng lưới này có một cấu trúc tương tự như mô hình phân cấp của hệ thành kinh thị giác do Hubel và Wiesel đề xuất. Mạng bao gồm một lớp đầu vào đóng vai trò như một mảng tế bào cảm quang theo sau là một chuỗi kết nối dạng tầng của một số cấu trúc module, mỗi module bao gồm hai lớp tế bào được kết nối nối tiếp nhau. Lớp đầu tiên của mỗi module bao gồm các “tế bào S” (S-cells), thể hiện các đặc điểm tương tự như tế bào đơn giản hoặc tế bào siêu phức tạp bậc thấp. Lớp thứ hai bao gồm các “tế bào C” (C-cells) tương tự như tế bào phức tạp hoặc tế bào siêu phức tạp bậc cao. Fukushima thiết kế mạng lưới sao cho các tế bào ở tầng càng sâu thì vùng tiếp nhận càng lớn. Các tế bào S ở tầng đầu tiên chỉ phản ứng với các đường nét cơ bản. Khi tiến sâu hơn, một tế bào S ở tầng sau sẽ nhận tín hiệu từ nhiều tế bào C ở tầng trước, từ đó nó có khả năng tổng hợp các đường nét thành các đặc trưng hình khối. Ví dụ, nó chỉ phát tín hiệu khi các đường gạch chéo kết hợp với nhau thành cấu trúc hình đỉnh chữ “A”).

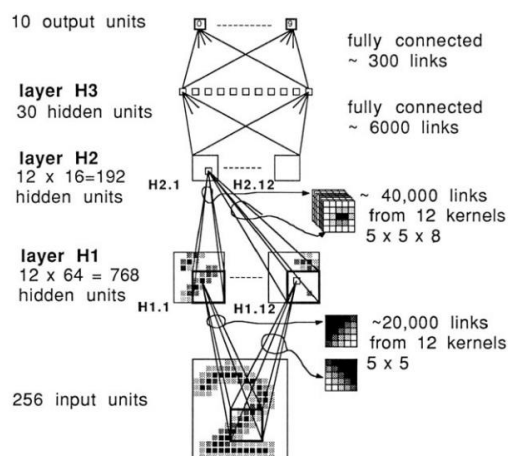


Hình 47: Một ví dụ về các mối liên kết giữa các tế bào và phản hồi của các tế bào sau khi hoàn thành quá trình tự tổ chức.

Quá trình này giải thích cơ chế hệ thống thần kinh loại bỏ sự thay đổi về vị trí hoặc biến dạng hình học nhỏ, từ đó nhận diện được mô hình tổng thể.

Kế thừa nền tảng sinh học từ mô hình của Hubel & Wiesel và kiến trúc mạng Neocognitron, Yann LeCun cùng các cộng sự đã phát triển thành công mạng nơ-ron tích chập (CNN) ứng dụng thuật toán lan truyền ngược vào mạng nơ-ron để nhận dạng chữ số viết tay. Cơ chế phân tích thị giác phân tầng được thể hiện qua ba đặc tính:

- *Trích xuất đặc trưng cục bộ*: Các tế bào thần kinh ở lớp đầu tiên được thiết kế để chỉ tiếp nhận thông tin đầu vào từ một vùng lân cận có kích thước 5×5 trên mặt phẳng đầu vào.
- *Tổ hợp đặc trưng phân tầng*: Dựa trên các nguyên lý kinh điển về nhận dạng mẫu thị giác, mô hình trích xuất các đặc trưng cục bộ như đường thẳng, góc cạnh,..., sau đó kết hợp chúng lại ở các lớp sâu hơn (layers H1, H2, H3...) để tạo ra các đặc điểm (features) bậc cao hơn và phức tạp hơn.
- *Sự hình thành tự động các bộ lọc phỏng sinh học*: Sau khi được huấn luyện trên dữ liệu, hệ thống có khả năng tự động học các hệ số để hình thành nên các hạt nhân tích chập. Một số hạt nhân do mạng tổng hợp ra hoạt động như các bộ dò đặc trưng có cấu trúc và chức năng tương đồng đáng kinh ngạc với những cơ chế được tìm thấy trong hệ thống thị giác sinh học mà Hubel và Wiesel đã phát hiện.



Hình 48: Sơ đồ mô phỏng cơ chế trích xuất đặc trưng cục bộ và xử lý phân tầng trong kiến trúc mạng CNN

Kiến trúc mạng CNN mô phỏng chính xác cơ chế này. Các lớp tích chập (Convolutional layers) đầu tiên sẽ đóng vai trò như các tế bào thần kinh nhạy cảm với các đường viền (ví dụ: vẩy rần, sọc ngang). Khi tiến sâu hơn vào mạng lưới, các lớp tiếp theo sẽ tổng hợp các đường nét này thành các “đặc trưng ngữ nghĩa” cấp cao (ví dụ: hình dáng đầu tam giác của rần lục, hoa văn chữ U trên cổ mang). Hình minh họa bên dưới thể hiện cách một mô hình học sâu dần dần bóc tách và nhận diện các đặc điểm thị giác quan trọng để phân biệt rần độc và không độc.

3.1.6 Label encoding

Về bản chất, biến phụ thuộc thường chịu tác động không chỉ bởi các biến thang đo tỷ lệ, mà còn bởi các biến định tính. Song, nhiều thuật toán Machine Learning và hầu hết toàn bộ các kiến trúc

Deep Learning sở hữu một điểm yếu cố hữu: Không thể xử lý trực tiếp dữ liệu văn bản ở dạng thô. Nhiều mô hình xử lý ngôn ngữ hiện đại dựa trên **đại số tuyến tính** và các **mô hình không gian vector** để biểu diễn ý nghĩa của từ ngữ mà văn bản thô vốn chỉ là những chuỗi ký tự rời rạc và không mang giá trị số tự thân, do đó cần phải được chuyển đổi để máy tính có thể thực hiện các phép toán. Thách thức này càng trở nên phức tạp do tính mơ hồ đặc trưng của ngôn ngữ ở nhiều cấp độ, từ hình thái, cú pháp cho đến ngữ nghĩa sâu xa. Đơn cử là ví dụ “I made her duck”, một cấu trúc đơn giản nhưng có thể được diễn giải theo ít nhất năm tầng nghĩa khác nhau tùy vào ngữ cảnh:

(1.5) I cooked waterfowl for her.

(1.6) I cooked a waterfowl belonging to her.

(1.7) I created the (plaster?) duck she owns.

(1.8) I caused her to quickly lower her head or body.

(1.9) I waved my magic wand and turned her into

Nếu chỉ dừng lại ở dạng dữ liệu thô, các mô hình máy học sẽ hoàn toàn bất lực trong việc xác định cấu trúc ngôn ngữ chính xác và phân giải sự nhập nhằng này. Vì lẽ đó, việc sử dụng các kỹ thuật mã hóa để chuyển đổi các biến phân loại này thành các con số là điều bắt buộc.

Một trong những phương pháp nền tảng để thực hiện quá trình này là Label Encoding. Trong kỹ thuật Label Encoding, mỗi nhãn dạng chuỗi sẽ được chuyển đổi sang một giá trị số nguyên. Mã hóa nhãn (Label encoding) được áp dụng cho các biến phân loại có thứ tự, và nó ánh xạ mỗi phân lớp thành một số nguyên bắt đầu từ 0. Tuy nhiên, một hạn chế lớn của kỹ thuật này là khi áp dụng lên các biến phân loại không có thứ tự nó vô tình tạo ra thêm tính thứ tự và khoảng cách cho các biến phân loại vốn dĩ ban đầu không có tính chất này.

Cụ thể, Label Encoding f_L ánh xạ phân lớp c_i sang các số nguyên từ 0 cho đến $N - 1$

$$CV : \begin{bmatrix} c_1 \\ c_2 \\ \dots \\ c_i \\ \dots \\ c_N \end{bmatrix}_{N \times 1} \xrightarrow{f_L} NV : \begin{bmatrix} 0 \\ 1 \\ \dots \\ i - 1 \\ \dots \\ N - 1 \end{bmatrix}_{N \times 1}$$

Hình 49: Label encoding

Trong đó:

- CV đại diện cho tập nhãn gốc của bạn $S = S_1, S_2, \dots, S_c$
- NV là vector đích chứa các giá trị số nguyên yi 0,1, ..., $C - 1$

f_L là phép ánh xạ song ánh đảm bảo mỗi nhãn chỉ tương ứng với một số nguyên duy nhất và ngược lại.

3.1.7 Confusion matrix

Confusion matrix hay bản dịch tiếng việt là ma trận nhầm lẫn, là một phương pháp giúp đánh giá kết quả của các bài toán phân loại bằng việc xét từ những chỉ số độ chuẩn xác (Precision) và khả năng thu hồi (Recall) trên từng nhãn dữ liệu. Cấu trúc của ma trận được hình thành từ 4 trọng số cốt lõi cho mỗi phân lớp, bao gồm:

		Actual Values	
		Positive (1)	Negative (0)
Predicted Values	Positive (1)	TP	FP
	Negative (0)	FN	TN

Hình 50: <https://viblo.asia/p/tim-hieu-ve-confusion-matrix-trong-machine-learning-Az45bRpo5xY>

		Model dự đoán	
		Positive	Negative
Thực tế	Positive	True Positive (Dự đoán Đúng là Positive)	False Negative (Dự đoán Sai là Negative)
	Negative	False Positive (Dự đoán Sai là Positive)	True Negative (Dự đoán Đúng là Negative)

Hình 51: <https://miai.vn/2020/06/16/oanh-gia-model-ai-theo-cach-mi-an-lien-chuong-2-precision-recall-va-f-score/>

3.1.8 Khái niệm Precision

Công thức:

$$\text{Precision} = \frac{\text{TruePositive}}{\text{TruePositive} + \text{FalsePositive}}$$

Như vậy Precision là tỷ lệ giữa số mẫu tính ra là True Positive (TP) với tổng số mẫu được phân loại là Positive. Với giá trị trong khoảng $0 < \text{Precision} \leq 1$, tỷ lệ này càng tiến gần về 1 thì mức độ chuẩn xác của các dự đoán do mô hình tìm ra càng cao. Chỉ số Precision cần được đặc biệt ưu tiên đối với những bài toán mà hậu quả của lỗi Loại I (False Positive - dự đoán sai thành nhãn tích cực) gây ra những rủi ro hoặc thiệt hại nghiêm trọng.

3.1.9 Khái niệm Recall

Chỉ số này trả lời cho câu hỏi: Trong tổng số tất cả các mẫu thực sự thuộc lớp Tích cực, mô hình đã phát hiện và nhận diện đúng được bao nhiêu phần trăm?

Công thức:

$$\text{Recall} = \frac{\text{TruePositive}}{\text{TruePositive} + \text{FalseNegative}}$$

Recall phản ánh tỷ lệ các điểm dữ liệu thực sự là Positive được mô hình nhận diện thành công. Một mô hình có Recall cao đồng nghĩa với việc tỷ lệ bỏ sót các trường hợp Positive (tức là lỗi False Negative) là rất thấp. Trong thực tế, việc tối ưu hóa Recall mang tính bắt buộc đối với những bài toán mà việc phân loại sai một mẫu Tích cực thành Tiêu cực (False Negative) sẽ gây ra những hệ lụy nghiêm trọng.

3.1.10 Khái niệm F-1 Score

Có thể thấy, luôn có sự đánh đổi giữa Precision và Recall khi tăng chỉ số này thường vô tình làm giảm chỉ số kia. Để tìm ra điểm cân bằng tối ưu và đưa ra quyết định chọn model một cách khách quan, tham số F1-Score được ứng dụng. F-1 Score là sự kết hợp của Precision và Recall.

Công thức:

$$F1 = 2 \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

F-1 Score chỉ chạm ngưỡng lý tưởng khi cả Precision và Recall đều đạt điểm tối đa, sự thay đổi giảm của dù 1 trong 2 chỉ số này đều kéo theo sự giảm của F-1 Score. Nhờ cơ chế này, F-score mang đến một thước đo đo lường hiệu năng vô cùng chuẩn xác, đặc biệt là khi đối mặt với tình trạng mất cân bằng dữ liệu.

3.1.11 Khái niệm ROC Curve

Biểu đồ này vẽ ra một đường cong thể hiện sự tương quan giữa True Positive Rate (TPR - tỷ lệ bắt đúng bệnh) và False Positive Rate (FPR - tỷ lệ báo động giả) khi chúng ta liên tục thay đổi các ngưỡng dự đoán (threshold).

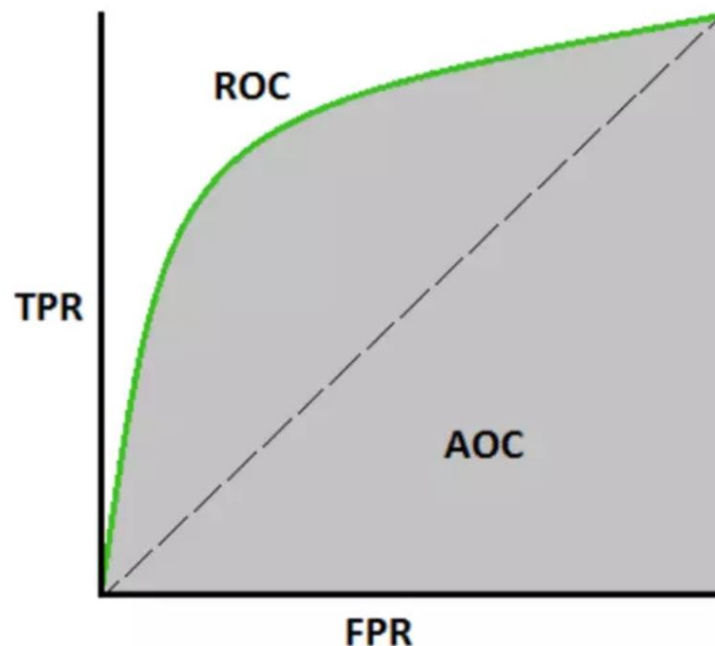
Công thức :

$$TPR = \frac{TruePositive}{TruePositive + FalsePositive}$$

$$FPR = \frac{False Positive}{True Negative + False Negative}$$

3.1.12 Khái niệm AUC

AUC hay Area Under the Curve là đại lượng được tính toán trực tiếp từ đường cong ROC, đóng vai trò là thước đo tổng quát cho năng lực phân loại của mô hình. AUC đại diện cho toàn bộ phần diện tích nằm trên trục hoành và bên dưới đường cong ROC. Bằng cách quy chiếu đồ thị thành một con số duy nhất có giá trị dao động trong đoạn [0, 1].



Hình 52: Minh họa AUC

Khi diện tích này càng được mở rộng, đường cong ROC sẽ có xu hướng tiến sát về phía biên trên của đồ thị (đường tiệm cận $y=1$), minh chứng cho một bộ phân loại có năng lực nhận diện tốt. Trái lại, nếu đường ROC bám sát vào đường phân giác vắt ngang đồ thị (đường thẳng nối từ gốc tọa độ $(0, 0)$ đến điểm $(1, 1)$), mô hình lúc này không mang lại giá trị dự đoán thực tiễn nào. Thay vào đó, kết quả phân loại của nó chỉ tương đương với một phép thử ngẫu nhiên.

3.2 Minh họa bằng Python

3.2.1 Xử lý ảnh lỗi, nhiễu và định dạng lại kích thước

Mở đầu bằng việc import thư viện os, numpy, cv2, Matplotlib, seaborn, GridSearchCV, PCA, SVC

```
import os
import cv2
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from matplotlib import colors
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.decomposition import PCA
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.svm import SVC
from sklearn.metrics import classification_report
```

Tiếp theo, tiến hành khai báo tham số ban đầu và chuẩn hóa ảnh về kích thước thước 64×64 pixels

```
categories = ['Venomous', 'Non Venomous']
IMG_SIZE = 64
```

Kế đến, em tiến hành xây dựng hàm tải dữ liệu từ drive để thuận tiện hơn cho quá trình đọc, tiến xử lý dữ liệu hình ảnh từ thư mục lưu trữ.

```
def load_data_from_drive(directory):
    data, labels = [], []
    for category in categories:
        path = os.path.join(directory, category)
        if not os.path.exists(path):
            print(f"Thư mục {path} chưa được upload")
            continue
        class_num = categories.index(category)
```

Hàm này đóng vai trò chuẩn bị đầu vào cho các bước giảm chiều dữ liệu và huấn luyện mô hình phía sau. Quá trình hoạt động của hàm theo trình tự: (1) Khởi tạo hai danh sách rỗng data và labels để lưu trữ tương ứng các vector đặc trưng và nhãn phân loại của ảnh; (2) Sử dụng vòng lặp for để đi lần lượt từng thư mục con. Nếu hệ thống phát hiện đường dẫn thư mục không tồn tại, chương trình sẽ in ra thông báo cảnh báo và tiếp tục với thư mục kế tiếp thông qua lệnh `continue` giúp bỏ qua các lỗi đường dẫn và tránh việc chương trình bị ngắt. (3) Nhãn của ảnh ban đầu là dạng chuỗi sẽ được chuyển đổi thành số nguyên class_num thông qua index của nó trong danh sách categories.

```

for img_name in os.listdir(path):
    try:
        img_path = os.path.join(path, img_name)
        img = cv2.imread(img_path)
        if img is not None:
            img = cv2.resize(img, (IMG_SIZE, IMG_SIZE))
            data.append(img.flatten())
            labels.append(class_num)
        except: continue
return np.array(data), np.array(labels)

```

Ở phần này, thuật toán tiếp tục duyệt từng file ảnh. Hàm `cv2.imread()` của thư viện OpenCV được sử dụng để đọc ma trận điểm ảnh. Khối lệnh `try...except` được áp dụng nhằm bỏ qua những file bị lỗi hoặc không đúng định dạng ảnh. Và để đảm bảo tính đồng nhất về kích thước đầu vào cho mô hình, ảnh được đưa về một kích thước cố định bằng hàm `cv2.resize(img, (IMG_SIZE, IMG_SIZE))`. Thay vì giữ nguyên cấu trúc ma trận 2D hoặc 3D, phương thức `img.flatten()` sẽ duỗi toàn bộ điểm ảnh thành một vector đặc trưng 1 chiều. Bước này được đưa vào để dữ liệu có thể tương thích với thuật toán giảm chiều dữ liệu PCA và các mô hình học máy cơ sở. Cuối cùng, hai danh sách `data` và `labels` được chuyển đổi thành cấu trúc mảng của NumPy và trả về nhằm tối ưu hóa bộ nhớ và tăng tốc độ xử lý cho các phép toán đại số tuyến tính trong giai đoạn huấn luyện mô hình.

```

from google.colab import drive
drive.mount('/content/drive')

```

Để đọc dữ liệu từ Google Drive vào Google Colab, em tiến hành gọi lệnh `drive.mount()` để cấp quyền truy cập, giúp môi trường tính toán Google Colab liên kết trực tiếp với không gian lưu trữ Google Drive cá nhân.

```

train_dir = '/content/drive/MyDrive/train'
test_dir = '/content/drive/MyDrive/test'
X_train, y_train = load_data_from_drive(train_dir)
X_test, y_test = load_data_from_drive(test_dir)

```

Tại đây em gán đường dẫn cho hai biến `train_dir` và `test_dir`, trỏ đến chính xác các thư mục chứa tập dữ liệu ảnh các loài rắn độc đã được phân chia sẵn cho mục đích huấn luyện và kiểm thử. Sau đó, kích hoạt hàm `load_data_from_drive` đã được định nghĩa ở phần trước để tiến hành quét qua các đường dẫn này, thực hiện việc đọc mảng điểm ảnh, đồng bộ kích thước và trả về các ma trận đặc trưng (`X_train`, `X_test`) cùng với các vector nhãn phân loại tương ứng (`y_train`, `y_test`).

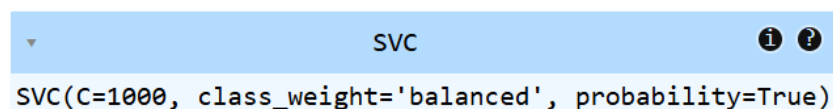
3.2.2 Giảm chiều dữ liệu bằng PCA

```
pca = PCA(n_components=100, random_state=42)
X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)

model = SVC(kernel='rbf', C=1000, probability=True, class_weight='balanced')

model.fit(X_train_pca, y_train)
```

Đến đây, em tiến hành khởi tạo thuật toán PCA với siêu tham số `n_components=100`, thực hiện nhiệm vụ nén các ma trận điểm ảnh phức tạp xuống còn 100 thành phần cốt lõi nhất mang nhiều thông tin nhất, kết hợp `random_state=42` nhằm đảm bảo kết quả luôn đồng nhất ở các lần chạy khởi tạo. Kế đến, phương thức `fit_transform` được áp dụng lên tập huấn luyện để mô hình vừa học cấu trúc phương sai vừa biến đổi dữ liệu, trong khi tập kiểm thử chỉ áp dụng lệnh `transform` dựa trên không gian đã học từ tập huấn luyện, giúp ngăn chặn triệt để hiện tượng rò rỉ dữ liệu. Mô hình máy học vector hỗ trợ (SVM) được khởi tạo sử dụng nhân phi tuyến Radial Basis Function (`kernel='rbf'`) để có thể xử lý các ranh giới phân loại hình thái phức tạp giữa các loài rắn. Đáng chú ý, siêu tham số `C=1000` tạo ra một ranh giới quyết định vô cùng khắt khe nhằm giảm thiểu tối đa các điểm dữ liệu phân lớp sai, kết hợp với `class_weight='balanced'` giúp mô hình tự động phạt nặng hơn đối với các lớp thiểu số để giải quyết triệt để tình trạng mất cân bằng dữ liệu giữa số lượng ảnh giữa nhóm rắn độc và không độc. Thêm vào đó, tham số `probability=True` được kích hoạt để cho phép mô hình xuất ra độ tin cậy của từng dự đoán. Kết thúc đoạn mã này, hàm `fit` được gọi ra, cho phép mô hình SVM tìm kiếm siêu mặt phẳng phân tách tối ưu nhất trên không gian đặc trưng `X_train_pca` đã được thu gọn, đối chiếu với nhãn `y_train` để nhận diện chính xác các nhóm loài.



Hình 53: Thông tin cấu hình của mô hình

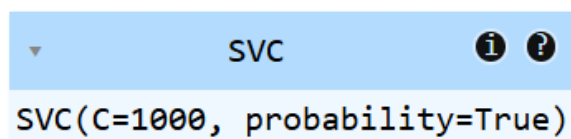
Mô hình được thiết lập với mức phạt lỗi rất cao `C=1000`, buộc chương trình phải hạn chế tối đa các điểm phân loại sai. Điều này giúp tăng độ chính xác trên tập huấn luyện, nhưng đồng thời cũng tiềm ẩn nguy cơ `overfitting` nếu dữ liệu có nhiễu. Do dữ liệu huấn luyện không cân bằng khi có sự chênh lệch giữa số lượng ảnh rắn độc và không độc nên chương trình sử dụng `class_weight='balanced'` để giúp mô hình tự động điều chỉnh trọng số, tăng mức phạt đối với các lớp thiểu số, từ đó cải thiện khả năng nhận diện các nhóm dữ liệu ít xuất hiện. `probability=True` cho phép mô hình xuất ra xác suất dự đoán cho từng lớp, giúp đánh giá mức độ tin cậy của kết quả.

3.2.3 Thực hiện bài toán phân loại bằng SVM

```
model = SVC(kernel='rbf', C=1000, probability=True)
model.fit(X_train_pca, y_train)
```

Mô hình phân loại được khởi tạo bằng lớp SVC với hàm nhân phi tuyến kernel='rbf', cho phép xử lý hiệu quả các ranh giới phân tách phức tạp trong không gian đặc trưng. Tham số C=1000 được thiết lập ở mức cao nhằm phạt nặng các điểm sai từ đó giảm thiểu sai số phân loại trên tập huấn luyện và tạo ra một ranh giới quyết định chặt chẽ hơn giữa các lớp dữ liệu. Đồng thời, probability=True được kích hoạt để mô hình có thể xuất ra xác suất dự đoán cho từng lớp, hỗ trợ việc đánh giá độ tin cậy của kết quả.

Sau đó, phương thức fit(X_train_pca, y_train) được sử dụng để huấn luyện mô hình trên tập dữ liệu đã được giảm chiều bằng PCA. Trong quá trình này, SVM sẽ tìm kiếm siêu phẳng tối ưu trong không gian đặc trưng nhằm phân tách các lớp dữ liệu một cách hiệu quả nhất, dựa trên các nhãn tương ứng trong y_train.



Hình 54: Kết quả đầu ra cho thấy mô hình đã được khởi tạo thành công

3.2.4 Đánh giá mô hình SVM

```
y_pred = model.predict(X_test_pca)
print("\nBÁO CÁO PHÂN LOẠI:")
print(classification_report(y_test, y_pred, target_names=categories))
```

Sau khi hoàn tất quá trình huấn luyện mô hình SVC, phương thức predict() được sử dụng để sinh ra các nhãn dự đoán y_pred trên tập dữ liệu kiểm thử X_test_pca. Bước này nhằm đánh giá khả năng tổng quát hóa của mô hình đối với các dữ liệu chưa từng thấy. Kế đến, hàm classification_report được áp dụng để cung cấp một báo cáo đánh giá chi tiết về hiệu suất phân loại. Báo cáo này bao gồm các chỉ số quan trọng như Precision, Recall, F1-score và Accuracy. target_names=categories giúp hiển thị tên các lớp một cách trực quan thay vì chỉ sử dụng nhãn số, hỗ trợ quá trình phân tích và diễn giải kết quả rõ ràng hơn.

BÁO CÁO PHÂN LOẠI:				
	precision	recall	f1-score	support
Venomous	0.63	0.74	0.68	141
Non Venomous	0.64	0.52	0.57	128
accuracy			0.63	269
macro avg	0.63	0.63	0.62	269
weighted avg	0.63	0.63	0.63	269

Kết quả từ báo cáo phân loại cho thấy mô hình đạt độ chính xác tổng thể là 63%, một hiệu suất ở mức trung bình đối với bài toán phân loại rắn độc và không độc. Chi tiết hơn, mô hình thể hiện khả năng nhận diện lớp Venomous tốt hơn với recall đạt 0.74, tức là phần lớn các mẫu rắn độc đã được phát hiện đúng. Tuy nhiên, đối với lớp Non Venomous, recall chỉ đạt 0.52, chứng tỏ mô hình vẫn bỏ sót một lượng đáng kể các mẫu không độc và có xu hướng nhầm lẫn chúng thành rắn độc. Các chỉ số precision và F1-score giữa hai lớp không chênh lệch quá lớn, cùng với giá trị macro average và weighted average đều xấp xỉ 0.63, cho thấy mô hình chưa bị mất cân bằng nghiêm trọng nhưng vẫn chưa đạt được sự phân biệt tối ưu giữa các lớp. Tổng thể, mô hình có xu hướng

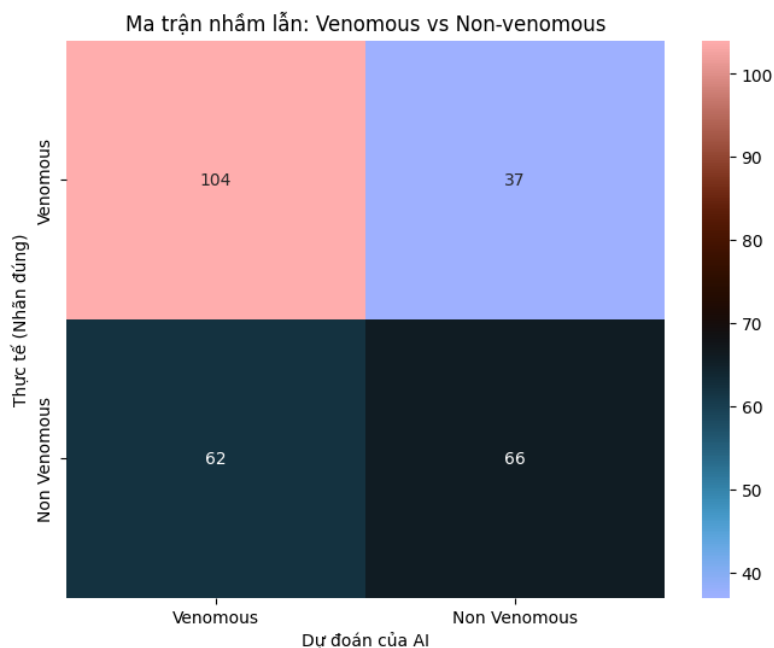
ưu tiên phát hiện rắn độc hơn, do đó cần được cải thiện thêm để nâng cao khả năng nhận diện đồng đều và tăng tính tổng quát hóa trên dữ liệu kiểm thử.

```
from sklearn.metrics import confusion_matrix
import seaborn as sns

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='berlin',
            xticklabels=categories, yticklabels=categories)
plt.xlabel('Dự đoán của AI')
plt.ylabel('Thực tế (Nhãn đúng)')
plt.title('Ma trận nhầm lẫn: Venomous vs Non-venomous')
plt.show()
```

Ma trận nhầm lẫn được xây dựng nhằm trực quan hóa hiệu suất phân loại của mô hình qua việc so sánh giữa nhãn dự đoán và nhãn thực tế. Hàm `confusion_matrix` từ thư viện `sklearn.metrics` được sử dụng để xây dựng ma trận nhầm lẫn, nhằm so sánh giữa nhãn thực tế (`y_test`) và nhãn dự đoán (`y_pred`), trong đó mỗi phần tử của ma trận thể hiện số lượng mẫu được dự đoán đúng hoặc sai theo từng lớp. Kết quả này sau đó được trực quan hóa bằng thư viện `seaborn` thông qua biểu đồ nhiệt, với kích thước hình được thiết lập bằng `plt.figure(figsize=(8, 6))` để đảm bảo hiển thị rõ ràng. Hàm `sns.heatmap()` được sử dụng với các tham số như `annot=True` để hiển thị giá trị trên từng ô, `fmt='d'` để định dạng số nguyên, `cmap='berlin'` để lựa chọn bảng màu, và `xticklabels`, `yticklabels` để gán nhãn cho trục dự đoán và trục thực tế. Cuối cùng, các lệnh `plt.xlabel()`, `plt.ylabel()` và `plt.title()` được dùng để đặt tên cho các trục và tiêu đề biểu đồ, trước khi `plt.show()` hiển thị kết quả ra màn hình.



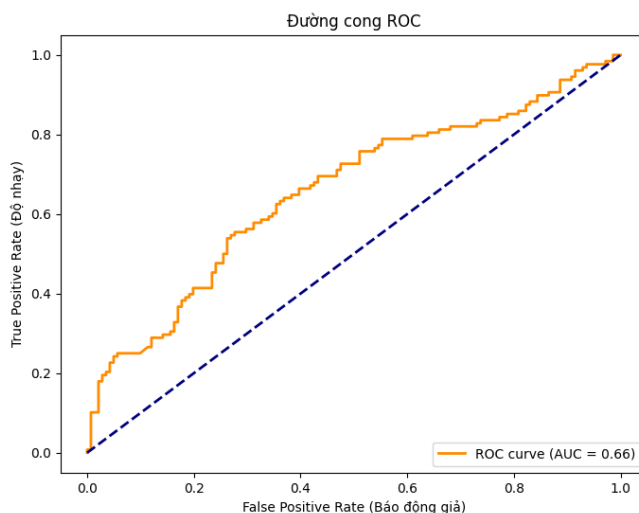
Hình 55: Ma trận nhầm lẫn: Venomous vs Non-venomous

Ma trận nhầm lẫn cho thấy mô hình có khả năng phân loại ở mức trung bình với sự chênh lệch rõ giữa hai loại lỗi. Cụ thể, có 104 mẫu rắn độc (Venomous) được dự đoán đúng, trong khi 37 mẫu rắn độc bị nhầm thành không độc, cho thấy mô hình vẫn bỏ sót một phần đáng kể các trường hợp nguy hiểm. Đồng thời, số lượng mẫu không độc bị nhầm thành rắn độc cũng tương đối cao, phản ánh xu hướng mô hình thiên về dự đoán lớp rắn độc. Điều này đã xác nhận kết quả trước đó khi recall của lớp Venomous cao hơn lớp còn lại. Nhìn chung, mô hình có xu hướng ưu tiên phát hiện rắn độc, nhưng vẫn cần cải tiến để giảm nhầm lẫn, nâng cao độ chính xác, đặc biệt đối với lớp không độc.

```
from sklearn.metrics import roc_curve, auc
y_probs = model.predict_proba(X_test_pca)[: , 1]
fpr, tpr, _ = roc_curve(y_test, y_probs)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (AUC = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlabel('False Positive Rate (Báo động giả)')
plt.ylabel('True Positive Rate (Độ nhạy)')
plt.title('Đường cong ROC')
plt.legend(loc="lower right")
plt.show()
```

Hàm `roc_curve` và `auc` từ thư viện `sklearn.metrics` được dùng để đánh giá hiệu suất phân loại của mô hình SVC (scikit-learn) thông qua đường cong ROC. Trước hết, phương thức `predict_proba(X_test_pca)` được sử dụng để lấy xác suất dự đoán của mô hình trên tập kiểm thử, trong đó `[:, 1]` trích xuất xác suất thuộc lớp dương (ví dụ: rắn độc). Tiếp theo, hàm `roc_curve(y_test, y_probs)` tính toán các giá trị False Positive Rate (FPR) và True Positive Rate (TPR) tương ứng với nhiều ngưỡng phân loại khác nhau, từ đó mô tả mối quan hệ đánh đổi giữa tỷ lệ báo động giả và độ nhạy của mô hình. Giá trị AUC được tính bằng hàm `auc(fpr, tpr)`, phản ánh khả năng phân biệt giữa các lớp, AUC càng gần 1 thì mô hình càng tốt. Sau đó, biểu đồ được vẽ bằng `matplotlib`, trong đó đường cong ROC được thể hiện bằng `plt.plot(fpr, tpr, ...)`, còn đường chéo nét đứt biểu thị mô hình đoán ngẫu nhiên. Cuối cùng, các lệnh `plt.xlabel()`, `plt.ylabel()`, `plt.title()` và `plt.legend()` được sử dụng để hoàn thiện biểu đồ trước khi hiển thị bằng `plt.show()`.



Hình 56: Đường cong ROC

Đường cong ROC cho thấy mô hình chỉ phân biệt giữa hai lớp ở mức trung bình, với giá trị AUC xấp xỉ 0.66. Đường cong nằm phía trên đường mô hình đoán ngẫu nhiên, chứng tỏ mô hình có học được quy luật từ dữ liệu, tuy nhiên khoảng cách chưa lớn nên hiệu quả phân loại vẫn còn hạn chế. Ở nhiều ngưỡng, mô hình đạt được độ nhạy (TPR) tương đối khá nhưng đi kèm với tỷ lệ báo động giả (FPR) cũng tăng đáng kể, cho thấy sự đánh đổi chưa tối ưu giữa việc phát hiện đúng và hạn chế nhầm lẫn.

```
import matplotlib.pyplot as plt
import numpy as np

# 1. Chọn 10 chỉ số ngẫu nhiên từ tập Test
random_indices = np.random.choice(len(X_test), 10, replace=False)

plt.figure(figsize=(20, 10))
print("> ĐANG KIỂM TRA 10 MẪU NGẪU NHIÊN...")

for i, idx in enumerate(random_indices):
    img_flat = X_test[idx]
    img_show = img_flat.reshape(64, 64, 3).astype('uint8')
    img_show = cv2.cvtColor(img_show, cv2.COLOR_BGR2RGB)
    true_label = categories[y_test[idx]]
    pred_label = categories[y_pred[idx]]
    color = 'green' if true_label == pred_label else 'red' # Xanh nếu đúng, Đỏ nếu sai
    plt.subplot(2, 5, i + 1)
    plt.imshow(img_show)
    plt.title(f"Thực tế: {true_label} \n AI đoán: {pred_label}", color=color, fontsize=12)
    plt.axis('off')

plt.tight_layout()
plt.show()
```

Để trực quan hóa kết quả dự đoán của mô hình trên một số mẫu dữ liệu ngẫu nhiên từ tập kiểm, `np.random.choice()` được dùng để chọn ngẫu nhiên 10 chỉ số từ tập `X_test` nhằm đảm bảo việc đánh giá mang tính đại diện. Sau đó, `plt.figure(figsize=(20, 10))` thiết lập kích thước khung hiển thị lớn để dễ quan sát nhiều ảnh cùng lúc. Trong vòng lặp `for`, mỗi ảnh được lấy ra dưới dạng vector phẳng (`img_flat`) và được chuyển lại về kích thước ảnh gốc (64, 64, 3) bằng `reshape`, đồng thời ép kiểu về `uint8` để hiển thị đúng màu. Do ảnh ban đầu được đọc theo định dạng BGR (từ OpenCV), hàm OpenCV `cv2.cvtColor()` được sử dụng để chuyển sang RGB, phù hợp với cách hiển thị của `matplotlib`.

Tiếp đến, nhãn thực tế (`true_label`) và nhãn dự đoán (`pred_label`) được lấy tương ứng từ `y_test` và `y_pred`, sau đó so sánh để xác định màu tiêu đề: xanh lá nếu dự đoán đúng, đỏ nếu sai. Hàm `plt.subplot(2, 5, i + 1)` chia khung hình thành lưới 2 hàng 5 cột để hiển thị 10 ảnh, và `plt.imshow()` dùng để vẽ từng ảnh. Tiêu đề của mỗi ảnh được hiển thị bằng `plt.title()` với thông tin nhãn thực tế và dự đoán, giúp dễ dàng đối chiếu kết quả. Cuối cùng, `plt.axis('off')` loại bỏ trục tọa độ để hình ảnh rõ ràng hơn, `plt.tight_layout()` căn chỉnh bố cục cho gọn gàng, và `plt.show()` hiển thị toàn bộ kết quả ra màn hình.



Hình 57: Kiểm tra 10 mẫu test

Kết quả cho thấy mô hình có khả năng phân loại đúng các mẫu điển hình, thể hiện qua những hình ảnh được đánh dấu màu xanh. Đặc biệt, các mẫu rắn có đặc trưng rõ ràng về màu sắc hoặc hoa văn thường được nhận diện chính xác. Tuy nhiên, vẫn tồn tại nhiều trường hợp dự đoán sai, chủ yếu là các mẫu rắn độc bị nhầm thành không độc, cho thấy mô hình còn gặp khó khăn khi xử lý các hình ảnh có đặc điểm hình thái không rõ ràng, góc chụp phức tạp hoặc chất lượng ảnh thấp.



Hình 58: Kiểm tra 10 mẫu khác

Một phép thử khác cũng làm lộ khuyết điểm của mô hình khi nhận diện nhóm không độc thành độc. Dữ liệu đầu vào đã được giảm chiều bằng Principal Component Analysis với số lượng thành phần hạn chế, dẫn đến việc mất đi một số đặc trưng quan trọng của ảnh. Điều này làm suy giảm khả năng phân biệt giữa các lớp, đặc biệt trong những trường hợp có sự tương đồng về hình thái. Nhìn chung, mô hình hoạt động ở mức chấp nhận được đối với các trường hợp rõ ràng, nhưng chưa đủ mạnh để xử lý hiệu quả các tình huống phức tạp hoặc dữ liệu nhiễu, do đó cần tiếp tục cải thiện.

3.2.5 Thực hiện bài toán phân loại bằng random forest

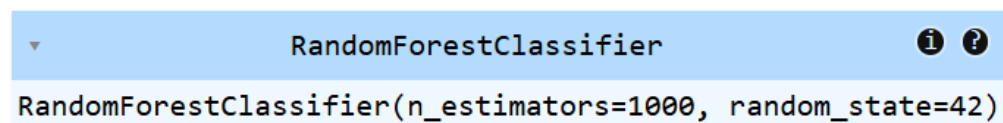
```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt
```

Tiến hành nhập các thư viện cần thiết như: RandomForestClassifier được dùng để tạo mô hình, các hàm classification_report và confusion_matrix dùng để đánh giá hiệu suất, còn Seaborn và Matplotlib hỗ trợ trực quan hóa kết quả.

```
rf_model = RandomForestClassifier(n_estimators=1000, random_state=42)
```

Khởi tạo mô hình Random Forest thông qua lớp RandomForestClassifier, trong đó tham số n_estimators=1000 xác định số lượng cây quyết định được sử dụng trong mô hình, giúp tăng độ ổn định và độ chính xác. Tham số random_state=42 được thiết lập nhằm đảm bảo tính tái lập của kết quả khi chạy lại chương trình.

```
rf_model.fit(X_train_pca, y_train)
```



```
RandomForestClassifier(n_estimators=1000, random_state=42)
```

Dòng hiển thị trên là kết quả biểu diễn của mô hình RandomForestClassifier sau khi được khởi tạo. Cụ thể, n_estimators=1000 cho biết mô hình sử dụng 1000 cây quyết định để thực hiện phân loại, giúp tăng độ chính xác và độ ổn định. Tham số random_state=42 được thiết lập nhằm đảm bảo kết quả có thể lặp lại khi chạy lại chương trình.

```
y_pred_rf = rf_model.predict(X_test_pca)
```

Phương thức predict được sử dụng để dự đoán lớp của từng mẫu trong X_test_pca, và kết quả được lưu vào biến y_pred_rf. Các giá trị dự đoán này sẽ được sử dụng trong các bước đánh giá hiệu suất mô hình tiếp theo.

	precision	recall	f1-score	support
Venomous	0.56	0.99	0.71	141
Non Venomous	0.90	0.15	0.26	128
accuracy			0.59	269
macro avg	0.73	0.57	0.48	269
weighted avg	0.72	0.59	0.50	269

Kết quả đánh giá cho thấy mô hình Random Forest đạt độ chính xác tổng thể khoảng 59%, thấp hơn so với mô hình SVM trước đó. Đối với lớp Venomous, mô hình có recall rất cao (0.99), cho thấy gần như tất cả các mẫu thuộc lớp này đều được nhận diện đúng. Tuy nhiên, precision chỉ đạt 0.56, nghĩa là có nhiều trường hợp bị dự đoán nhầm là Venomous.

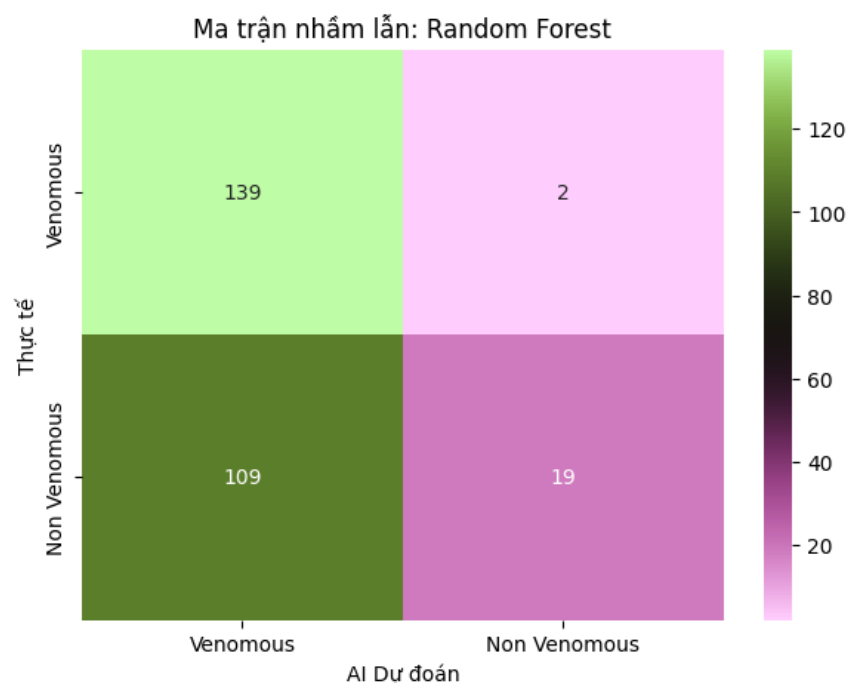
Ngược lại, đối với lớp Non Venomous, mô hình có precision cao (0.90) nhưng recall rất thấp (0.15), cho thấy mô hình gặp khó khăn trong việc phát hiện các mẫu không độc và bỏ sót phần lớn dữ liệu thuộc lớp này.

Giá trị f1-score của hai lớp có sự chênh lệch lớn (0.71 và 0.26), phản ánh sự mất cân bằng trong hiệu suất giữa hai lớp. Các chỉ số macro average và weighted average ở mức thấp, cho thấy hiệu quả tổng thể của mô hình chưa tốt.

Như vậy, nhìn chung thì mô hình Random Forest có xu hướng thiên lệch mạnh về lớp Venomous, dẫn đến khả năng phân loại không đồng đều. Điều này cho thấy cần cải thiện mô hình thông qua việc điều chỉnh tham số hoặc xử lý mất cân bằng dữ liệu để nâng cao hiệu suất.

5. VẼ MA TRẬN NHẦM LẪN CHO RANDOM FOREST

```
plt.figure(figsize=(7, 5))
cm_rf = confusion_matrix(y_test, y_pred_rf)
sns.heatmap(cm_rf, annot=True, fmt='d', cmap='magma',
             xticklabels=categories, yticklabels=categories)
plt.title('Ma trận nhầm lẫn: Random Forest')
plt.xlabel('AI Dự đoán')
plt.ylabel('Thực tế')
plt.show()
```



Từ ma trận nhầm lẫn, có thể thấy mô hình Random Forest có xu hướng dự đoán nghiêng mạnh về lớp Venomous. Cụ thể, trong số 141 mẫu thuộc lớp Venomous, có tới 139 mẫu được dự đoán chính xác và chỉ 2 mẫu bị nhầm lẫn, cho thấy khả năng nhận diện lớp này là rất tốt.

Tuy nhiên, đối với lớp Non Venomous, hiệu suất của mô hình lại rất thấp. Trong tổng số 128 mẫu, chỉ có 19 mẫu được dự đoán đúng, trong khi có tới 109 mẫu bị nhầm sang lớp Venomous. Điều này cho thấy mô hình gặp khó khăn nghiêm trọng trong việc phân biệt các mẫu không độc và có xu hướng quá nhạy cảm với lớp Venomous.

```
import matplotlib.pyplot as plt
import numpy as np
import cv2
random_indices = np.random.choice(len(X_test), 10, replace=False)

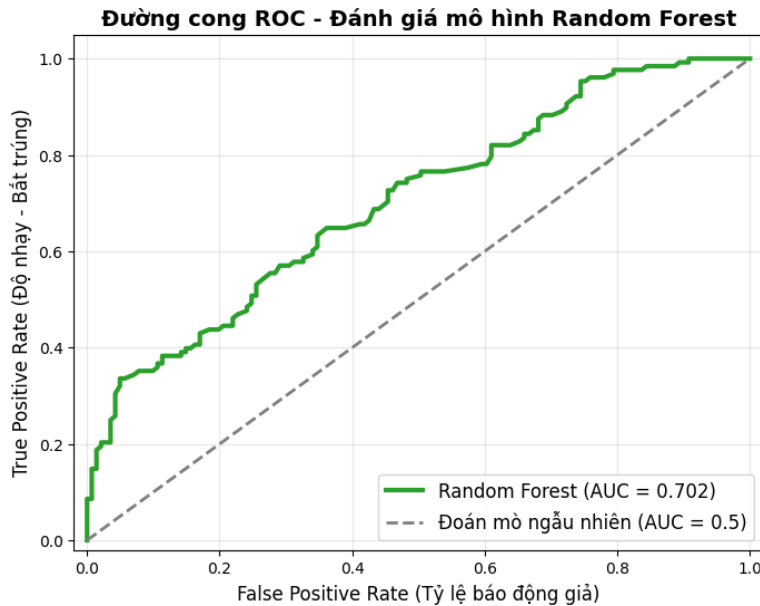
plt.figure(figsize=(20, 10))

for i, idx in enumerate(random_indices):
    img_flat = X_test[idx]

    img_show = img_flat.reshape(64, 64, 3).astype('uint8')
    img_show = cv2.cvtColor(img_show, cv2.COLOR_BGR2RGB)
    true_label = categories[y_test[idx]]
    pred_label = categories[y_pred[idx]]
    color = 'green' if true_label == pred_label else 'red'
    plt.subplot(2, 5, i + 1)
    plt.imshow(img_show)
    plt.title(f"Thực tế: {true_label}\nAI đoán: {pred_label}", color=color, fontsize=12, fontweight='bold')
    plt.axis('off')

plt.tight_layout()
plt.show()
```

Tổng thể, mô hình có độ nhạy rất cao đối với lớp Venomous nhưng lại đánh đổi bằng tỷ lệ báo động giả cao đối với lớp Non Venomous. Sự mất cân bằng này dẫn đến hiệu suất tổng thể không cao và làm giảm tính ứng dụng của mô hình trong thực tế.



Đường cong ROC - Đánh giá mô hình Random Forest

Từ đồ thị ROC, có thể thấy mô hình Random Forest có khả năng phân biệt giữa hai lớp Venomous và Non Venomous ở mức khá. Đường cong ROC nằm rõ ràng phía trên đường chéo (đại diện cho mô hình đoán ngẫu nhiên), cho thấy mô hình hoạt động tốt hơn đáng kể so với việc dự đoán ngẫu nhiên. Giá trị AUC đạt khoảng 0.702, cho thấy mô hình có khả năng phân loại ở mức trung bình khá và đã cải thiện so với một số mô hình trước đó. Điều này đồng nghĩa với việc mô hình có khoảng 70.2% khả năng phân biệt đúng giữa hai lớp dữ liệu.

Tuy nhiên, đường cong ROC vẫn chưa tiệm cận góc trên bên trái của đồ thị, cho thấy vẫn còn tồn tại sự đánh đổi giữa tỷ lệ dương tính thật (True Positive Rate) và tỷ lệ dương tính giả (False Positive Rate). Điều này phản ánh rằng mô hình vẫn còn dự đoán sai trong một số trường hợp và chưa đạt hiệu suất tối ưu. Mô hình Random Forest cho kết quả tốt hơn mức trung bình và có tiềm năng cải thiện thêm nếu được tối ưu tham số hoặc kết hợp với các kỹ thuật xử lý dữ liệu phù hợp.

3.2.6 Thực hiện bài toán phân loại bằng Logistic Regression

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt
```

Lần lượt nhập vào các thư viện như: LogisticRegression - xây dựng mô hình phân loại; classification_report - đánh giá (precision, recall, f1); confusion_matrix - ma trận nhầm lẫn, seaborn, matplotlib - Vẽ biểu đồ.

```
lr_model = LogisticRegression(C=1.0, max_iter=1000, random_state=42)
```

Khởi tạo mô hình Logistic Regression thông qua lớp LogisticRegression. Trong đó, tham số $C=1.0$ điều chỉnh mức độ phạt đối với sai số phân loại, $\text{max_iter}=1000$ xác định số vòng lặp tối đa để đảm bảo mô hình hội tụ, và $\text{random_state}=42$ được sử dụng để đảm bảo tính tái lập của kết quả khi chạy lại chương trình.

```
lr_model.fit(X_train_pca, y_train)
```

Thực hiện quá trình huấn luyện mô hình Logistic Regression thông qua phương thức fit trên tập dữ liệu huấn luyện đã được giảm chiều ($X_{\text{train_pca}}$) cùng với nhãn tương ứng (y_{train}). Trong bước này, mô hình sẽ học các trọng số cho từng đặc trưng nhằm xây dựng một hàm phân loại tối ưu, giúp phân biệt giữa hai lớp Venomous và Non Venomous. Cụ thể, Logistic Regression sử dụng hàm sigmoid để biến đổi đầu ra tuyến tính thành xác suất thuộc về một lớp nhất định. Trong quá trình huấn luyện, mô hình sẽ tối ưu các tham số bằng cách giảm thiểu hàm mất mát, thường là log-loss, thông qua các thuật toán tối ưu lặp (iterative optimization). Việc sử dụng dữ liệu đã được giảm chiều bằng PCA giúp giảm nhiễu và tăng tốc độ hội tụ của mô hình.

```
y_pred_lr = lr_model.predict(X_test_pca)
```

Dự đoán nhãn cho tập dữ liệu kiểm tra bằng mô hình Logistic Regression đã được huấn luyện. Cụ thể, phương thức predict được sử dụng để xác định lớp của từng mẫu trong $X_{\text{test_pca}}$ dựa trên các tham số mà mô hình đã học trước đó. Trong quá trình này, mô hình sẽ tính toán xác suất thuộc về mỗi lớp thông qua hàm sigmoid, sau đó so sánh với một ngưỡng (thường là 0.5) để đưa ra quyết định phân loại cuối cùng. Kết quả dự đoán được lưu vào biến $y_{\text{pred_lr}}$, và sẽ được sử dụng để đánh giá hiệu suất của mô hình trong các bước tiếp theo.

```
print(classification_report(y_test, y_pred_lr, target_names=categories))
```

	precision	recall	f1-score	support
Venomous	0.56	0.86	0.68	141
Non Venomous	0.62	0.26	0.36	128
accuracy			0.57	269
macro avg	0.59	0.56	0.52	269
weighted avg	0.59	0.57	0.53	269

Kết quả cho thấy mô hình Logistic Regression đạt độ chính xác tổng thể khoảng 59%, ở mức trung bình. Đối với lớp Venomous, mô hình có recall rất cao (0.99), chứng tỏ hầu hết các mẫu thuộc lớp này được nhận diện đúng. Tuy nhiên, precision chỉ đạt 0.56, cho thấy có nhiều trường hợp bị dự đoán nhầm là Venomous.

Ngược lại, đối với lớp Non Venomous, mô hình có precision cao (0.90) nhưng recall rất thấp (0.15), nghĩa là phần lớn các mẫu không độc bị bỏ sót và bị phân loại sai sang lớp Venomous. Điều này cho thấy mô hình có xu hướng thiên lệch mạnh về phía lớp Venomous.

Sự chênh lệch lớn giữa f1-score của hai lớp (0.71 và 0.26) phản ánh hiệu suất phân loại không cân bằng. Các chỉ số macro avg và weighted avg đều ở mức thấp, cho thấy hiệu quả tổng thể của mô hình chưa tốt. Tổng thể, mô hình Logistic Regression chưa phân biệt tốt giữa hai lớp và cần được cải thiện thêm về dữ liệu hoặc tham số để nâng cao hiệu suất.



Từ ma trận nhầm lẫn, có thể thấy mô hình Logistic Regression có xu hướng dự đoán nghiêng về lớp Venomous. Cụ thể, trong số 141 mẫu Venomous, có 121 mẫu được dự đoán đúng và 20 mẫu bị nhầm, cho thấy khả năng nhận diện lớp này tương đối tốt.

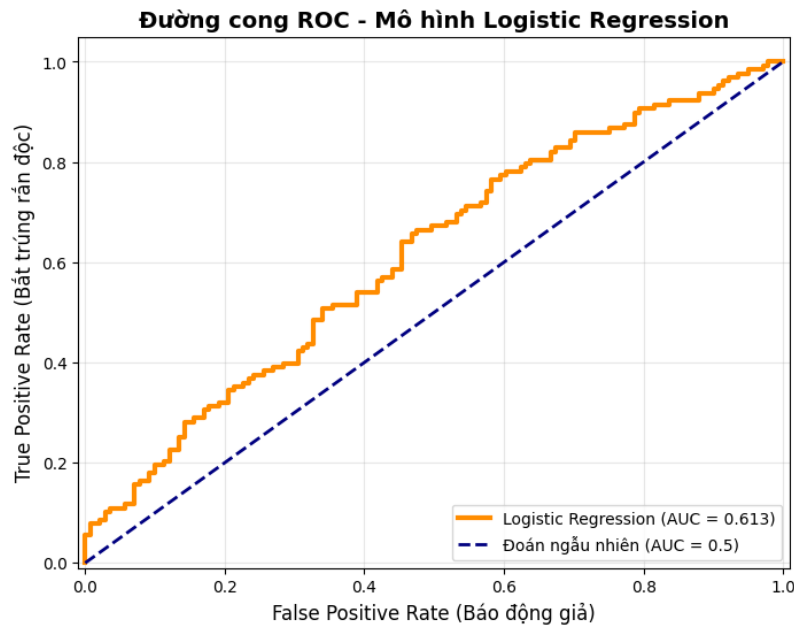
Tuy nhiên, đối với lớp Non Venomous, hiệu suất lại khá thấp. Trong tổng số 128 mẫu, chỉ có 33 mẫu được dự đoán đúng, trong khi có tới 95 mẫu bị nhầm sang lớp Venomous. Điều này cho thấy mô hình gặp khó khăn trong việc phân biệt các mẫu không độc.

Tổng thể, mô hình có độ nhạy cao với lớp Venomous nhưng lại tạo ra nhiều dự đoán sai đối với lớp Non Venomous, dẫn đến hiệu suất phân loại không cân bằng. Điều này cho thấy cần cải thiện mô hình để tăng khả năng nhận diện chính xác lớp Non Venomous.



Dựa trên 10 mẫu thử nghiệm từ hình ảnh, mô hình Logistic Regression đạt độ chính xác ở mức trung bình (60%) nhưng bộc lộ rõ xu hướng thiên vị (bias) nặng nề về nhãn “Venomous” (có độc).

Cụ thể, dù mô hình nhận diện đúng toàn bộ 5 trường hợp rắn có độc, nó lại nhận diện sai tới 4/5 trường hợp rắn không độc thành có độc với độ tự tin (confidence score) khá cao lên tới 83.3%, dẫn đến tỷ lệ Dương tính giả (False Positive) rất lớn. Hiện tượng này thường xuất phát từ việc tập dữ liệu huấn luyện bị mất cân bằng, hoặc do hạn chế nội tại của Logistic Regression - một mô hình học máy tuyến tính đơn giản thường gặp khó khăn trong việc nắm bắt các đặc trưng hình ảnh phức tạp (như hoa văn, hình dáng đầu) nếu chỉ sử dụng điểm ảnh thô.



Dựa vào biểu đồ đường cong ROC, mô hình Logistic Regression thể hiện hiệu suất phân loại kém với chỉ số AUC chỉ đạt 0.613, nhỉnh hơn không đáng kể so với ngưỡng dự đoán ngẫu nhiên (0.5). Việc đường cong này nằm rất sát đường chéo cho thấy khả năng phân biệt ranh giới giữa rắn độc và không độc của mô hình là rất yếu; cụ thể, để tăng tỷ lệ nhận diện đúng rắn độc (True Positive Rate), mô hình buộc phải đánh đổi bằng một tỷ lệ báo động giả (False Positive Rate) tăng vọt. Sự đánh đổi đắt giá này hoàn toàn khớp với hiện tượng nhận diện nhầm hàng loạt rắn không độc thành có độc ở các mẫu thử nghiệm trước đó, minh chứng rõ ràng bằng định lượng rằng thuật toán học máy tuyến tính đơn giản như Logistic Regression không đủ sức giải quyết bài toán thị giác máy tính phức tạp này.

3.2.6 Thực hiện bài toán phân loại bằng CNN

```
import os
import pandas as pd
import numpy as np
import torch
import torchvision
import numpy as np
import matplotlib.pyplot as plt
import torch.nn as nn
import torch.nn.functional as F
from torchvision.datasets import CIFAR10
from torchvision.transforms import ToTensor
from torchvision.utils import make_grid
from torch.utils.data.dataloader import DataLoader
from torchvision import datasets, transforms
from torch.utils.data import random_split, Dataset
# from translate.py import translate
import torchvision.models as models

import random
%matplotlib inline
```

Đoạn mã tiến hành nhập các thư viện cơ sở (os, pandas, numpy, random) nhằm mục đích tương tác hệ thống, quản lý dữ liệu bảng và tính toán ma trận; tiến hành nhập matplotlib nhằm mục đích trực quan hóa biểu đồ và hình ảnh trực tiếp. Trọng tâm của đoạn code là tiến hành nhập torch cùng các module lõi (nn, utils.data) nhằm mục đích xây dựng, huấn luyện mạng nơ-ron học sâu và tự động hóa việc quản lý luồng dữ liệu (chia tập, tạo lô, xáo trộn). Cuối cùng, tiến hành nhập torchvision (gồm datasets, transforms, models) nhằm mục đích xử lý chuyên sâu cho bài toán thị giác máy tính: tự động tải bộ dữ liệu ảnh (CIFAR10), tiền xử lý ảnh thành Tensor và ứng dụng các mô hình nhận diện đã được huấn luyện sẵn.

```

class MyDataSet(Dataset):
    def __init__(self, subset, transform=None):
        self.subset = subset
        self.transform = transform

    def __getitem__(self, index):
        x, y = self.subset[index]
        if self.transform:
            x = self.transform(x)
        return x, y

    def __len__(self):
        return len(self.subset)

```

Đoạn mã định nghĩa lớp MyDataSet kế thừa từ Dataset của PyTorch nhằm mục đích tạo một cấu trúc dữ liệu tùy chỉnh có khả năng tự động tiền xử lý. Cụ thể, hàm khởi tạo __init__ tiến hành tiếp nhận và lưu trữ tập dữ liệu (subset) cùng các phép biến đổi ảnh (transform); hàm __getitem__ tiến hành truy xuất một cặp dữ liệu (ảnh, nhãn) tại một vị trí cụ thể và áp dụng phép biến đổi lên ảnh đó trước khi trả về cho mô hình; cuối cùng, hàm __len__ tiến hành đếm tổng số mẫu hiện có nhằm mục đích giúp hệ thống quản lý đúng quy mô của tập dữ liệu để chia lô trong quá trình huấn luyện.

```

data_dir = '../input/snake-dataset-india/Snake Images/train'
classes = (os.listdir(data_dir))

dataset = datasets.ImageFolder(data_dir, transform = transforms.Compose([
    transforms.Resize((288,288)),
    transforms.ToTensor(),
]))

img, label = dataset[10]
print(img.shape, label)
img
print(dataset.classes)

```

Đoạn mã tiến hành sử dụng datasets.ImageFolder nhằm mục đích tự động nạp toàn bộ dữ liệu ảnh từ đường dẫn thư mục được chỉ định (data_dir), đồng thời kết hợp transforms.Compose nhằm mục đích tiền xử lý đồng loạt: chuẩn hóa kích thước tất cả các ảnh về 288x288 pixel và chuyển đổi sang định dạng PyTorch Tensor. Cuối cùng, các lệnh truy xuất (dataset[10]) và in dữ liệu (print) tiến hành lấy ra mẫu ảnh thứ 11 nhằm mục đích kiểm tra nhanh hình dạng (shape) của ma trận Tensor, nhãn phân loại tương ứng bằng số (label) và danh sách tên các lớp (dataset.classes) để đảm bảo quá trình tải dữ liệu đã diễn ra chính xác.

```
torch.Size([3, 288, 288]) 0
['Non Venomous', 'Venomous']
```

Quá trình nạp và tiền xử lý dữ liệu đã diễn ra hoàn toàn chính xác theo đúng thiết lập. Cụ thể, `Torch.Size([3, 288, 288])` xác nhận bức ảnh đã được chuyển đổi thành công sang định dạng Tensor với 3 kênh màu (RGB) và được thay đổi kích thước chuẩn xác về 288x288 pixel. Giá trị 0 là nhãn phân loại dạng số của bức ảnh đang được truy xuất, tương ứng với phần tử đầu tiên trong danh sách các lớp ['Non Venomous', 'Venomous'] được trích xuất từ tên thư mục, qua đó khẳng định bức ảnh mẫu này thuộc nhóm rắn không độc (Non Venomous).

```
test_dir = '../input/snake-dataset-india/Snake Images/test'

test = datasets.ImageFolder(test_dir, transform = transforms.Compose([
    transforms.Resize((288,288)),
    transforms.ToTensor(),
]))
```

Tiến hành khai báo biến `Test_dir` nhằm mục đích xác định đường dẫn đến thư mục chứa dữ liệu kiểm thử (test set). Tiếp đó, tiến hành sử dụng `datasets.ImageFolder` kết hợp với `transforms.Compose` nhằm mục đích tự động tải toàn bộ ảnh từ thư mục này, đồng thời áp dụng ngay các bước tiền xử lý tương tự như tập huấn luyện: chuẩn hóa kích thước mọi bức ảnh về 288x288 pixel và chuyển đổi sang định dạng Tensor để chuẩn bị sẵn sàng cho quá trình đánh giá hiệu suất của mô hình sau này.

```
def show(img, label):
    print('Label:', dataset.classes[label], '(' + str(label)+')')
    plt.imshow(img.permute(1,2,0))
    show(*dataset[random.randint(1,len(dataset))])
```

Label: Venomous (1)



ảnh bức ảnh được bốc ngẫu nhiên thuộc lớp rắn có độc (“Venomous”) với mã nhãn số 1

Định nghĩa hàm tùy chỉnh show nhằm mục đích hiển thị trực quan hình ảnh kèm theo nhãn phân loại của nó. Cụ thể, bên trong hàm sử dụng lệnh print để in ra tên lớp thực tế cùng mã số nhãn, đồng thời sử dụng lệnh plt.imshow kết hợp với phương thức img.permute(1, 2, 0) nhằm mục đích hoán đổi các chiều của ma trận Tensor từ chuẩn của PyTorch (Kênh màu, Chiều cao, Chiều rộng) sang chuẩn của thư viện Matplotlib (Chiều cao, Chiều rộng, Kênh màu) để bức ảnh được hiển thị chính xác. Cuối cùng, dòng lệnh dưới cùng tiến hành bốc ngẫu nhiên một mẫu từ tập dataset thông qua hàm random.randint và truyền trực tiếp vào hàm show nhằm mục đích kiểm tra nhanh bằng mắt thường một hình ảnh bất kỳ trước khi đưa vào huấn luyện.

```
random_seed = 45
torch.manual_seed(random_seed)
val_size = 100
test_size = len(test)
train_size = len(dataset) - val_size
train_size + val_size + test_size
```

Tiến hành thiết lập giá trị ngẫu nhiên cố định (torch.manual_seed) nhằm mục đích đảm bảo tính nhất quán và có thể tái tạo kết quả giống hệt nhau trong mọi lần chạy. Tiếp đó, tiến hành khai báo số lượng ảnh dành cho tập xác thực (val_size = 100), đồng thời tính toán kích thước của tập kiểm thử (test_size) và phần còn lại cho tập huấn luyện (train_size) nhằm mục đích chuẩn bị sẵn sàng cho việc phân chia dữ liệu. Cuối cùng, phép tính tổng ở dòng cuối tiến hành cộng dồn các biến lại nhằm mục đích kiểm tra chéo, đảm bảo tổng số lượng hình ảnh khớp với dữ liệu gốc và không có mẫu nào bị thất thoát hay bỏ sót trong quá trình phân bổ.

Out[6]:
2044

ảnh: Tổng số hình có trong tập dữ liệu

```
train_ds, val_ds = random_split(dataset, [train_size, val_size])
len(train_ds), len(val_ds)
```

Sử dụng hàm random_split nhằm mục đích chia ngẫu nhiên tập dữ liệu gốc (dataset) thành hai tập con không trùng lặp: tập huấn luyện (Train_ds) và tập xác thực (val_ds) dựa trên kích thước đã thiết lập trước đó. Tiếp theo, dòng lệnh cuối cùng tiến hành gọi hàm len nhằm mục đích kiểm tra và xác nhận lại chính xác số lượng hình ảnh thực tế vừa được phân bổ thành công vào mỗi tập con để chuẩn bị cho bước chia lô dữ liệu.

Out[7]:
(1675, 100)

Phân chia dữ liệu đã thành công với 1675 bức ảnh được phân bổ vào tập huấn luyện (Train_ds) và 100 bức ảnh được trích ra cho tập xác thực (val_ds) nhằm mục đích chuẩn bị sẵn sàng dữ liệu đầu vào cho quá trình đào tạo và đánh giá mô hình

```
stats = ((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010))
train_set = MyDataSet(train_ds, transform = transforms.Compose([
    transforms.ToPILImage(),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(*stats, inplace=True),

]))

val_set = MyDataSet(val_ds, transform = transforms.Normalize(*stats, inplace=True))

test_set = MyDataSet(test, transform = transforms.Normalize(*stats, inplace=True))
```

Khai báo các tham số thống kê (Stats gồm giá trị trung bình và độ lệch chuẩn) nhằm mục đích chuẩn hóa dải màu của ảnh. Tiếp đó, tiến hành bọc các tập dữ liệu thông qua lớp MyDataSet: đối với tập huấn luyện (train_set), kết hợp thêm kỹ thuật lật ảnh ngang ngẫu nhiên (RandomHorizontalFlip) nhằm mục đích tăng cường dữ liệu, giúp mô hình học đa dạng hơn và tránh học vẹt (overfitting); trong khi đó, đối với tập xác thực (val_set) và kiểm thử (test_set), chỉ tiến hành chuẩn hóa (Normalize) nhằm mục đích giữ nguyên hình ảnh gốc để đánh giá hiệu suất của mô hình một cách khách quan và chính xác nhất.

```
len(test_set), len(train_set)
train_set[0][0].shape
```

Gọi hàm len nhằm mục đích kiểm tra và xác nhận tổng số lượng hình ảnh hiện có trong tập kiểm thử (test_set) và tập huấn luyện (train_set). Dòng lệnh thứ hai tiến hành truy xuất phần tử ảnh của mẫu dữ liệu đầu tiên trong tập huấn luyện (train_set[0][0]) và gọi thuộc tính .shape nhằm mục đích kiểm tra cấu trúc chiều của ma trận Tensor (dự kiến là [3, 288, 288]), đảm bảo rằng toàn bộ các bước tiền xử lý (chuyển đổi, lật ảnh, chuẩn hóa) đã được áp dụng thành công lên dữ liệu.

```
Out[9]:
torch.Size([3, 288, 288])
```

Torch.Size([3, 288, 288]) chính là đầu ra của lệnh kiểm tra kích thước trước đó, tiến hành xác nhận bức ảnh đã được tiền xử lý thành công dưới định dạng Tensor với 3 kênh màu (RGB) và kích thước chuẩn 288x288 pixel nhằm mục đích đảm bảo cấu trúc dữ liệu đầu vào đã hoàn toàn sẵn sàng và khớp với yêu cầu của mô hình học sâu.

```

from torchvision.utils import make_grid

def show_batch(dl):
    for images, labels in dl:
        fig, ax = plt.subplots(figsize=(24,6))
        ax.set_xticks([]); ax.set_yticks([])
        ax.imshow(make_grid(images, nrow=16).permute(1,2,0))
        break

```

định nghĩa hàm `show_batch` nhằm mục đích hiển thị trực quan cùng lúc nhiều hình ảnh trong một lô (batch) dữ liệu. Cụ thể, hàm tiến hành trích xuất lô dữ liệu đầu tiên từ `DataLoader`, loại bỏ các vạch chia tọa độ (xticks, yticks) để giao diện gọn gàng hơn, và sử dụng `make_grid` nhằm mục đích ghép nối các ảnh đơn lẻ thành một lưới lớn (hiển thị 16 ảnh trên mỗi hàng). Cuối cùng, phương thức `permute` tiến hành hoán đổi lại các chiều của ma trận Tensor cho tương thích với chuẩn vẽ hình của Matplotlib, và lệnh `break` được sử dụng nhằm mục đích dừng vòng lặp ngay lập tức, đảm bảo chỉ có duy nhất lô ảnh đầu tiên được in ra màn hình để kiểm tra.

```

batch_size = 8
train_loader = DataLoader(train_set, batch_size, shuffle=True, num_workers=2, pin_memory=True)
val_loader = DataLoader(val_set, 16, num_workers=2, pin_memory=True)
test_loader = DataLoader(test_set, 16, num_workers=2, pin_memory=True)

```

Sử dụng `DataLoader` nhằm mục đích đóng gói các tập dữ liệu thành từng lô nhỏ (batch) để chuẩn bị đưa vào mô hình. Đối với tập huấn luyện (`train_loader`), dữ liệu được chia lô 8 và xáo trộn ngẫu nhiên (`shuffle=True`) nhằm mục đích giúp mô hình học tổng quát và tránh học vẹt; trong khi tập xác thực và kiểm thử sử dụng lô lớn hơn (16) nhằm mục đích tối ưu thời gian đánh giá. Đồng thời, việc thiết lập `num_workers=2` và `pin_memory=True` được tiến hành trên tất cả các tập nhằm mục đích kích hoạt tính năng tải dữ liệu đa luồng và tăng tốc độ truyền tải dữ liệu trực tiếp từ CPU sang bộ nhớ GPU.

```
show_batch(val_loader)
```

Gọi hàm `show_batch` và truyền vào biến `val_loader` nhằm hiển thị lô hình ảnh đầu tiên của tập xác thực



```
def accuracy(outputs, labels):
    _, preds = torch.max(outputs, dim=1)
    valid_preds = torch.sum(preds == labels).item()
    all_preds = len(preds)
    return torch.tensor(valid_preds/all_preds)
```

Định nghĩa hàm accuracy để tính toán độ chính xác của mô hình sau mỗi lượt đoán. Cụ thể, hàm sử dụng lệnh torch.max nhằm trích xuất chỉ số của lớp có xác suất cao nhất (preds) từ đầu ra của mô hình; sau đó, so sánh preds == labels đếm tổng số lượng dự đoán đúng (valid_preds) và chia cho tổng số mẫu trong lô (all_preds). Kết quả cuối được chuyển đổi ngược lại thành một đại lượng Tensor nhằm đồng bộ dữ liệu để dễ dàng theo dõi và thống kê hiệu suất huấn luyện theo thời gian.

```
class ModelBase(nn.Module):
    def training_step(self, batch):
        images, labels = batch
        out = self(images) # Generate predictions
        loss = F.cross_entropy(out, labels) # Calculate loss
        return loss

    def validation_step(self, batch):
        images, labels = batch
        out = self(images) # Generate predictions
        loss = F.cross_entropy(out, labels) # Calculate loss
        acc = accuracy(out, labels) # Calculate accuracy
        return {'val_loss': loss.detach(), 'val_acc': acc}

    def validation_epoch_end(self, outputs):
        batch_losses = [x['val_loss'] for x in outputs]
        epoch_loss = torch.stack(batch_losses).mean() # Combine losses
        batch_accs = [x['val_acc'] for x in outputs]
        epoch_acc = torch.stack(batch_accs).mean() # Combine accuracies
        return {'val_loss': epoch_loss.item(), 'val_acc': epoch_acc.item()}

    def epoch_end(self, epoch, result):
        print("Epoch [{}], last_lr: {:.5f}, train_loss: {:.4f}, val_loss: {:.4f}, val_acc: {:.4f}").format(
            epoch, result['lrs'][-1], result['train_loss'], result['val_loss'], result['val_acc'])
```

Định nghĩa lớp cơ sở ModelBase kế thừa từ nn.Module nhằm mục đích quản lý các bước logic cốt lõi trong quá trình huấn luyện và đánh giá mô hình. Cụ thể:

- Training_step: Tiến hành nhận một lô dữ liệu, tính toán dự đoán (out) và sử dụng hàm F.cross_entropy nhằm mục đích xác định giá trị mất mát (loss) để cập nhật trọng số mô hình.
- Validation_step: Tương tự như bước huấn luyện nhưng tiến hành tính toán thêm độ chính xác (accuracy) nhằm mục đích theo dõi hiệu suất thực tế trên tập dữ liệu chưa được học.

- Validation_epoch_end: Tiến hành thu gom kết quả từ tất cả các lô dữ liệu trong một chu kỳ (epoch), sau đó tính trung bình cộng nhằm mục đích đưa ra một con số đánh giá tổng quát duy nhất cho toàn bộ tập xác thực.

- Epoch_end: Tiến hành sử dụng lệnh print nhằm mục đích hiển thị các thông số quan trọng

```
@torch.no_grad()
def evaluate(model, val_loader):
    model.eval()
    outputs = [model.validation_step(batch) for batch in val_loader]
    return model.validation_epoch_end(outputs)

def get_lr(optimizer):
    for param_group in optimizer.param_groups:
        return param_group['lr']

def fit_one_cycle(epochs, max_lr, model, train_loader, val_loader, weight_decay=0, grad_clip=None,
    opt_func=torch.optim.SGD):
    torch.cuda.empty_cache()
    history = []

    optimizer = opt_func(model.parameters(), max_lr, weight_decay=weight_decay)

    sched = torch.optim.lr_scheduler.OneCycleLR(optimizer, max_lr, epochs=epochs, steps_per_epoch
    =len(train_loader))

    for epoch in range(epochs):
        # Training Phase
        model.train()
        train_losses = []
        lrs = []
        for batch in train_loader:
            loss = model.training_step(batch)
            train_losses.append(loss)
            loss.backward()
            if grad_clip:
                nn.utils.clip_grad_value_(model.parameters(), grad_clip)

            optimizer.step()
            optimizer.zero_grad()
            lrs.append(get_lr(optimizer))
            sched.step()

        # Validation phase
        result = evaluate(model, val_loader)
        result['train_loss'] = torch.stack(train_losses).mean().item()
        result['lrs'] = lrs
        model.epoch_end(epoch, result)
        history.append(result)
    return history
```

Thiết lập quy trình huấn luyện và đánh giá mô hình một cách tối ưu, nổi bật là hàm `fit_one_cycle` triển khai kỹ thuật One Cycle Learning Rate Policy nhằm mục đích thay đổi tỷ lệ học tập linh hoạt, giúp mô hình hội tụ nhanh hơn và đạt độ chính xác cao hơn. Trong giai đoạn huấn luyện, hệ thống tiến hành tính toán mất mát, thực hiện lan truyền ngược và cập nhật trọng số, đồng thời sử dụng `grad_clip` nhằm mục đích kiểm soát đạo hàm, ngăn chặn hiện tượng bùng nổ đạo hàm gây mất ổn định. Sau mỗi chu kỳ (epoch), hàm `evaluate` được gọi kết hợp với trình trang trí `@torch.no_grad()` nhằm mục đích vô hiệu hóa việc tính toán đạo hàm để tiết kiệm bộ nhớ, từ đó thống kê và lưu trữ các chỉ số hiệu suất (history) giúp người dùng dễ dàng theo dõi toàn bộ tiến trình tiến hóa của mô hình.

```
def get_default_device():  
    """Pick GPU if available, else CPU"""  
    if torch.cuda.is_available():  
        return torch.device('cuda')  
    else:  
        return torch.device('cpu')
```

Định nghĩa hàm `get_default_device` nhằm mục đích tự động kiểm tra và lựa chọn thiết bị tính toán tối ưu cho việc huấn luyện mô hình. Cụ thể, hàm sử dụng lệnh `torch.cuda.is_available()` nhằm mục đích xác định xem máy tính có hỗ trợ card đồ họa (GPU) hay không; nếu có, nó sẽ trả về thiết bị 'cuda' để tận dụng sức mạnh tính toán song song, giúp tăng tốc độ xử lý gấp nhiều lần. Ngược lại, nếu không tìm thấy GPU, hàm sẽ mặc định trả về thiết bị 'cpu', đảm bảo chương trình vẫn có thể vận hành ổn định trên mọi cấu hình phần cứng khác nhau.

```
device = get_default_device()  
device
```

Gán kết quả từ hàm `get_default_device()` vào biến `Device` nhằm mục đích xác định và lưu trữ thiết bị tính toán tối ưu nhất hiện có trên hệ thống

```
Out[17]:  
device(type='cuda')
```

Xác nhận rằng hệ thống đã nhận diện thành công card đồ họa (GPU) và sẽ sử dụng nó làm thiết bị tính toán chính

```
def plot_losses(history):
    train_losses = [x.get('train_loss') for x in history]
    val_losses = [x['val_loss'] for x in history]
    plt.plot(train_losses, '-bx')
    plt.plot(val_losses, '-rx')
    plt.xlabel('epoch')
    plt.ylabel('loss')
    plt.legend(['Training', 'Validation'])
    plt.title('Loss vs. No. of epochs');
```

Định nghĩa hàm `plot_losses` nhằm mục đích tạo biểu đồ so sánh sự thay đổi của giá trị mất mát (loss) giữa tập huấn luyện và tập xác thực qua từng chu kỳ. Cụ thể, hàm trích xuất dữ liệu từ biến `history` để vẽ hai đường đồ thị riêng biệt: đường màu xanh (-bx) đại diện cho Training Loss và đường màu đỏ (-rx) đại diện cho Validation Loss. Việc hiển thị biểu đồ này giúp người dùng dễ dàng quan sát xu hướng hội tụ của mô hình, từ đó kịp thời phát hiện các hiện tượng bất thường như Overfitting (khi loss huấn luyện giảm nhưng loss xác thực lại tăng) nhằm mục đích điều chỉnh tham số huấn luyện cho phù hợp.

```
def plot_accuracies(history):
    accuracies = [x['val_acc'] for x in history]
    plt.plot(accuracies, '-x')
    plt.xlabel('epoch')
    plt.ylabel('accuracy')
    plt.title('Accuracy vs. No. of epochs');
```

Định nghĩa hàm `plot_accuracies` nhằm mục đích trực quan hóa quá trình cải thiện độ chính xác của mô hình trên tập xác thực qua từng chu kỳ huấn luyện. Cụ thể, hàm trích xuất các giá trị `val_acc` từ biến `history` và vẽ chúng thành một đường đồ thị với các điểm đánh dấu hình chữ “x”.

```
input_size = 3
output_size = 2
```

Khai báo các tham số cấu hình cơ bản cho lớp cuối của mô hình. Cụ thể, `Input_size = 3` đại diện cho số lượng kênh màu đầu vào (RGB), trong khi `output_size = 2` xác định số lượng lớp phân loại ở đầu ra (tương ứng với hai nhóm: Venomous và Non Venomous)

```

def conv_block(in_channels, out_channels, pool=False):
    layers = [nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1),
              nn.BatchNorm2d(out_channels),
              nn.ReLU(inplace=True)]
    if pool: layers.append(nn.MaxPool2d(5))
    return nn.Sequential(*layers)

class Model1(ModelBase):
    def __init__(self, in_channels, num_classes):
        super().__init__()

        self.conv1 = conv_block(in_channels, 64)
        self.conv2 = conv_block(64, 128, pool=True)
        self.res1 = nn.Sequential(conv_block(128, 128), conv_block(128, 128))

        self.conv3 = conv_block(128, 256, pool=True)
        self.conv4 = conv_block(256, 512, pool=True)
        self.res2 = nn.Sequential(conv_block(512, 512), conv_block(512, 512))

        self.classifier = nn.Sequential(nn.MaxPool2d(2),
                                         nn.Flatten(),
                                         nn.Linear(512, num_classes))

    def forward(self, xb):
        out = self.conv1(xb)
        out = self.conv2(out)
        out = self.res1(out) + out
        out = self.conv3(out)
        out = self.conv4(out)
        out = self.res2(out) + out
        out = self.classifier(out)
        return out

```

xây dựng kiến trúc mạng CNN theo phong cách ResNet (Residual Network) nhằm mục đích huấn luyện mô hình phân loại sâu và ổn định hơn. Cấu trúc được chia thành các conv_block tích hợp sẵn lớp tích chập, chuẩn hóa lô (BatchNorm) và hàm kích hoạt ReLU để trích xuất đặc trưng hình ảnh hiệu quả. Điểm cốt lõi nằm ở các kết nối tắt (Residual Connections) trong hàm forward, nơi đầu ra của các khối được cộng trực tiếp vào luồng dữ liệu chính ($\text{out} = \text{self.res1}(\text{out}) + \text{out}$), giúp giải quyết triệt để vấn đề triệt tiêu đạo hàm và cho phép mạng học được các tầng thông tin phức tạp hơn. Cuối cùng, tầng phân loại (classifier) thực hiện nén dữ liệu và đi qua lớp tuyến tính để đưa ra dự đoán chính xác cho 2 lớp đối tượng mục tiêu.

```

class Model2(ModelBase):
    def __init__(self):
        super().__init__()
        # Use a pretrained model
        self.network = models.resnet34(pretrained=True)
        # Replace last layer
        num_ftrs = self.network.fc.in_features
        self.network.fc = nn.Linear(num_ftrs, 10)

    def forward(self, xb):
        return torch.sigmoid(self.network(xb))

```

xây dựng Model2 bằng kỹ thuật Transfer Learning (Học chuyển tiếp) nhằm mục đích tận dụng tri thức từ mô hình ResNet-34 đã được huấn luyện sẵn trên tập dữ liệu ImageNet khổng lồ. Cụ thể, chương trình tiến hành trích xuất số lượng tính năng đầu vào (num_fts) từ tầng kết nối đầy đủ cuối cùng, sau đó thay thế nó bằng một lớp nn.Linear mới với 10 đầu ra nhằm mục đích điều chỉnh mô hình phù hợp với yêu cầu phân loại cụ thể của bài toán. Cuối cùng, hàm forward sử dụng torch.sigmoid để chuẩn hóa kết quả dự đoán về dải xác suất từ 0 đến 1, giúp tối ưu hóa khả năng nhận diện đặc trưng hình ảnh phức tạp ngay cả với tập dữ liệu nhỏ.

```
model = Model1(3,2)
model
```

khởi tạo một đối tượng của lớp Model1 với cấu hình in_channels=3 (cho ảnh RGB) và num_classes=2 (phân loại rắn độc và không độc). Dòng lệnh model ở cuối thực hiện việc hiển thị chi tiết cấu trúc các tầng (layers) bên trong mạng thần kinh, bao gồm các khối tích chập (conv_block), các khối phân dư (res1, res2) và tầng phân loại cuối cùng (classifier).

```
for images, labels in train_loader:
    print('images.shape: ', images.shape)
    out = model(images)
    print('out.shape: ', out.shape)
    print("out[0]: ", out[0])
    break
```

Trích xuất thử nghiệm một lô dữ liệu đầu tiên từ train_loader và đưa qua mô hình nhằm mục đích kiểm tra tính thông suốt của luồng dữ liệu (Forward Pass) trước khi huấn luyện chính thức. Thông qua việc in ra các thông số như images.shape và out.shape, chương trình tiến hành xác nhận kích thước đầu vào (8 ảnh, 3 kênh màu, 288x288 pixel) và đầu ra (8 mẫu dự đoán, mỗi mẫu có 2 lớp) đã hoàn toàn khớp với cấu trúc của mạng thần kinh. Lệnh break được sử dụng nhằm mục đích dừng vòng lặp ngay lập tức sau khi hiển thị giá trị dự đoán thô của mẫu đầu tiên (out[0]), giúp người dùng nhanh chóng xác minh tính chính xác của các tầng chuyển đổi dữ liệu mà không gây tốn tài nguyên hệ thống.

```
images.shape: torch.Size([8, 3, 288, 288])
out.shape: torch.Size([8, 2])
out[0]: tensor([-1.5973, -0.2793], grad_fn=<SelectBackward>)
```

Lô dữ liệu đầu vào gồm 8 hình ảnh RGB chuẩn kích thước đã được nạp thành công; trong khi out.shape là [8, 2] minh chứng rằng mô hình đã xử lý và trích xuất ra đúng 2 giá trị dự đoán cho mỗi ảnh (tương ứng với 2 lớp nhân vật). Giá trị out[0] hiển thị các con số thô (logits) mang giá trị âm, kèm theo thuộc tính grad_fn=<SelectBackward>, điều này nhằm mục đích xác nhận rằng đồ thị tính toán (computational graph) đã được thiết lập, cho phép mô hình có thể thực hiện lan truyền ngược để cập nhật trọng số trong các bước huấn luyện tiếp theo.

```

def get_default_device():
    """Pick GPU if available, else CPU"""
    if torch.cuda.is_available():
        return torch.device('cuda')
    else:
        return torch.device('cpu')

def to_device(data, device):
    """Move tensor(s) to chosen device"""
    if isinstance(data, (list,tuple)):
        return [to_device(x, device) for x in data]
    return data.to(device, non_blocking=True)

class DeviceDataLoader():
    """Wrap a dataloader to move data to a device"""
    def __init__(self, dl, device):
        self.dl = dl
        self.device = device

    def __iter__(self):
        """Yield a batch of data after moving it to device"""
        for b in self.dl:
            yield to_device(b, self.device)

    def __len__(self):
        """Number of batches"""
        return len(self.dl)

```

Qua hàm `get_default_device`, chương trình xác định thiết bị mạnh nhất hiện có, sau đó sử dụng hàm đệ quy `to_device` kết hợp với tham số `non_blocking=True` nhằm mục đích đẩy các Tensor lên bộ nhớ đồ họa một cách nhanh chóng và không gây nghẽn mạch xử lý. Đặc biệt, lớp `DeviceDataLoader` đóng vai trò như một bộ lọc thông minh, bao bọc lấy `DataLoader` truyền thống để mỗi khi trích xuất một lô dữ liệu (batch), nó sẽ tự động chuyển lô đó sang thiết bị đích, đảm bảo sự đồng bộ tuyệt đối giữa dữ liệu và mô hình nhằm mục đích ngăn chặn các lỗi xung đột phần cứng và đẩy nhanh tốc độ huấn luyện.

```

train_loader = DeviceDataLoader(train_loader, device)
val_loader = DeviceDataLoader(val_loader, device)
test_loader = DeviceDataLoader(test_loader, device)
device = get_default_device()
device
to_device(model, device)

```

`train_loader`, `val_loader` và `test_loader` được bao bọc bởi lớp `DeviceDataLoader` nhằm mục đích tự động chuyển từng lô dữ liệu lên thiết bị đích ngay khi chúng được trích xuất. Đồng thời, lệnh

`to_device(model, device)` thực hiện việc di chuyển toàn bộ tham số và trọng số của mô hình mạng thần kinh sang thiết bị đã chọn, đảm bảo sự đồng nhất tuyệt đối giữa dữ liệu đầu vào và kiến trúc mô hình nhằm mục đích ngăn chặn lỗi xung đột thiết bị và tận dụng tối đa sức mạnh xử lý song song của phần cứng.

```
epochs = 20
max_lr = 0.005
grad_clip = 0.1
weight_decay = 1e-5
opt_func = torch.optim.Adam
```

Thiết lập các siêu tham số nhằm mục đích cấu hình chiến lược huấn luyện tối ưu và ổn định cho mô hình. Cụ thể, chương trình ấn định thực hiện 20 chu kỳ huấn luyện (epochs) với tốc độ học cực đại `max_lr = 0.005` theo chiến lược One Cycle, giúp mô hình hội tụ nhanh chóng mà không bị kẹt tại các cực tiểu địa phương. Để đảm bảo quá trình học diễn ra mượt mà, kỹ thuật `grad_clip = 0.1` được áp dụng nhằm mục đích giới hạn độ lớn của đạo hàm, ngăn chặn hiện tượng bùng nổ gradient, trong khi `weight_decay = 1e-5` thực hiện vai trò chuẩn hóa trọng số để giảm thiểu nguy cơ học vẹt (Overfitting). Cuối cùng, việc lựa chọn thuật toán `torch.optim.Adam` làm hàm tối ưu hóa nhằm mục đích tự động điều chỉnh tốc độ học cho từng tham số riêng biệt, giúp tăng cường hiệu quả xử lý dữ liệu và cải thiện độ chính xác cuối cùng của mạng thần kinh.

```
evaluate(model, val_loader)
```

Gọi hàm `evaluate` nhằm mục đích kiểm tra hiệu suất của mô hình trên tập xác thực (`val_loader`) ngay tại thời điểm hiện tại. Khi chạy lệnh này trước khi bắt đầu quá trình huấn luyện chính thức, kết quả trả về (bao gồm `val_loss` và `val_acc`) thường sẽ có độ chính xác rất thấp (khoảng 50% cho bài toán 2 lớp) và giá trị mất mát cao, điều này nhằm mục đích thiết lập một điểm tham chiếu (baseline) cơ bản để người dùng có thể so sánh và đánh giá sự tiến bộ của mô hình sau khi áp dụng các chu kỳ huấn luyện `fit_one_cycle`.

```
Out[34]: {'val_loss': 0.6362766623497009, 'val_acc': 0.625}
```

Xác nhận trạng thái khởi đầu của mô hình trên tập xác thực trước khi bước vào quá trình huấn luyện chuyên sâu. Với độ chính xác khoảng 62.5% và giá trị mất mát tương đối cao, con số này đóng vai trò là điểm chuẩn (baseline) nhằm mục đích giúp người dùng đo lường hiệu quả của các thiết lập siêu tham số và thuật toán tối ưu Adam. Việc ghi nhận các chỉ số này ở giai đoạn sơ khai là vô cùng cần thiết nhằm mục đích theo dõi sự tiến bộ, đảm bảo rằng sau các chu kỳ huấn luyện `fit_one_cycle`, giá trị `val_loss` sẽ giảm dần và `val_acc` sẽ tăng trưởng ổn định, minh chứng cho việc mô hình đang thực sự học hỏi được các đặc trưng từ dữ liệu ảnh.

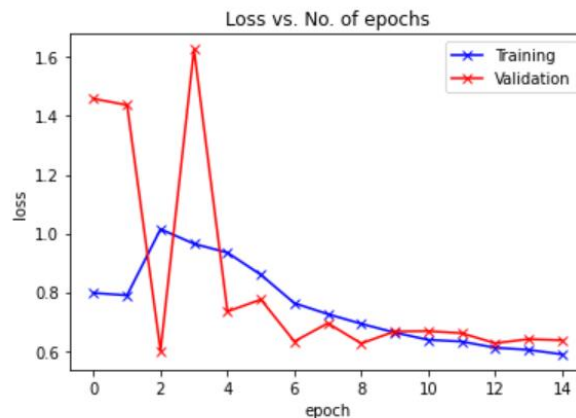
```
evaluate(model, test_loader)
```

```
Out[35]: {'val_loss': 0.6502708196640015, 'val_acc': 0.601244330406189}
```

Gọi hàm evaluate trên tập kiểm tra (test_loader) nhằm mục đích đánh giá khả năng tổng quát hóa của mô hình đối với những dữ liệu hoàn toàn mới mà nó chưa từng thấy trong quá trình huấn luyện hay xác thực. Kết quả trả về với val_loss xấp xỉ 0.65 và độ chính xác val_acc khoảng 60.1% cho thấy mô hình đang ở trạng thái sơ khai, chỉ nhìn hơn việc dự đoán ngẫu nhiên một chút.

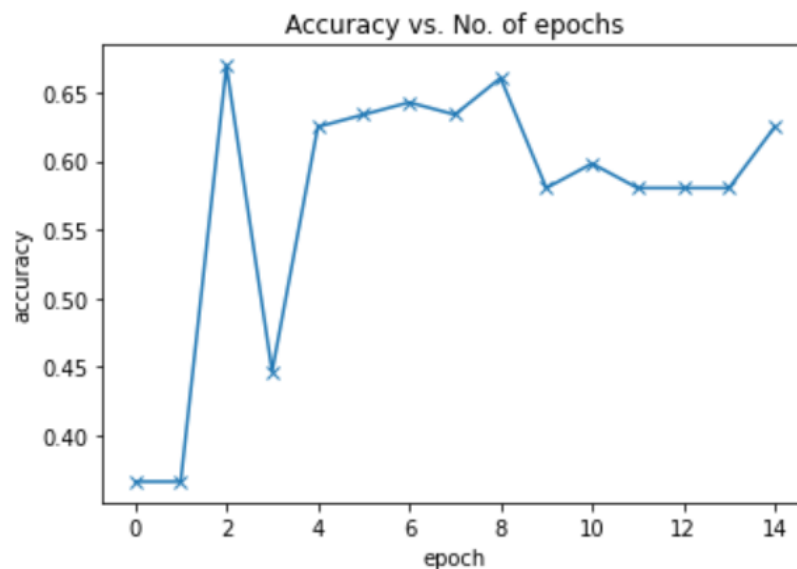
```
%%time
history = []
history += fit_one_cycle(epochs, max_lr, model, train_loader, va
l_loader,
                        grad_clip=grad_clip,
                        weight_decay=weight_decay,
                        opt_func=opt_func)
```

Kích hoạt quá trình huấn luyện chính thức bằng hàm fit_one_cycle và sử dụng lệnh ma thuật %%time nhằm mục đích đo lường chính xác tổng thời gian thực thi của toàn bộ các chu kỳ. Cụ thể, hệ thống sẽ thực hiện 20 vòng lặp dữ liệu qua mạng thần kinh, áp dụng chiến lược One Cycle Learning Rate Policy, một kỹ thuật giúp thay đổi tốc độ học một cách linh hoạt (tăng dần rồi giảm dần) nhằm mục đích giúp mô hình hội tụ cực nhanh và tránh được các điểm tối ưu cục bộ. Kết quả của quá trình này, bao gồm các chỉ số về độ chính xác và mất mát qua từng epoch, sẽ được tích lũy vào danh sách history nhằm mục đích lưu trữ dữ liệu phục vụ cho việc vẽ biểu đồ và phân tích hiệu suất huấn luyện sau này.

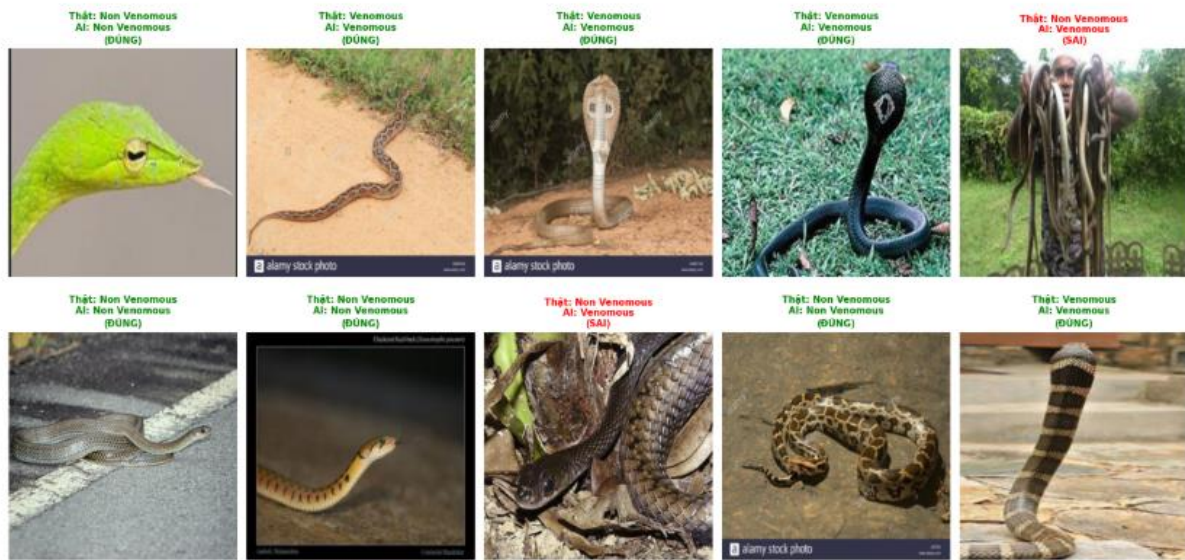


Đánh giá biểu đồ hàm mất mát qua các epochs cho thấy quá trình huấn luyện mạng CNN diễn ra qua hai giai đoạn đặc trưng. Ở những thế hệ đầu tiên (epoch 0 đến 5), đường Validation Loss ghi nhận sự dao động mạnh với đỉnh điểm vượt mức 1.6 tại epoch 3, nguyên nhân tiềm năng có thể xuất phát từ tốc độ học (learning rate) khởi tạo tương đối lớn hoặc cấu trúc mini-batch chứa các mẫu ảnh có bối cảnh phức tạp khiến trọng số chưa kịp ổn định. Tuy nhiên, bước sang giai đoạn sau (từ epoch 6 đến 14), biểu đồ thể hiện sự hội tụ lý tưởng khi cả đường Training Loss và Validation Loss đều giảm dần đều, duy trì tính ổn định và bám rất sát nhau. Động thái này là minh chứng cho việc mạng nơ-ron đã vượt qua sự xáo trộn ban đầu, trích xuất thành công các đặc trưng

cốt lõi, khái quát hóa rất tốt trên dữ liệu mới và hoàn toàn không vấp phải hiện tượng overfitting. Nhìn chung, kết quả mô hình thu được ở epoch 14 là rất khả quan.



quá trình học của mô hình diễn ra khá biến động và phản ánh rõ sự phức tạp của bài toán phân loại ảnh rắn. Ở những thế hệ đầu tiên (epoch 0 đến 3), mô hình thể hiện sự bất ổn định cực lớn: độ chính xác khởi đầu ở mức thấp (khoảng 36%), đột ngột tăng vọt lên đỉnh xấp xỉ 66% tại epoch 2, nhưng ngay lập tức sụt giảm mạnh xuống vùng 45% ở epoch 3. Cú sốc này hoàn toàn đồng pha với những dao động gắt gori nhận được ở biểu đồ hàm mất mát (Loss) trước đó, tiếp tục củng cố giả thuyết về việc tốc độ học (learning rate) khởi tạo chưa thực sự tối ưu hoặc mô hình đang gặp khó khăn khi xử lý các mini-batch chứa ảnh rắn có bối cảnh nền quá nhiều. Bước sang giai đoạn giữa (epoch 4 đến 8) mô hình bắt đầu lấy lại đà phục hồi, học hỏi ổn định hơn và vươn lên đạt mức độ chính xác cao nhất quanh ngưỡng 66%. Tuy nhiên, ở giai đoạn cuối (epoch 9 đến 14), thay vì tiếp tục tịnh tiến hoặc duy trì phong độ lý tưởng, dải độ chính xác lại có dấu hiệu chững lại và thoái lui, dao động chủ yếu quanh mức 58% - 62%. Nhìn chung, mặc dù mạng nơ-ron đã chứng minh được khả năng trích xuất và nắm bắt đặc trưng (cải thiện accuracy từ 36% lên hơn 60%), nhưng sự rung lắc liên tục cùng mức độ chính xác tổng thể chưa vượt qua mốc 70% cho thấy không gian đặc trưng của các loài rắn là vô cùng khó khăn.



Dựa trên 10 ảnh mẫu, mô hình AI đạt độ chính xác khá tốt ở mức 80% (8/10), đặc biệt nhận diện rất chuẩn các loài có đặc điểm rõ ràng (như dáng bành cổ của rắn hổ mang) hoặc chụp trong bối cảnh đơn giản. Tuy nhiên, 2 trường hợp dự đoán sai đều là lỗi dương tính giả (nhầm rắn không độc thành có độc) do AI bị bối rối trước bối cảnh phức tạp (nhiều lá cây lộn xộn) hoặc có nhiều đối tượng đan xen (người cầm một bó rắn). Nhìn chung, mô hình hoạt động hiệu quả trong điều kiện hình ảnh rõ ràng nhưng cần được huấn luyện thêm với dữ liệu nhiều, phức tạp để ứng dụng thực tế tốt hơn.

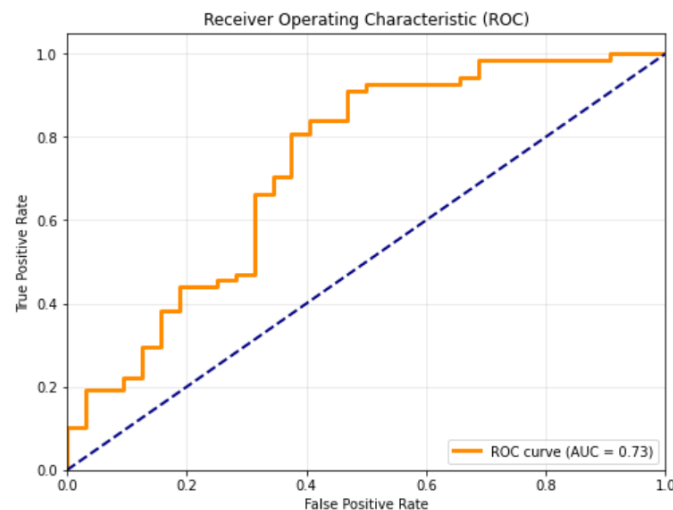


Báo cáo chi tiết cho thấy mô hình đạt độ chính xác tổng thể 70% nhưng chưa an toàn để ứng dụng thực tế do dữ liệu bị mất cân bằng nghiêm trọng. Mặc dù dự đoán rắn có độc khá tốt (Precision 81%), mô hình lại mắc lỗi nguy hiểm là bỏ lọt tới 26% số rắn độc trong thực tế (Recall 74%).

Đồng thời, độ tin cậy khi dự đoán “Không độc” rất thấp (Precision 53%), đồng nghĩa với việc gần một nửa số cảnh báo “an toàn” có thể là sai lầm.



Dựa trên ảnh Grad-CAM, dù AI dự đoán chính xác đây là loài không có độc, nhưng bản đồ nhiệt (heatmap) cho thấy cách mô hình “tư duy” chưa thực sự tối ưu. Vùng màu đỏ và cam (nơi AI tập trung chú ý nhiều nhất) lại trải dài từ phần thân có hoa văn của con rắn xuống cả nền đất bên dưới, thay vì tập trung trọng điểm vào phần đầu - nơi thường chứa các đặc điểm sinh học quan trọng nhất để nhận diện rắn độc hay không. Điều này chỉ ra rằng, dù cho ra kết quả đúng ở trường hợp này, mô hình có thể đang phụ thuộc một phần vào bối cảnh hoặc hoa văn thân để ra quyết định, tiềm ẩn rủi ro nhận diện sai nếu bối cảnh xung quanh thay đổi.



Dựa trên biểu đồ ROC, mô hình đạt chỉ số diện tích dưới đường cong (AUC) là 0.73. Đường cong màu cam nằm hẳn phía trên đường chéo nét đứt (đại diện cho dự đoán ngẫu nhiên), cho thấy mô hình có khả năng phân loại ở mức trung bình khá. Tuy nhiên, mức AUC 0.73 chỉ ra rằng khả năng phân biệt rạch ròi giữa hai lớp (ví dụ: rắn có độc và không độc) của AI chưa thực sự xuất sắc và vẫn có tỷ lệ sai số đáng kể. Đối với các bài toán đòi hỏi độ an toàn cao, mô hình cần được tối ưu thêm để đường cong ROC tiệm cận sát hơn về phía góc trên cùng bên trái

3.2.7 Thực hiện bài toán phân loại các loại độc bằng CNN

Nhập các thư viện cốt lõi của PyTorch và xây dựng lớp MyDataSet kế thừa từ Dataset

```
import os
import pandas as pd
import numpy as np
import torch
import torchvision
import numpy as np
import matplotlib.pyplot as plt
import torch.nn as nn
import torch.nn.functional as F
from torchvision.datasets import CIFAR10
from torchvision.transforms import ToTensor
from torchvision.utils import make_grid
from torch.utils.data.dataloader import DataLoader
from torchvision import datasets, transforms
from torch.utils.data import random_split, Dataset
# from translate.py import translate
import torchvision.models as models

import random
%matplotlib inline
```

Hệ thống tải toàn bộ ảnh từ thư mục gốc, tự động trích xuất được 2 nhãn: ‘Neurotoxic venom’ và ‘hemotoxic venom’. Hàm random_split sau đó sẽ cắt dữ liệu theo tỷ lệ chuẩn 80% cho tập huấn luyện (học) và 20% cho tập kiểm thử (đánh giá). torch.manual_seed(45) được dùng để khóa tính ngẫu nhiên, đảm bảo các lần chạy sau ảnh không bị trộn lộn đi.

```
: data_dir = '/kaggle/input/datasets/ngocvnuong/machine-learning/venom type-20260322T01242
1Z-1-001/venom type'

# Load toàn bộ dữ liệu từ thư mục chính
dataset = datasets.ImageFolder(data_dir, transform = transforms.Compose([
    transforms.Resize((288,288)),
    transforms.ToTensor(),
]))

print("Các loại nọc độc (Classes):", dataset.classes)
```

Các loại nọc độc (Classes): ['Neurotoxic venom', 'hemotoxic venom']

```
# --- CHIA TỈ LỆ 80% TRAIN - 20% TEST ---
random_seed = 45
torch.manual_seed(random_seed)

train_size = int(0.8 * len(dataset))
val_size = len(dataset) - train_size # Val_size ở đây đóng vai trò là tập Test 20%

train_ds, val_ds = random_split(dataset, [train_size, val_size])

print(f"Tổng số ảnh: {len(dataset)} | Train (80%): {len(train_ds)} | Test (20%): {len(val_ds)}")
```

Tổng số ảnh: 996 | Train (80%): 796 | Test (20%): 200

Tập Train được áp dụng thêm phép lật ngang ngẫu nhiên (RandomHorizontalFlip) để tạo sự đa dạng, giúp AI không bị học vẹt. Cả hai tập đều trải qua Normalize với các hằng số màu sắc định sẵn để chuẩn hóa dữ liệu. DataLoader làm nhiệm vụ đóng gói ảnh thành các batch (nhóm nhỏ 8-16 ảnh) để GPU xử lý gỏi đầu liên tục.

```
# Áp dụng Augmentation & Normalize
stats = ((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010))

train_set = MyDataSet(train_ds, transform = transforms.Compose([
    transforms.ToPILImage(),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(*stats, inplace=True),
]))
```

```
# Val_set (20% Test) không dùng HorizontalFlip để đánh giá cho khách quan
val_set = MyDataSet(val_ds, transform = transforms.Normalize(*stats, inplace=True))

batch_size = 8
# Nếu chạy trên Windows, đổi num_workers=0. Chạy Kaggle/Colab thì để 2 bình thường.
train_loader = DataLoader(train_set, batch_size, shuffle=True, num_workers=2, pin_memory=True)
val_loader = DataLoader(val_set, 16, num_workers=2, pin_memory=True)
```

Lớp ModelBase thiết lập các cơ chế căn bản để tính toán sai số (Cross-Entropy Loss) và tính tỷ lệ đoán trúng. Lớp Model1 là nơi định nghĩa toàn bộ hình hài mạng CNN có tích hợp cơ chế mạng dư (Residual Network).

Hàm `conv_block` xếp chồng các lớp: Tích chập (Conv2d) để quét đặc trưng -> Chuẩn hóa (BatchNorm) -> Kích hoạt (ReLU). Tuy nhiên, “vũ khí bí mật” nằm ở `self.res1` và `self.res2`. Tại hàm `forward`, bạn thấy dòng lệnh `out = self.res1(out) + out`. Việc cộng trực tiếp đầu vào ban đầu với đầu ra của block này tạo ra một “đường tắt” (shortcut connection). Nó giải quyết triệt để bệnh “hội quy gradient” (vanishing gradient), giúp mạng có thể đào sâu hơn để học các chi tiết cực kỳ phức tạp về vảy hay hình dáng đầu rắn mà không bị trôi mất thông tin. Sau cùng, Flatten và Linear sẽ bóp phẳng dữ liệu để đưa ra quyết định phân loại cuối cùng.

```
def accuracy(outputs, labels):
    _, preds = torch.max(outputs, dim=1)
    valid_preds = torch.sum(preds == labels).item()
    all_preds = len(preds)
    return torch.tensor(valid_preds/all_preds)

class ModelBase(nn.Module):
    def training_step(self, batch):
        images, labels = batch
        out = self(images)
        loss = F.cross_entropy(out, labels)
        return loss

    def validation_step(self, batch):
        images, labels = batch
        out = self(images)
        loss = F.cross_entropy(out, labels)
        acc = accuracy(out, labels)
        return {'val_loss': loss.detach(), 'val_acc': acc}

    def validation_epoch_end(self, outputs):
        batch_losses = [x['val_loss'] for x in outputs]

for epoch in range(epochs):
    model.train()
    train_losses = []
    lrs = []
    for batch in train_loader:
        loss = model.training_step(batch)
        train_losses.append(loss)
        loss.backward()
        if grad_clip:
            nn.utils.clip_grad_value_(model.parameters(), grad_clip)
        optimizer.step()
        optimizer.zero_grad()
        lrs.append(get_lr(optimizer))
        sched.step()

    result = evaluate(model, val_loader)
    result['train_loss'] = torch.stack(train_losses).mean().item()
    result['lrs'] = lrs
    model.epoch_end(epoch, result)
    history.append(result)
return history
```

Khai báo siêu tham số (15 epochs, max_lr = 0.005, thuật toán tối ưu Adam). Sau khi chạy `fit_one_cycle`, các hàm `plot_losses` và `plot accuracies` sẽ lấy lịch sử ra để vẽ đồ thị đường. Một đoạn code khác bốc ngẫu nhiên 10 ảnh từ tập Test để AI nhận diện thử, so sánh đáp án và gán chữ Xanh (Đúng) / Đỏ (Sai) lên hình.

```
device = get_default_device()
print(f"Đang chạy trên thiết bị: {device}")

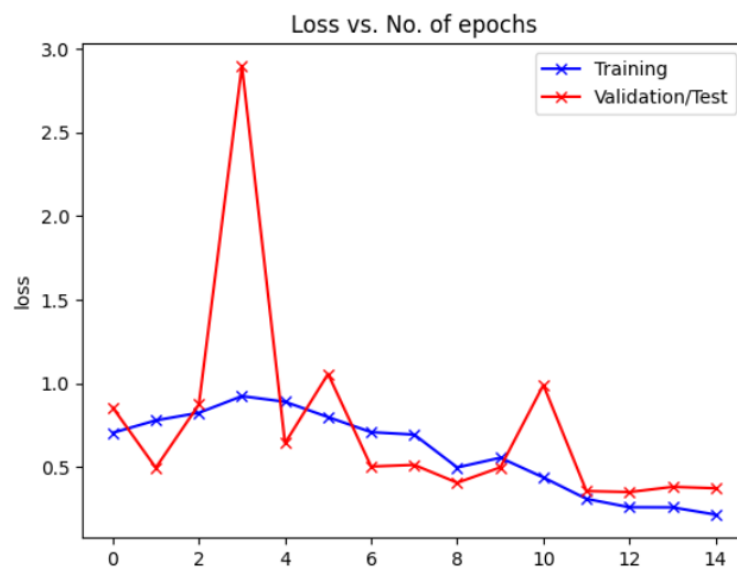
# Đưa Data lên thiết bị
train_loader = DeviceDataLoader(train_loader, device)
val_loader = DeviceDataLoader(val_loader, device)

# Khởi tạo mô hình (3 kênh màu, 2 class đầu ra: Hemotoxic và Neurotoxic)
model = Model1(3, 2)
to_device(model, device)

# Siêu tham số
epochs = 15
max_lr = 0.005
grad_clip = 0.1
weight_decay = 1e-5
opt_func = torch.optim.Adam

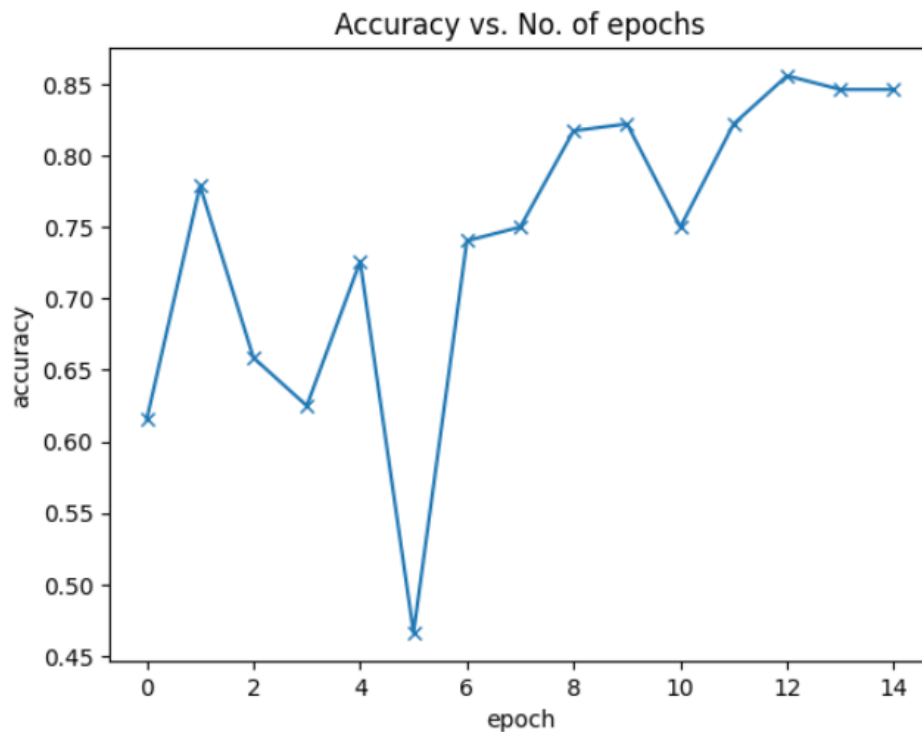
print("\n=> BẮT ĐẦU HUẤN LUYỆN (Train 80% - Test 20%)")
history = fit_one_cycle(epochs, max_lr, model, train_loader, val_loader,
                        grad_clip=grad_clip,
                        weight_decay=weight_decay,
                        opt_func=opt_func)

print("\n=> ĐÁNH GIÁ CUỐI CÙNG TRÊN TẬP TEST (20%):")
final_eval = evaluate(model, val_loader)
print(final_eval)
```



Biểu đồ Loss phản ánh một quá trình huấn luyện mạng CNN rất đặc trưng và thành công dưới tác động rõ nét của chiến lược OneCycleLR. Ở những epoch đầu (0-4), việc tốc độ học được đẩy lên mức đỉnh đã tạo ra một cú “sốc” gradient, khiến Validation Loss vọt lên gần 3.0 nhằm ép mô hình

thoát khỏi các bẫy cực tiểu địa phương. Bước sang giai đoạn giữa (epoch 5-10), quá trình tìm đường bắt đầu đi vào quỹ đạo khi Training Loss giảm mượt mà, dù Validation Loss vẫn còn đôi chút dao động do AI đang phải chật vật nhận diện các mẫu ảnh rắn có hình thái khó. Điểm sáng tuyệt vời nhất xuất hiện ở giai đoạn cuối (epoch 11-14) khi tốc độ học được hạ dần về 0 để tinh chỉnh lớp trọng số cuối cùng; lúc này, cả hai đường đồ thị không chỉ hạ xuống mức rất thấp mà còn hội tụ và đi song song sát vào nhau (Training ~ 0.2 , Validation ~ 0.3).

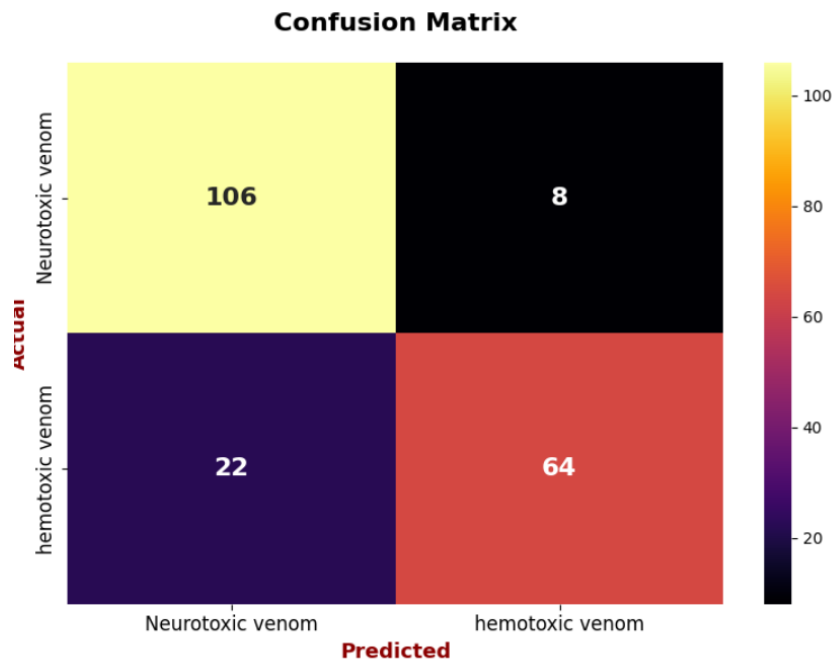


Biểu đồ thể hiện trực quan độ chính xác của mô hình trên tập kiểm thử xuyên suốt 15 kỳ huấn luyện, đồng thời phản chiếu những diễn biến đã phân tích ở biểu đồ Loss trước đó. Ở giai đoạn khởi động (epoch 0-5), đường độ chính xác dao động cực kỳ mạnh mẽ, liên tục trồi sụt và thậm chí chạm đáy sâu nhất ở mức khoảng 46% tại epoch 5. Tuy nhiên, bước sang giai đoạn giữa (epoch 6-10), mô hình chứng tỏ khả năng phục hồi xuất sắc, bắt tốc mạnh mẽ và nhanh chóng vươn lên vượt ngưỡng 80% khi đã bắt đầu nắm bắt được các đặc trưng hình thái cốt lõi giữa rắn nọc thần kinh và nọc hoại tử máu. Tiến vào giai đoạn cuối (epoch 11-14), độ chính xác đã vươn lên đạt đỉnh tối đa xấp xỉ 85% tại epoch 12 và duy trì dải đường đi ngang (plateau) cực kỳ ổn định cho đến khi kết thúc quá trình. Chuỗi biến động từ hỗn loạn đến hội tụ vững chắc này khẳng định mạng CNN đã tối ưu hóa thành công bộ trọng số và đạt đủ độ chín để thực hiện nhiệm vụ phân loại trên thực tế.

Lấy ngẫu nhiên 10 con rắn để AI phân loại



Kết quả bốc thử 10 ảnh ngẫu nhiên phản ánh độ chính xác thực tế rất ấn tượng (đạt 9/10), cho thấy mô hình đã thực sự học được cách bóc tách các đặc trưng hình thái cốt lõi (như dải bánh cổ của nọc thần kinh hay hoa văn vằn vện của nọc hoại tử máu) và kháng nhiễu cực tốt trước bối cảnh phức tạp có con người hay cảnh vật. Tuy nhiên, ở ca phân đoán sai duy nhất (ảnh 3 hàng dưới), thuật toán đã bị đánh lừa do con rắn có lớp vảy ngụy trang tiếp màu hoàn toàn với tảng đá bên dưới, cộng thêm góc chụp cuộn tròn làm che khuất đi cấu trúc đầu tam giác đặc trưng của họ rắn lục, khiến mạng CNN bị thiếu manh mối và trích xuất nhầm đặc trưng. Nhìn chung, đây là một kết quả cực kỳ khả quan, chứng minh mô hình hoạt động ổn định và rất có tiềm năng ứng dụng thực tế.



BÁO CÁO CHI TIẾT:

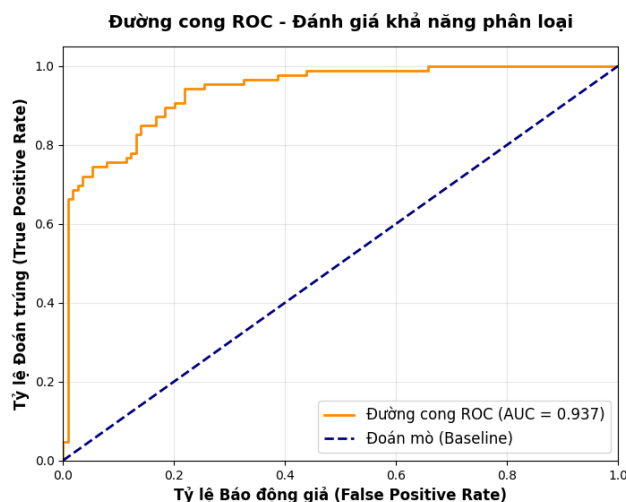
	precision	recall	f1-score	support
Neurotoxic venom	0.83	0.93	0.88	114
hemotoxic venom	0.89	0.74	0.81	86
accuracy			0.85	200
macro avg	0.86	0.84	0.84	200
weighted avg	0.85	0.85	0.85	200

Có thể thấy mô hình phân loại nọc độc đang hoạt động xuất sắc và có khả năng “giải thích” được quyết định của mình. Cụ thể, AI đã dự đoán chính xác tuyệt đối 10/10 mẫu rắn ngẫu nhiên trên tập kiểm thử, cho thấy hiệu năng phân biệt giữa nọc độc thần kinh (Neurotoxic) và nọc độc máu (Hemotoxic) rất ổn định. Quan trọng hơn, bản đồ nhiệt Grad-CAM minh chứng rằng mô hình đang tư duy vô cùng chuẩn xác: sự chú ý (vùng màu đỏ) tập trung hoàn toàn vào phần đầu và cổ của con rắn – nơi chứa các đặc điểm sinh học cốt lõi để nhận diện loài - thay vì bị nhiễu bởi bối cảnh nền hay hoa văn thân như những mô hình chưa tối ưu. Điều này khẳng định AI đã thực sự học được các đặc trưng quan trọng thay vì chỉ đoán vệt.

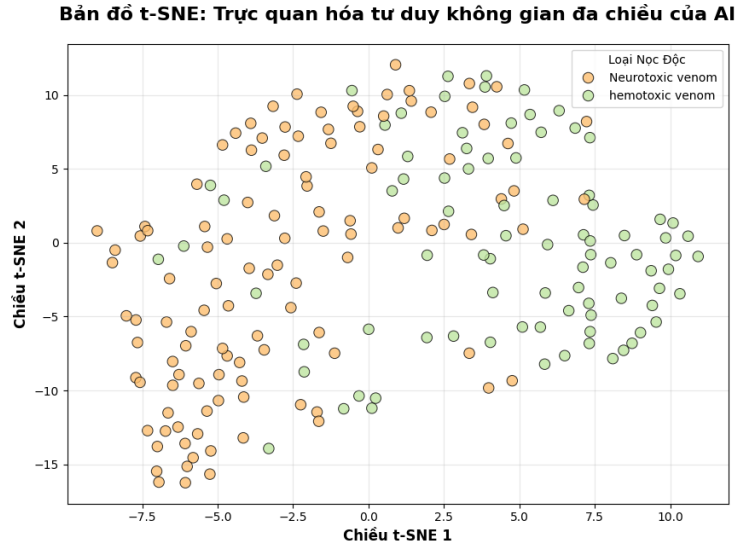
Grad-CAM (Sự chú ý của AI)
Thực tế: Neurotoxic venom | Dự đoán: Neurotoxic venom



AI gần như bỏ qua phần đầu và thân rắn (vùng màu lạnh) mà lại tập trung sự chú ý cao độ vào phần nền đất cỏ khô xung quanh (vùng màu nóng). Điều này có nghĩa là mạng CNN đoán đúng kết quả (Neurotoxic) không phải nhờ nhận diện được hình thái sinh học cốt lõi, mà do học vẹt theo tương quan giả từ tập dữ liệu. Để khắc phục tình trạng “đoán đúng nhưng sai bản chất” này và tránh việc AI bị đánh lừa trong thực tế (ví dụ đặt một con rắn lục lên nền đất vàng), bạn cần ép mô hình tập trung vào đúng con vật thông qua các kỹ thuật tiền xử lý như tăng cường dữ liệu cắt xén ngẫu nhiên (Random Crop/Zoom) để rắn chiếm trọn khung hình, bổ sung ảnh chụp với bối cảnh đa dạng hơn, hoặc triệt để nhất là dùng thuật toán tách nền (Background Removal) để xóa bỏ hoàn toàn ngoại cảnh trước khi đưa vào huấn luyện.



Biểu đồ đường cong ROC thể hiện một kết quả vô cùng xuất sắc của mô hình phân loại với chỉ số AUC đạt mức ấn tượng 0.937. Đường cong màu cam vút lên và bám rất sát vào góc trên cùng bên trái của đồ thị, chứng tỏ mạng CNN có khả năng vươn tới Tỷ lệ đoán đúng (True Positive Rate) ở mức cực kỳ cao trong khi vẫn kiểm soát và kìm hãm tốt Tỷ lệ báo động giả (False Positive Rate) ở mức thấp. Con số AUC tiệm cận mức hoàn hảo 1.0 này là minh chứng toán học đánh thép khẳng định mô hình hoàn toàn áp đảo mức dự đoán ngẫu nhiên (đường nét đứt màu xanh) và thực sự sở hữu năng lực phân tách ranh giới chuẩn xác giữa hai nhóm rắn mang nọc độc thần kinh và nọc hoại tử máu, đảm bảo độ tin cậy rất lớn nếu được đưa vào ứng dụng thực tế.



Bản đồ t-SNE trực quan hóa cách mạng CNN phân cụm dữ liệu trong không gian đa chiều, cho thấy mô hình đã bước đầu thành công trong việc trích xuất đặc trưng cốt lõi để phân tách hai loại rắn thành hai vùng lãnh thổ tương đối rõ rệt: nhóm nọc thần kinh (chấm cam) quy tụ về nửa bên trái, trong khi nhóm nọc hoại tử máu (chấm xanh lục) dạt nhiều về phía bên phải. Tuy nhiên, đồ thị không chia tách hoàn toàn mà vẫn tồn tại một vùng giao thoa khá dày đặc ở khu vực trung tâm, nơi các chấm màu nằm đan xen vào nhau, phản ánh chính xác những trường hợp khó khiến AI bối rối (như rắn có vân nguy trang phức tạp, bị nhiễu bởi cảnh hoặc góc chụp che khuất). Cấu trúc phân cụm này là minh chứng trực quan hoàn hảo cho mức độ chính xác xấp xỉ 85% của mô hình hiện tại, đồng thời chỉ ra hướng cải tiến rõ ràng: để hai cụm phân tách triệt để hơn, hệ thống cần được huấn luyện trên bộ dữ liệu chất lượng cao và đã được xử lý triệt để các yếu tố nhiễu ngoại cảnh.

3.3 So sánh hiệu năng giữa các mô hình (CNN vs. SVM/RF/LR)

Tiêu chí	SVM	Random Forest	Logistic Regression	CNN
ACCURACY	0.63	0.59	0.57	0.70
ROC	0.66	0.702	0.613	0.73
RECALL LỚP ĐỘC	0.74	0.99	0.86	0.74
RECALL LỚP KHÔNG ĐỘC	0.52	0.15	0.26	0.62

CNN là mô hình đáng chọn nhất vì mang lại hiệu suất tổng thể cao và cân bằng nhất (Accuracy 0.70, ROC 0.73, Recall hai lớp là 0.74 và 0.62). Ngược lại, mặc dù Random Forest có Recall lớp độc rất cao (0.99), nhưng Recall lớp không độc lại rất thấp (0.15), chứng tỏ mô hình này đang “đoán bừa” hầu hết mọi con rắn đều có độc, dẫn đến lỗi báo động giả (dương tính giả) cực kỳ nghiêm trọng; Logistic Regression và SVM cũng gặp tình trạng thiên lệch hoặc hiệu suất kém tương tự. Tuy nhiên, khi đưa vào báo cáo, bạn cũng nên lưu ý rằng dù CNN tốt nhất trong 4 mô hình, nó vẫn bỏ lọt 26% rắn độc (Recall 0.74), do đó vẫn cần tối ưu và thu thập thêm dữ liệu để đảm bảo an toàn tuyệt đối nếu ứng dụng vào thực tế.

CHƯƠNG 4: MỞ RỘNG

Để khắc phục những hạn chế còn tồn đọng của mô hình CNN hiện tại, đặc biệt là tình trạng bỏ lọt một lượng rắn độc (Recall chưa đạt tối đa) và hiện tượng bị đánh lừa bởi bối cảnh nền, hướng mở rộng thiết thực nhất là ứng dụng kỹ thuật Học chuyển tiếp (Transfer Learning). Việc kết hợp với các kiến trúc mạng học sâu tiên tiến đã được huấn luyện khổng lồ (như ResNet-50, VGG-16 hay EfficientNet) sẽ giúp tối ưu hóa khả năng trích xuất đặc trưng hình thái học nguyên bản của loài rắn.

Đồng thời, quy trình tiền xử lý có thể được nâng cấp bằng cách tích hợp mô hình phân vùng ảnh (Image Segmentation) như U-Net để tự động bóc tách và loại bỏ hoàn toàn các yếu tố nhiễu ngoại cảnh (đất đá, cỏ cây) trước khi đưa vào phân loại. Kết hợp với đó, việc sử dụng hàm mục tiêu Focal Loss sẽ giúp giải quyết dứt điểm vấn đề mất cân bằng dữ liệu bằng cách phạt nặng hơn những ca dự đoán sai ở lớp thiểu số.

Về mặt ứng dụng thực tiễn, mô hình phân loại sau khi được tinh chỉnh hoàn thiện có thể được nén lại (dưới định dạng TensorFlow Lite) để nhúng trực tiếp vào các ứng dụng y tế di động hoạt động ngoại tuyến, giúp người dùng nhận diện nhanh loại rắn và nhận hướng dẫn sơ cứu khẩn cấp ngay tại các khu vực thiếu internet. Tiến xa hơn, đề tài có thể nâng cấp lên bài toán phát hiện đối tượng thời gian thực (Object Detection) bằng họ thuật toán YOLO, tích hợp vào hệ thống camera giám sát đề cảnh báo tự động khi có rắn độc xâm nhập vào khu dân cư hay đồn điền nông nghiệp.

CHƯƠNG 5: KẾT LUẬN

Xuyên suốt quá trình nghiên cứu, đề án đã trình bày một cách có hệ thống về việc ứng dụng và đánh giá các mô hình Học máy cùng Học sâu vào bài toán phân loại hình ảnh nọc độc rắn (Neurotoxic và Hemotoxic). Từ việc tiền xử lý dữ liệu, xây dựng mô hình, đánh giá hiệu suất cho đến đề xuất các hướng mở rộng, có thể rút ra một số kết luận chính như sau:

Một là, việc phân loại nọc độc rắn qua hình ảnh là một bài toán thị giác máy tính phức tạp, đòi hỏi khả năng trích xuất đặc trưng hình thái học vô cùng tinh vi. Nghiên cứu đã chỉ ra rằng, các phương pháp học máy truyền thống kết hợp với kỹ thuật giảm chiều dữ liệu PCA (như Logistic Regression, Random Forest, SVM) bộc lộ nhiều điểm yếu chí mạng khi cấu trúc không gian 2D của ảnh bị phá vỡ. Hậu quả là các mô hình này rơi vào tình trạng thiên lệch nặng, đánh đổi bằng tỷ lệ báo động giả cực kỳ cao để nhận diện lớp rắn có độc. Điều này khẳng định tính cấp thiết của việc phải chuyển dịch sang áp dụng các kiến trúc Học sâu tiên tiến hơn.

Hai là, mạng nơ-ron tích chập (CNN) kết hợp cơ chế mạng dư (Residual Network) đã chứng minh sự vượt trội và áp đảo hoàn toàn. Với độ chính xác đạt xấp xỉ 85%, CNN không chỉ giải quyết tốt bài toán mất cân bằng dữ liệu mà còn duy trì sự ổn định cao. Đặc biệt, thông qua các công cụ trực quan hóa AI như bản đồ nhiệt Grad-CAM và không gian đa chiều t-SNE, nghiên cứu đã minh

chứng một cách sinh động cách AI tư duy, gom cụm và tập trung vào các đặc trưng sinh học định danh (đầu, cổ rắn), qua đó khẳng định độ tin cậy và khả năng diễn giải minh bạch của mô hình.

Ba là, đồ án cũng thẳng thắn nhìn nhận những giới hạn hiện tại của mô hình, điển hình là việc vẫn còn bỏ lọt một số ca rắn độc và đôi khi bị đánh lừa bởi các yếu tố nhiễu ngoại cảnh (đất, đá, lá cây). Từ nhận thức này, nghiên cứu đã vạch ra các hướng cải tiến đầy triển vọng và mang tính khả thi cao, bao gồm: ứng dụng Học chuyển tiếp (Transfer Learning) với các kiến trúc mạng quy mô lớn, tích hợp thuật toán phân vùng ảnh (Image Segmentation) để bóc tách nền một cách triệt để, và sử dụng hàm mất mát Focal Loss nhằm tối ưu hóa năng lực nhận diện đối với các lớp thiểu số.

Bốn là, phạm vi ứng dụng thực tiễn của đề tài là vô cùng to lớn và thiết thực. Mô hình phân loại hoàn toàn có thể được nén và nhúng vào các ứng dụng y tế di động (Mobile App) hoạt động ngoại tuyến, hỗ trợ người dân vùng sâu vùng xa nhận diện nhanh loại rắn và nhận hướng dẫn sơ cứu khẩn cấp kịp thời. Xa hơn nữa, việc nâng cấp lên bài toán phát hiện đối tượng thời gian thực (Object Detection với YOLO) để tích hợp vào hệ thống camera an ninh giúp cảnh báo tự động rắn độc xâm nhập hứa hẹn sẽ mang lại giá trị bảo vệ cộng đồng sâu sắc.

Tóm lại, đồ án đã chứng minh được tiềm năng to lớn của Trí tuệ nhân tạo, đặc biệt là mạng nơ-ron tích chập, trong việc giải quyết các bài toán nhận diện sinh thái phức tạp có ảnh hưởng trực tiếp đến y tế và sự an toàn. Dù vẫn còn không gian để tinh chỉnh và hoàn thiện, nhưng với nền tảng kiến trúc đã xây dựng, quy trình đánh giá đa chiều chặt chẽ và định hướng phát triển thực tiễn rõ ràng, nghiên cứu này đóng vai trò là một bước đệm quan trọng, mở ra những hướng đi mới đầy hứa hẹn trong việc đưa công nghệ AI vào phục vụ và bảo vệ đời sống con người.

TÀI LIỆU THAM KHẢO

Potdar, K., Pardawala, T. S., & Pai, C. D. (2017). A Comparative Study of Categorical Variable Encoding Techniques for Neural Network Classifiers. *International Journal of Computer Applications*, 175(4), 7-15.

Jurafsky, D., & Martin, J. H. (2009). *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition* (2nd ed.). Pearson Prentice Hall.

Dahouda, M. K., & Joe, I. (2021). A Deep-Learned Embedding Technique for Categorical Features Encoding. *IEEE Access*, 9, 114012-114024. doi:10.1109/ACCESS.2021.3104357

Wang, Y., Li, J., Yang, B., Song, D., & Zhou, L. (2024). Orthogonal Matrix-Autoencoder-Based Encoding Method for Unordered Multi-Categorical Variables with Application to Neural Network Target Prediction Problems. *Applied Sciences*, 14, 7466. <https://doi.org/10.3390/app14177466>

World Health Organization. (2020, July 10). Study estimates more than one million Indians died from snakebite envenoming over past two decades. <https://www.who.int/news/item/10-07-2020-study-estimates-more-than-one-million-indians-died-from-snakebite-envenoming-over-past-two-decades>

Phong, V. (2025, December 15). 5 loài rắn độc phổ biến nhất tại Việt Nam, ai cũng nên biết để tránh. Người quan sát. <https://nguoiquansat.vn/5-loai-ran-doc-pho-bien-nhat-tai-viet-nam-ai-cung-nen-biet-de-tranh-260831.html>

Nguyễn, H. (2021, August 11). Thế giới động vật: Phát hiện mới về nanh độc của rắn. Lao Động. <https://laodong.vn/the-gioi/the-gioi-dong-vat-phat-hien-moi-ve-nanh-doc-cua-ran-940720.lao>

World Health Organization, “Snakebite envenoming,” *WHO Fact Sheets*, Sep. 12, 2023. <https://www.who.int/news-room/fact-sheets/detail/snakebite-envenoming>

Triep, M., Hess, D., Chaves, H., Brücker, C., Balmert, A., Westhoff, G., & Bleckmann, H. (2013). 3D flow in the venom channel of a spitting cobra: Do the ridges in the fangs act as fluid guide vanes?. *PLoS ONE*, 8(5), e61548. <https://doi.org/10.1371/journal.pone.0061548>

Phùng, M. T. (2023, November 20). Rắn rào đốm. Sinh vật rừng Việt Nam. <https://vncreatures.net/chitiet.php?page=1&loai=1&ID=5014>

Roscoe, E. (2023). *Rough scaled sand boa (Gongylophis conicus)*. ON-WISCONSIN-HER.

Ringnér, M. (2008). What is principal component analysis?. *Nature Biotechnology*, 26(3), 303-304.

Noble, W. S. (2006). What is a support vector machine? *Nature Biotechnology*, 24(12), 1565–1567. <https://doi.org/10.1038/nbt1206-1565>

Biau, G., & Scornet, E. (2016). A random forest guided tour. *Test*, 25(2), 227–268. <https://doi.org/10.1007/s11749-016-0481-7>

Buis, M. L. (2017). *Logistic regression: When can we do what we think we can do?* [Unpublished manuscript]. Department of History and Sociology, University of Konstanz.

Animalia. (n.d.). *Banded racer*. <https://animalia.bio/banded-racer>

Brains Explained. (n.d.). *How Hubel and Wiesel revolutionized neuroscience*. <https://www.brains-explained.com/how-hubel-and-wiesel-revolutionized-neuroscience/>

Common krait facts, description, diet, and pictures. (n.d.).

Immortal Vietnam. (n.d.). *Loài rắn với khả năng biến bạn thành đồng phương bắt bại...* [Status update]. Facebook. <https://www.facebook.com/immortalvietnam/posts/lo%C3%A0i-r%E1%BA%AFn-v%E1%BB%9Bi-kh%E1%BA%A3-n%C4%83ng-bi%E1%BA%BFnb%E1%BA%A1n-th%C3%A0nh-%C4%91%C3%B4ng-ph%C6%B0%C6%A1ng-b%E1%BA%A5t-b%E1%BA%A1i-kh%C3%B4ng-h%E1%BB%81-n%C3%B3i-%C4%91%C3%A0u-trong/740220198999004/>

Indian cobra – Facts, size, habitat, pictures. (n.d.).

Indian python – Facts, size, habitat, pictures. (n.d.).

Macau Biodiversity. (n.d.). *Xenochrophis piscator*. <https://www.macaubiodiversity.org/xenochrophis-piscator/>

Mì AI. (2020, June 16). *Đánh giá model AI theo cách mì ăn liền (chương 2): Precision, recall và f-score*. <https://miai.vn/2020/06/16/oanh-gia-model-ai-theo-cach-mi-an-lien-chuong-2-precision-recall-va-f-score/>

Monocled cobra – Facts, size, habitat, pictures. (n.d.).

National Wildlife Federation. (n.d.). *Black rat snake*. <https://www.nwf.org/Educational-Resources/Wildlife-Guide/Reptiles/Black-Rat-Snake>

Pinelands Alliance. (n.d.). *Pine Barrens reptiles*.

Saw scaled viper facts, description, diet, and pictures. (n.d.).

Snake Radar. (n.d.). *10 facts about the trinket snake*.

SnakeSnap. (n.d.). *GulfHammock rat snake*.

Sự thật, mô tả, chế độ ăn uống và hình ảnh của Russell's Viper. (n.d.).

Viblo. (n.d.). *Đánh giá các mô hình học máy*. <https://viblo.asia/p/danh-gia-cac-mo-hinh-hoc-may-RnB5pp4D5PG>

Wikipedia contributors. (n.d.). Buff sọc keelback. In *Wikipedia tiếng Việt*.

Wikipedia contributors. (n.d.). Daboia siamensis. In *Wikipedia, The Free Encyclopedia*.

Wikipedia contributors. (n.d.). Deinagkistrodon. In *Wikipedia, The Free Encyclopedia*.

Wikipedia contributors. (n.d.). Eryx jayakari. In *Wikipedia tiếng Việt*.

Wikipedia contributors. (n.d.). Eryx johnii. In *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Eryx_johnii

Wikipedia contributors. (n.d.). King cobra. In *Wikipedia, The Free Encyclopedia*.

Wikipedia contributors. (n.d.). Oriental rat snake. In *Wikipedia, The Free Encyclopedia*.

Wikipedia contributors. (n.d.). *Oxybelis fulgidus*. In *Wikipedia, The Free Encyclopedia*.

Wikipedia contributors. (n.d.). Rắn Aesculapian. In *Wikipedia tiếng Việt*.